Introduction to AI

Domain-independent planning

Alex Shleyfman and Gal A. Kaminka

Dept. of Computer Science, Bar-Ilan University

Winter 2026/2027

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Work Plan

1. ~~Introduction~~ ~~Uninformed Search~~ (14.10)
2. ~~Heuristics Search~~ (21.10)
3. Domain-Independent Heuristics (28.10)
4. Inference CSP/SAT (04.11) Ex. 1 published
5. Bayesian inference (11.11)
6. Markov Decision Processes (18.11)
7. Multi-Armed Bandits (25.11) Ex. 2 published
8. Reinforcement Learning (02.12)
9. Logic Inference (09.12)
10. Games (16.12) Ex. 3 published
11. Multi-Agent Systems (23.12)
12. MARL (30.12)
13. Plan-Space Planning (06.12)
14. Rehearsal (13.01)

Reminder: Are we solver?

General idea

(Admissible) heuristic functions obtained as (optimal) cost functions of relaxed problems

- OK, but heuristic is yet another input to our agent!
- Satisfactory for general solvers?
- Satisfactory in special purpose solvers?

Towards domain-independent agents

- How to get heuristics automatically?
- Can such automatically derived heuristics dominate the domain-specific heuristics crafted by hand?

Model-based Problem Solving

We want planning to take the form of model-based solving

$$\texttt{Problem} \Longrightarrow \texttt{Language} \Longrightarrow \texttt{Planner} \Longrightarrow \texttt{Solution}$$

1. **models** for defining, classifying, and understanding problems
   - what is a *planning problem*
   - what is a *solution* (*plan*), and
   - what is an *optimal solution*
2. **languages** for representing problems
3. **algorithms** for solving them

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
State variables
Tasks
Action Languages
STRIPS language
Obtaining heuristics
Concrete Heuristics

Succinct representation of transition systems

- More compact representation of actions than as relations is often
  - possible because of symmetries and other regularities,
  - unavoidable because the relations are too big.

- Represent different aspects of the world in terms of different state variables.
  $\rightsquigarrow$ A state is a valuation of state variables.

- Represent actions in terms of changes to the state variables.

State variables – Finite Domain Representation (FDR)

- The state of the world is described in terms of a finite set of finite-valued state variables.

Example

hour: $\{0, \ldots, 23\} = 11$
minute: $\{0, \ldots, 59\} = 30$
location: $\{112, 214, 215, 216\} = 112$
weather: $\{\text{sunny}, \text{cloudy}, \text{rainy}\} = \text{sunny}$
holiday: $\{\text{T}, \text{F}\} = \text{F}$

- Actions change the values of the state variables.

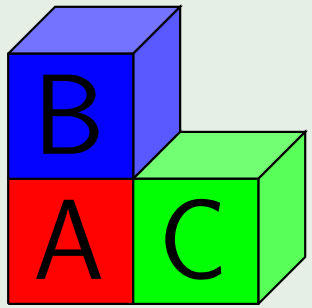
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
State variables
Tasks
Action Languages
STRIPS language
Obtaining heuristics
Concrete Heuristics

Blocks world with state variables – Finite Domain Representation (FDR)

State variables:
location-of-A: $\{\mathrm{B}, \mathrm{C}, \mathrm{table}\}$
location-of-B: $\{\mathrm{A}, \mathrm{C}, \mathrm{table}\}$
location-of-C: $\{\mathrm{A}, \mathrm{B}, \mathrm{table}\}$

Example

$$s(\textit{location-of-A}) = \mathrm{table}$$
$$s(\textit{location-of-B}) = \mathrm{A}$$
$$s(\textit{location-of-C}) = \mathrm{table}$$



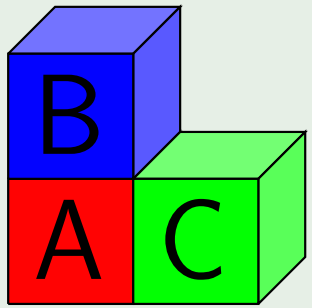
Not all valuations correspond to an intended blocks world state, e. g. s such that $s(\textit{location-of-A}) = \mathrm{B}$ and $s(\textit{location-of-B}) = \mathrm{A}$.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
State variables
Tasks
Action Languages
STRIPS language
Obtaining heuristics
Concrete Heuristics

Blocks world with Boolean state variables

Example

$$s(\textit{A-on-B}) = 0$$
$$s(\textit{A-on-C}) = 0$$
$$s(\textit{A-on-table}) = 1$$
$$s(\textit{B-on-A}) = 1$$
$$s(\textit{B-on-C}) = 0$$
$$s(\textit{B-on-table}) = 0$$
$$s(\textit{C-on-A}) = 0$$
$$s(\textit{C-on-B}) = 0$$
$$s(\textit{C-on-table}) = 1$$



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
State variables
Tasks
Action Languages
STRIPS language
Obtaining heuristics
Concrete Heuristics

Deterministic planning tasks

Definition (deterministic planning task)

A deterministic planning task is a 4-tuple $\Pi = \langle V, I, A, G \rangle$

- V is a finite set of state variables,
- I is an initial state over V,
- A is a finite set of actions over V, and
- G is a *formula* over V describing the goal states.
  - in what follows: partial assignment to V

$\rightsquigarrow$ Mapping planning tasks to transition systems

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
State variables
Tasks
Action Languages
STRIPS language
Obtaining heuristics
Concrete Heuristics

Planning Languages

Key issue

Models represented implicitly in a declarative language

Play two roles

- specification: concise model description
- computation: reveal useful info about problem's *structure*

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
State variables
Tasks
Action Languages
STRIPS language
Obtaining heuristics
Concrete Heuristics

Why STRIPS is interesting

- STRIPS operators are particularly simple, yet expressive enough to capture general planning problems.
- In particular, STRIPS planning is no easier than general planning problems.
- Most algorithms in the planning literature are only presented for STRIPS operators (generalization is often, but not always, obvious).

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

Real World: Problem to solve

STRIPS

STanford **R**esearch **I**nstitute **P**roblem **S**olver – is an automated planner developed by Richard Fikes and Nils Nilsson in 1971 at SRI International. The same name was later used to refer to the formal language of the inputs to this planner.

This language includes:

- World description (State space)
- Legal moves (Actions)
- Initial position (Initial state)
- Desired outcome (Goal states)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

STRIPS Encoding

Planning task in STRIPS is a quadruple $\langle P, A, I, G \rangle$:

- P: finite set of predicates
- A: finite set of actions of a form $\langle pre, add, del \rangle$

  (preconditions/add effects/delete effects: subsets of predicates)
- I: initial state

  (subset of predicates, others assumed to be false)
- G: goal description

  (subset of predicates)

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

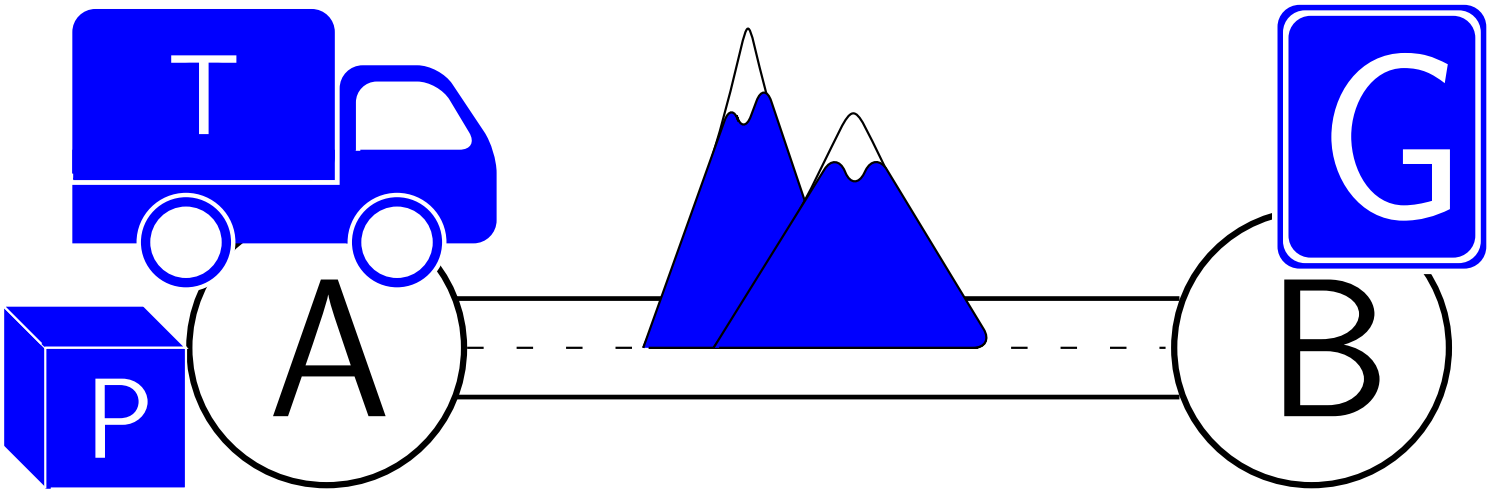
STRIPS language

Example

Obtaining heuristics

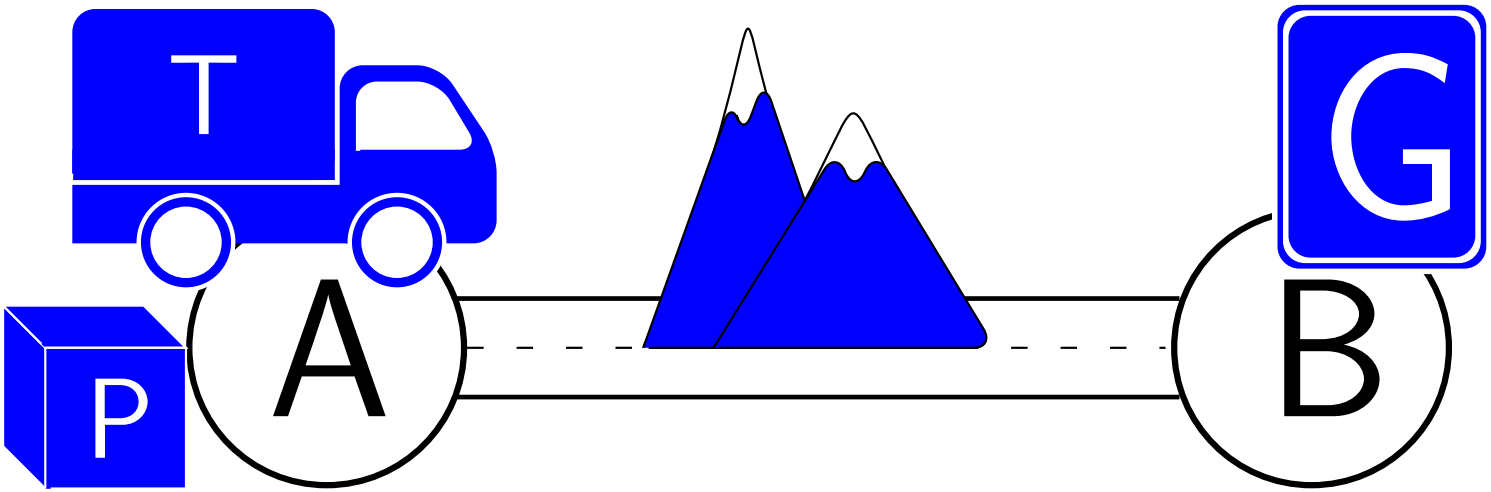
Concrete Heuristics

STRIPS Example



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

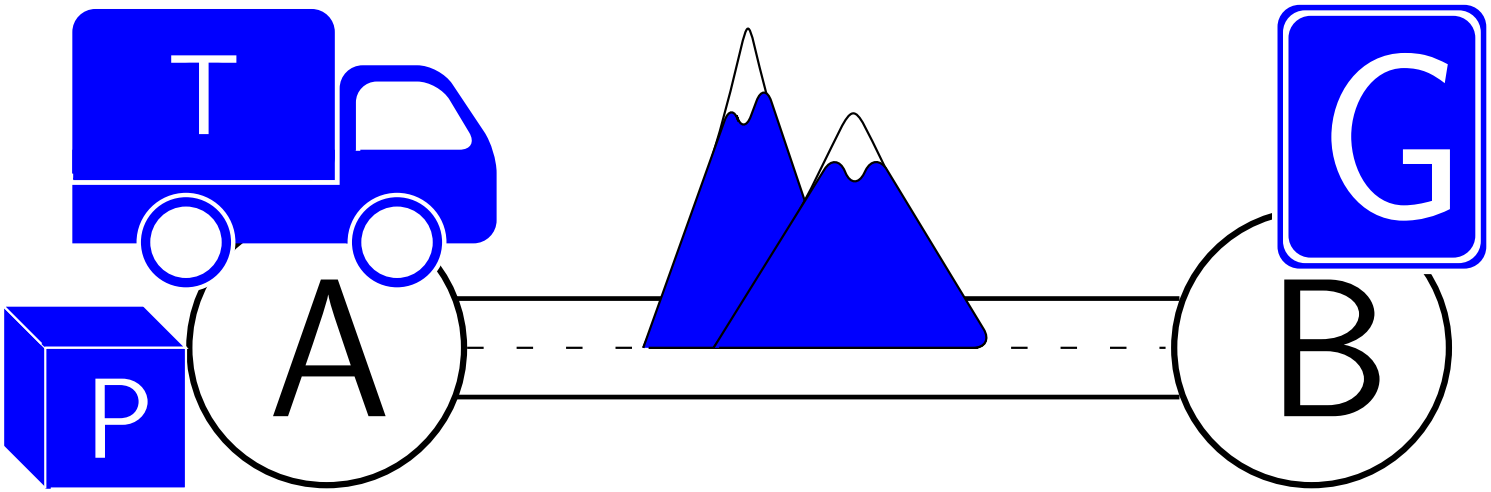
STRIPS Example



P: set of binary predicates that can be either True or False

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

STRIPS Example



P: set of binary predicates that can be either True or False

Predicates for Package P_A P_B P_T Predicates for Track T_A T_B

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

STRIPS Example

A: set of actions:

STRIPS Example

A: set of actions:

DriveAB | LoadA | UnloadA

DriveBA | LoadB | UnloadB

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Example

Obtaining heuristics

Concrete Heuristics

STRIPS Example

A: set of actions:

DriveAB	LoadA	UnloadA
DriveBA	LoadB	UnloadB

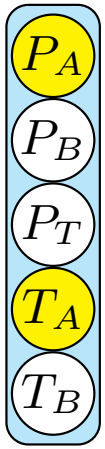
Example of action presentation:

$$\text{LoadA} = \begin{cases} pre = \{P_A, T_A\} \\ add = \{P_T\} \\ del = \{P_A\} \end{cases}, \text{DriveAB} = \begin{cases} pre = \{T_A\} \\ add = \{T_B\} \\ del = \{T_A\} \end{cases}$$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

STRIPS Example

I: initial state



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

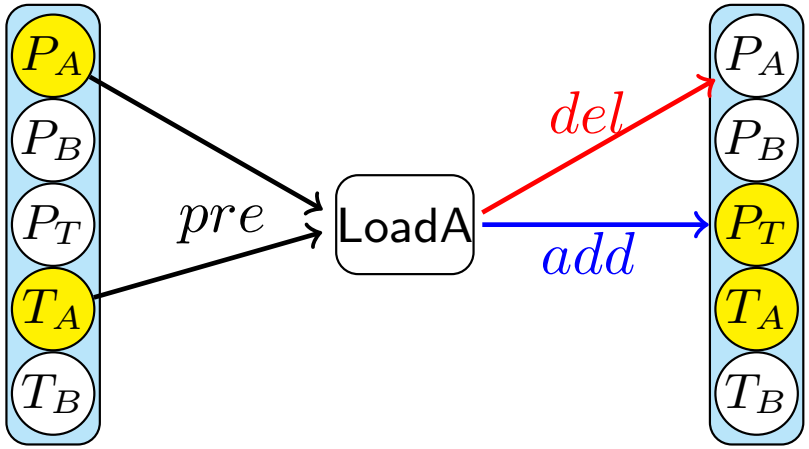
Example

Obtaining heuristics

Concrete Heuristics

STRIPS Example

I: initial state

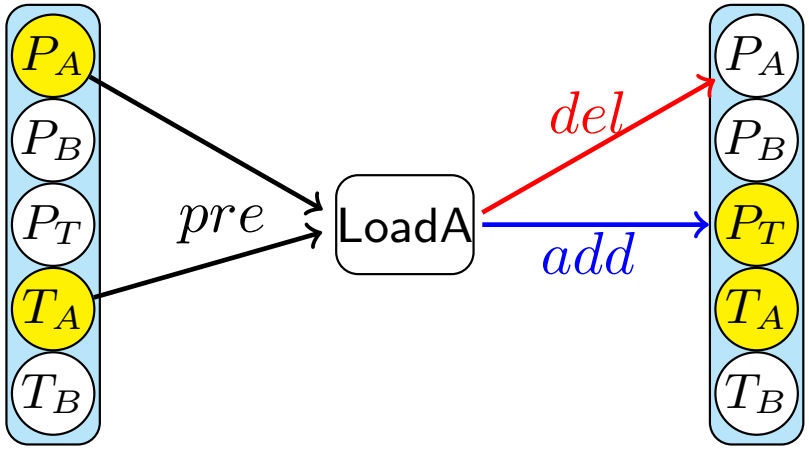


Example of application of LoadA

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

STRIPS Example

I: initial state



Example of application of LoadA

G: goal state

– each state that contains the predicate P_B

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Example

Obtaining heuristics

Concrete Heuristics

The STRIPS language

A problem in STRIPS is a tuple $\langle P, A, I, G \rangle$

- P stands for a finite set of **atoms** (boolean vars)
- $I \subseteq P$ stands for **initial situation**
- $G \subseteq P$ stands for **goal situation**
- A is a finite set of **actions** a specified via $pre(a)$, $add(a)$, and $del(a)$, all subsets of P

- States are collections of atoms
- An action a is applicable in a state s iff $pre(a) \subseteq s$
- Applying an applicable action a at s results in $s' = (s \setminus del(a)) \cup add(a)$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Example
Obtaining heuristics
Concrete Heuristics

A simple heuristic for deterministic planning

STRIPS (Fikes & Nilsson, 1971) used the number of state variables that differ in current state s and a STRIPS goal $G = \{g_1, \ldots, g_k\}$:

$$h(s) := |G \setminus s|.$$

Intuition: more true goal literals $\rightsquigarrow$ closer to the goal

$\rightsquigarrow$ STRIPS heuristic (properties?)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Criticism of the STRIPS heuristic

What is wrong with the STRIPS heuristic?

- quite uninformative:
  the range of heuristic values in a given task is small;
  typically, most successors have the same estimate
- very sensitive to reformulation:
  can easily transform any planning task into an equivalent one where $h(s) = 1$ for all non-goal states (how?)
- ignores almost all problem structure:
  heuristic value does not depend on the set of actions!

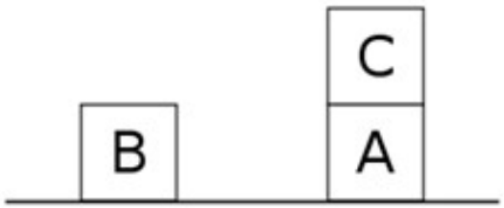
$\rightsquigarrow$ need a better, principled way of coming up with heuristics

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

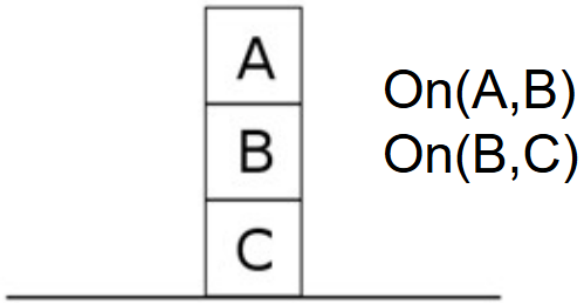
Sussman Anomaly

Challenges of planning: “Sussman anomaly”

Start state:

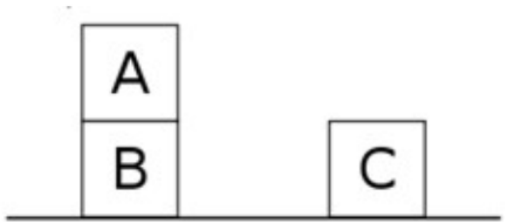


Goal state:

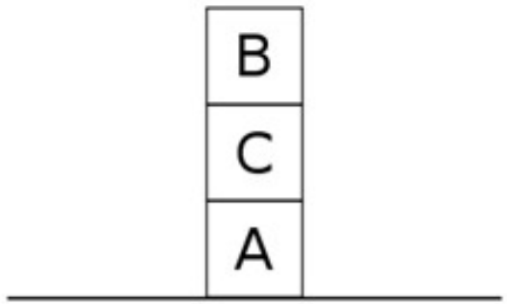


On(A,B)
On(B,C)

Let’s try to achieve On(A,B):



Let’s try to achieve On(B,C):



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

STRIPS heuristic

Relaxation heuristics

The relaxation lemma

Greedy algorithm

Optimality

Discussion

Concrete Heuristics

ChatGPT vs. Sussman Anomaly

Suppose you have three blocks on the table that you can move. A and B lie on the table. and C lies on A. Give me a sequence of actions that will build the tower A ontop of B ontop of C optimally.

To build the tower with A on top of B and C on top of A, here is an optimal sequence of actions:

1. **Move C from A to B** (this places C on top of B).
2. **Move A from the table to C** (this places A on top of C).

Now you have the tower: A on top of C, and C on top of B, achieving the desired configuration optimally.

< 2/2 >

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

STRIPS heuristic

Relaxation heuristics

The relaxation lemma

Greedy algorithm

Optimality

Discussion

Concrete Heuristics

Coming up with heuristics in a principled way

General procedure for obtaining a heuristic

Solve an easier version of the problem.

Two common methods:

- relaxation: consider less constrained version of the problem
- abstraction: consider smaller version of real problem

Both have been very successfully applied in planning (separately and *together*).
We consider relaxation.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Relaxing a problem

How do we relax a problem?

Example (Route planning for a road network)

The road network is formalized as a weighted graph over points in the Euclidean plane. The weight of an edge is the road distance between two locations.

A relaxation drops constraints of the original problem.

Example (Relaxation for route planning)

Use the Euclidean distance $\sqrt{|x_1 - y_1|^2 + |x_2 - y_2|^2}$ as a heuristic for the road distance between (x_1, x_2) and (y_1, y_2)
This is a lower bound on the road distance ($\rightsquigarrow$ admissible).

$\rightsquigarrow$ We drop the constraint of having to travel on roads.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Relaxations for planning

- Relaxation is a general technique for heuristic design:
  - Straight-line heuristic (route planning): Ignore the fact that one must stay on roads.
  - Manhattan heuristic (15-puzzle): Ignore the fact that one cannot move through occupied tiles.
- We want to apply the idea of relaxations to planning.
- Informally, we want to ignore bad side effects of applying actions.

Example (8-puzzle)

If we move a tile from x to y, then the good effect is (in particular) that x is now free.
The bad effect is that y is not free anymore, preventing us for moving tiles through it.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Relaxed planning tasks: idea

In STRIPS, good and bad effects are easy to distinguish:

- Effects that make atoms true are good (add effects).
- Effects that make atoms false are bad (delete effects).

Idea for the heuristic: Ignore all delete effects.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Relaxed planning tasks: idea

In STRIPS, good and bad effects are easy to distinguish:

- Effects that make atoms true are good (add effects).
- Effects that make atoms false are bad (delete effects).

Idea for the heuristic: Ignore all delete effects.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Relaxed planning tasks

Definition (relaxation of actions)

The relaxation a^+ of a STRIPS action $a = \langle pre(a), add(a), del(a) \rangle$ is the action $a^+ = \langle pre(a), add(a), \emptyset \rangle$.

Definition (relaxation of planning tasks)

The relaxation Π^+ of a STRIPS planning task $\Pi = \langle P, A, I, G \rangle$ is the planning task $\Pi^+ := \langle P, \{a^+ \mid a \in A\}, I, G \rangle$.

Definition (relaxation of action sequences)

The relaxation of an action sequence $\pi = a_1 \ldots a_n$ is the action sequence $\pi^+ := {a_1}^+ \ldots {a_n}^+$.

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

STRIPS heuristic

Relaxation heuristics

The relaxation lemma

Greedy algorithm

Optimality

Discussion

Concrete Heuristics

Relaxed planning tasks: terminology

- STRIPS planning tasks without delete effects are called relaxed planning tasks.
- Plans for relaxed planning tasks are called relaxed plans.
- If Π is a STRIPS planning task and π^+ is a plan for Π^+, then π^+ is called a relaxed plan for Π.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Example: delete relaxation intuition

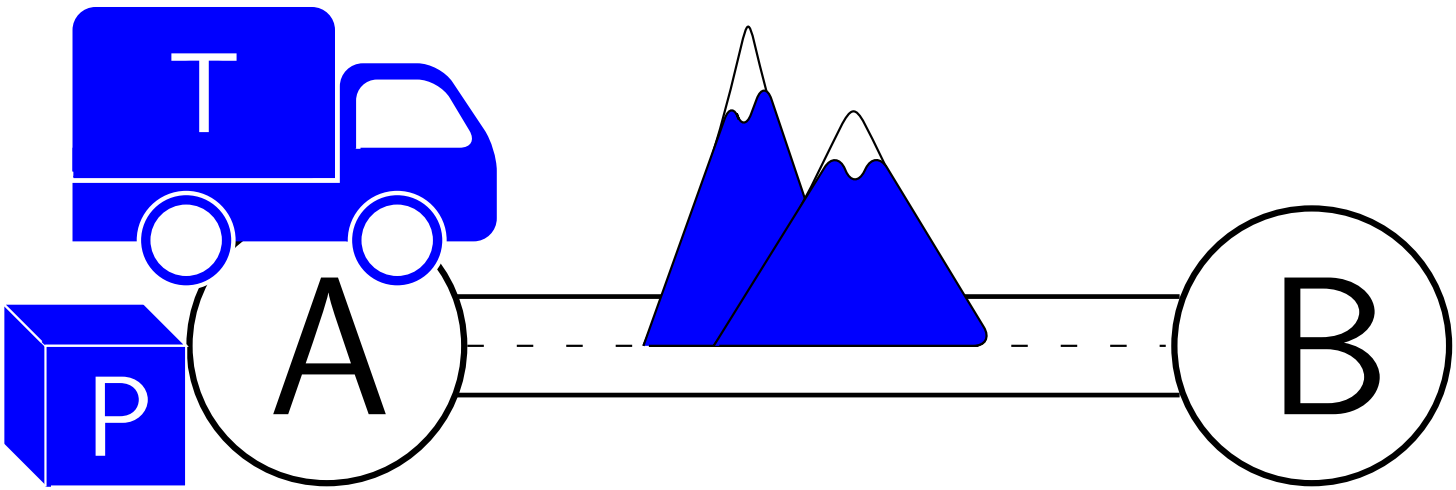


Figure: Applying DriveAB$^+$ to the initial state I.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Example: delete relaxation intuition

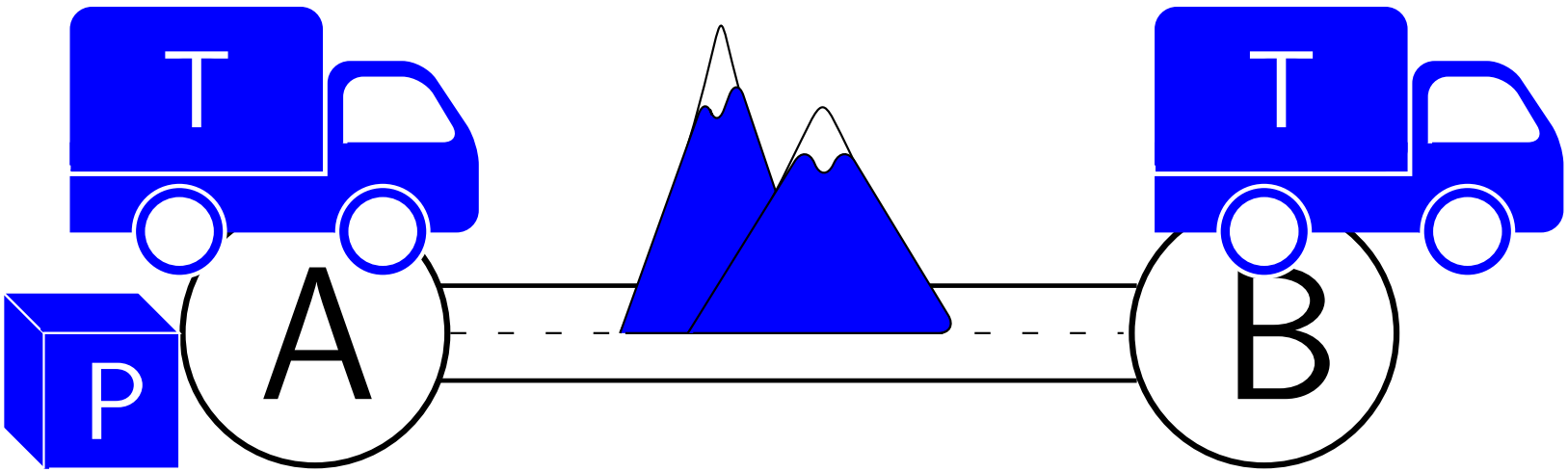
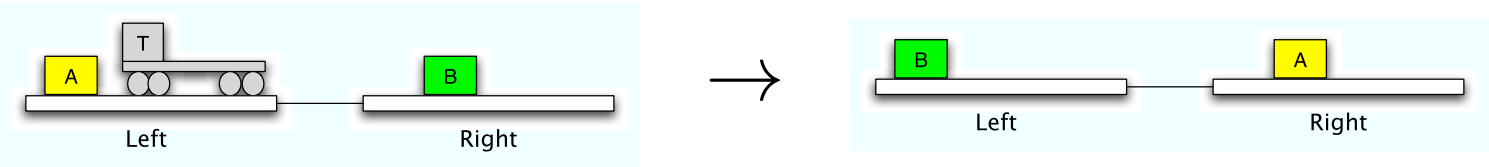


Figure: Applying DriveAB$^+$ to the initial state I.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

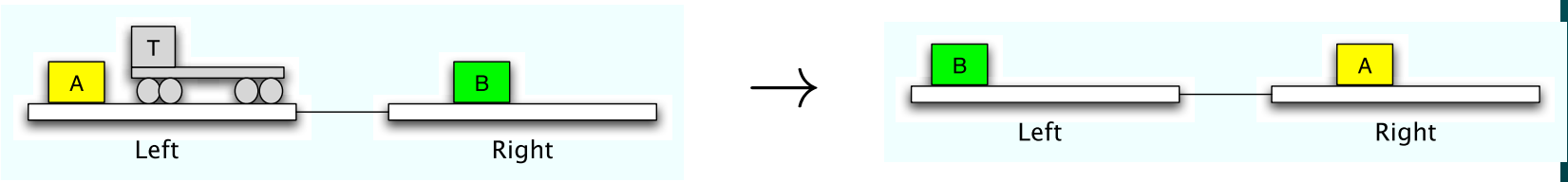
Example: Logistics



- Initial state I: $\{at(A, Left), at(T, Left), at(B, Right)\}$
- $f\left(I, Drive(Left, Right)\right) =$
  $\{at(A, Left), at(T, Right), at(B, Right)\}$
- $f\left(I, Drive(Left, Right)^{+}\right) =$
  $\{at(A, Left), at(T, Left), at(T, Right), at(B, Right)\}$
- $f\left(I, \langle Drive(Left, Right), Load(A, Left)\rangle\right)$ is undefined
- $f\left(I, \langle Drive(Left, Right)^{+}, Load(A, Left)^{+}\rangle\right) =$
  $\{at(A, Left), at(T, Left), at(T, Right), at(B, Right), in(A, T)\}$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Example: Logistics

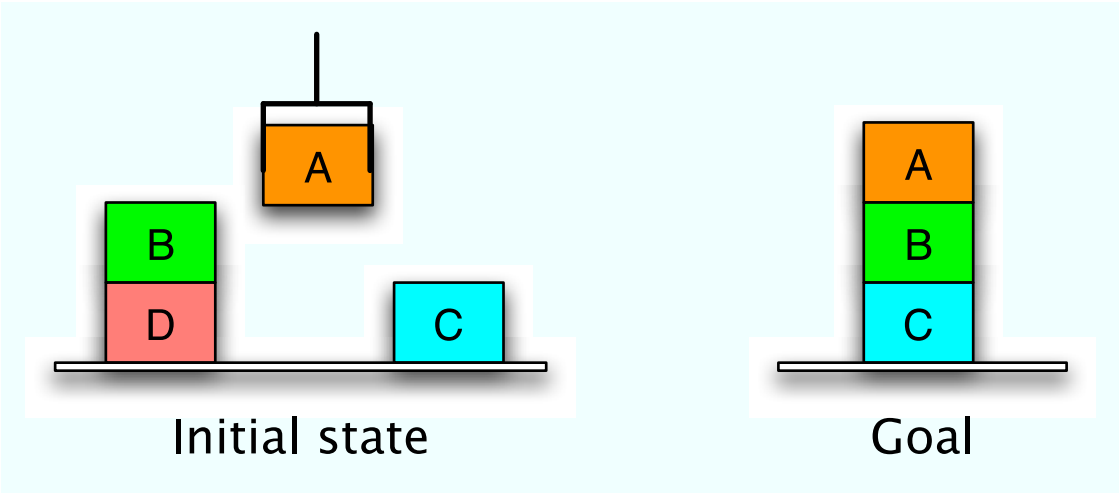


- Optimal plan:
  1. $load(A, T, Left)$,
  2. $drive(Left, Right)$,
  3. $unload(A, T, Right)$,
  4. $load(B, T, Right)$,
  5. $drive(Right, Left)$,
  6. $unload(B, T, Left)\}$
- Optimal relaxed plan: ??? (subsequence of the optimal plan)
- $h^*(I) = 6$, $h^+(I) =$???

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Always subsequence? (Just curious)

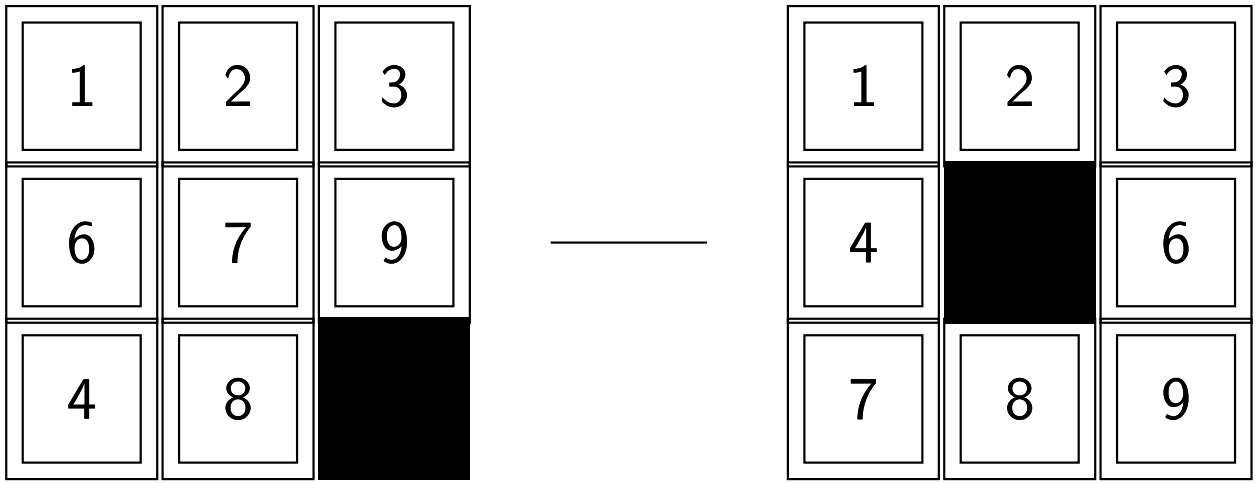
An optimal relaxed plan can *not* always be obtained by skipping actions from the (real) optimal plan.



- Optimal plan:
  $\langle putdown(A), unstack(B, D), stack(B, C), pickup(A), stack(A, B)\rangle$
- Optimal relaxed subsequence: ???
- Optimal relaxed plan: ???

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

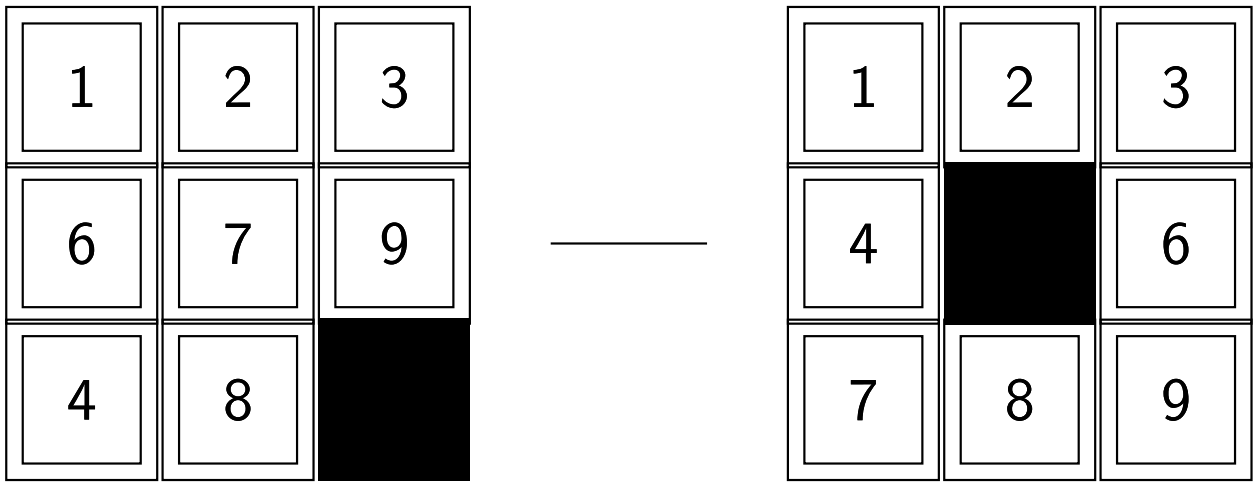
Example: 8-Puzzle



- Real problem:
  - A tile can move from square A to square B if A is adjacent to B and B is blank
- Monotonically relaxed problem:
  - A tile can move from square A to square B if A is adjacent to B and B is blank (!!!)
  - In effect ...

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Example: 8-Puzzle

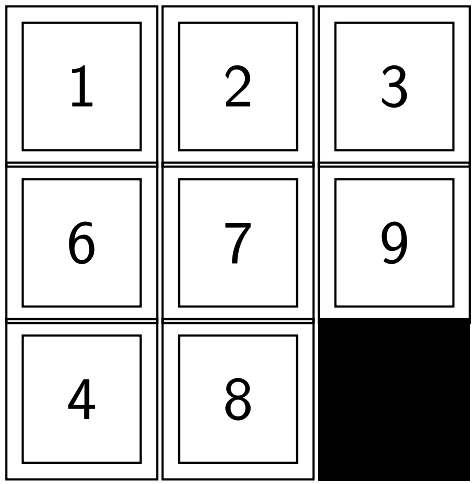


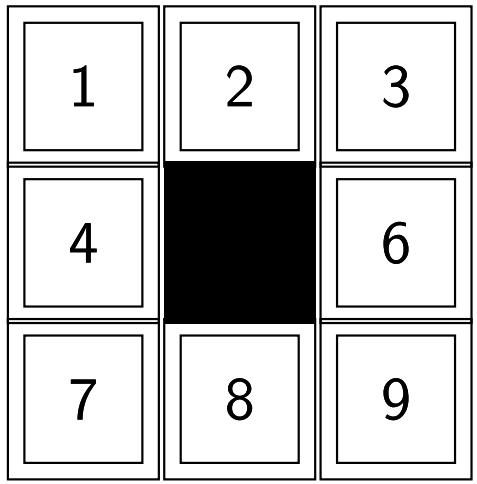
- A tile can move from square A to square B if A is adjacent to B and B is blank - solution distance h^*
- A tile can move from square A to square B if A is adjacent to B - manhattan distance heuristic h^{MD}
- A tile can move from square A to square B if A is adjacent to B and B is blank; in effect, the tile is at both A and B, and both A and B are blank - h^+

Here: $h^*(s_0) = 8$, $h^{MD}(s_0) = 6$, $h^+(s_0) = ???$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Example: 8-Puzzle





Optimal MD plan:

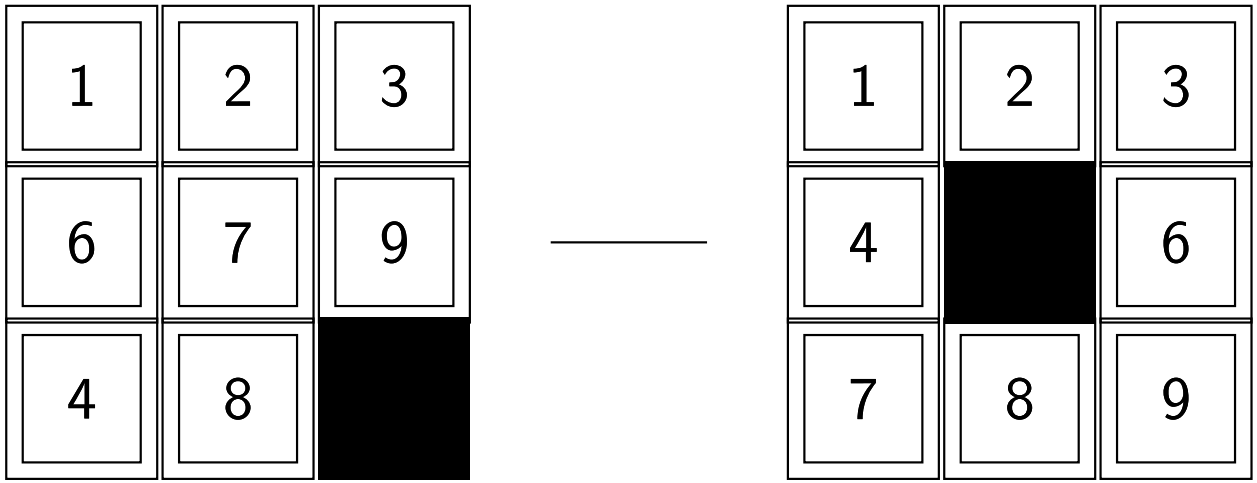
1. $move(t_9, p_6, p_9)$
2. $move(t_7, p_5, p_8)$
3. $move(t_6, p_4, p_5)$
4. $move(t_6, p_5, p_6)$
5. $move(t_4, p_7, p_4)$
6. $move(t_7, p_8, p_7)$

Optimal relaxed plan:

1. $move(t_9, p_6, p_9)$
2. $move(t_8, p_8, p_9)$
3. $move(t_7, p_5, p_8)$
4. $move(t_6, p_4, p_5)$
5. $move(t_6, p_5, p_6)$
6. $move(t_4, p_7, p_4)$
7. $move(t_7, p_8, p_7)$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Example: 8-Puzzle



Optimal MD plan:

1. $move(t_9, p_6, p_9)$
2. $move(t_7, p_5, p_8)$
3. $move(t_6, p_4, p_5)$
4. $move(t_6, p_5, p_6)$
5. $move(t_4, p_7, p_4)$
6. $move(t_7, p_8, p_7)$

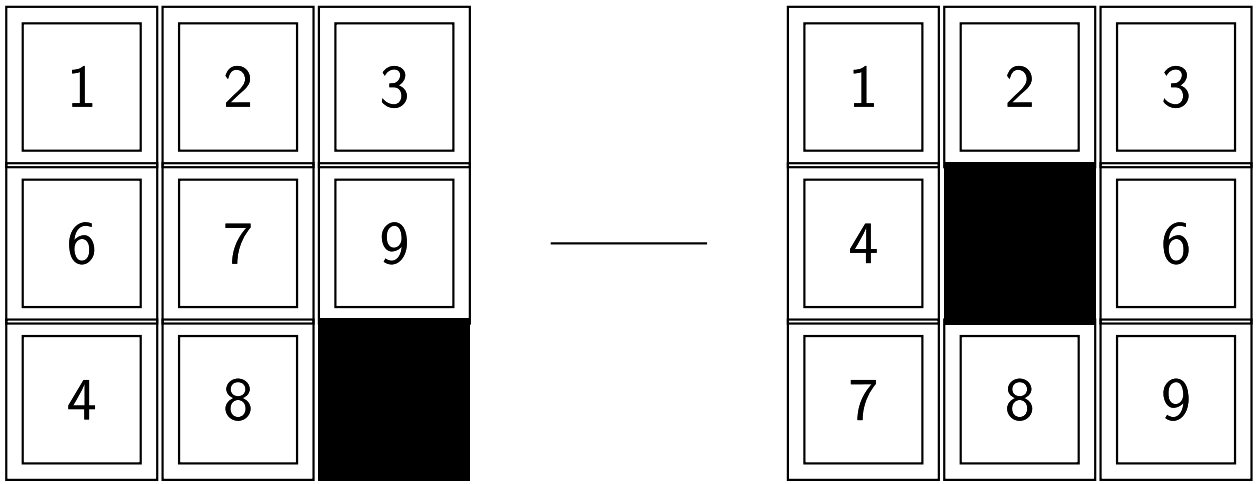
Optimal relaxed plan:

1. $move(t_9, p_6, p_9)$
2. $move(t_8, p_8, p_9)$
3. $move(t_7, p_5, p_8)$
4. $move(t_6, p_4, p_5)$
5. $move(t_6, p_5, p_6)$
6. $move(t_4, p_7, p_4)$
7. $move(t_7, p_8, p_7)$

So $h^*(s_0) = 8$, $h^{MD}(s_0) = 6$, $h^+(s_0) = 7 (> h^{MD}!)$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

8-Puzzle: h^+ vs. h^{MD}



h^+ dominates h^{MD}

- The goal is given as a conjunction of $at(t_i, p_j)$ atoms
- Achieving each single one of them takes at least as many steps as the respective tile's Manhattan distance
- Each action moves a single tile only

And we have just seen that h^+ strictly dominates h^{MD}

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Dominating states

A state s' dominates another state s iff $s \subseteq s'$.

Lemma (relaxation)

Let s be a state, let s' be a state that dominates s, and let π be an action sequence which is applicable in s. Then π^+ is applicable in s' and $app_{\pi^+}(s')$ dominates $app_\pi(s)$. Moreover, if π leads to a goal state from s, then π^+ leads to a goal state from s'.

Proof.

The "moreover" part is immediate from $app_{\pi^+}(s')$ dominating $app_\pi(s)$. Prove the rest by induction over the length of π.

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

STRIPS heuristic

Relaxation heuristics

The relaxation lemma

Greedy algorithm

Optimality

Discussion

Concrete Heuristics

Consequences of the relaxation lemma

Corollary (relaxation leads to dominance and preserves plans)

Let π be an action sequence which is applicable in state s.
Then π^+ is applicable in s and $app_{\pi^+}(s)$ dominates $app_\pi(s)$.
If π is a plan for Π, then π^+ is a plan for Π^+.

Proof.

Apply relaxation lemma with $s' = s$. □

- $\rightsquigarrow$ Relaxations of plans are relaxed plans.
- $\rightsquigarrow$ Relaxations are no harder to solve than the original task.
- $\rightsquigarrow$ Optimal relaxed plans are never longer than optimal plans for original tasks.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Consequences of the relaxation lemma (ctd.)

Corollary (relaxation preserves dominance)

Let s be a state, let s' be a state that dominates s, and let π^+ be a relaxed action sequence applicable in s. Then π^+ is applicable in s' and $app_{\pi^+}(s')$ dominates $app_{\pi^+}(s)$.

Proof.

Apply relaxation lemma with π^+ for π, noting that $(\pi^+)^+ = \pi^+$. □

- $\rightsquigarrow$ If there is a relaxed plan starting from state s, the same plan can be used starting from a dominating state s'.
- $\rightsquigarrow$ Making a transition to a dominating state never hurts in relaxed planning tasks.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Monotonicity of relaxed planning tasks

We need one final property before we can provide an algorithm for solving relaxed planning tasks.

Lemma (monotonicity)

Let $a^+ = \langle pre(a), add(a), \emptyset \rangle$ be a relaxed action and let s be a state in which a^+ is applicable.
Then $app_{a^+}(s)$ dominates s.

Proof.

Since relaxed actions only have positive effects, we have $s \subseteq s \cup add(a) = app_{a^+}(s)$. □

$\rightsquigarrow$ Together with our previous results, this means that making a transition in a relaxed planning task never hurts.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Greedy algorithm for relaxed planning tasks

The relaxation and monotonicity lemmas suggest the following algorithm for solving relaxed planning tasks:

Greedy planning algorithm for $\langle P, A^+, I, G \rangle$

$s := I$
$\pi^+ := \epsilon$
forever:
    if $G \subseteq s$:
        return π^+
    else if there is an action $a^+ \in A^+$ applicable in s
        with $app_{a^+}(s) \neq s$:
        Append such an action a^+ to π^+.
        $s := app_{a^+}(s)$
    else:
        return unsolvable

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Correctness of the greedy algorithm

The algorithm is sound:

- If it returns a plan, this is indeed a correct solution.
- If it returns "unsolvable", the task is indeed unsolvable
  - Upon termination, there clearly is no relaxed plan from s.
  - By iterated application of the monotonicity lemma, s dominates I.
  - By the relaxation lemma, there is no solution from I.

What about completeness (termination) and runtime?

- Each iteration of the loop adds at least one atom to s.
- This guarantees termination after at most $|P|$ iterations.
- Thus, the algorithm can clearly be implemented to run in polynomial time.
  - A good implementation runs in $O(\|\Pi\|)$.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Using the greedy algorithm as a heuristic

We can apply the greedy algorithm within heuristic search:

- In a search node σ, solve the relaxation of the planning task with $\mathit{state}(\sigma)$ as the initial state.
- Set $h(\sigma)$ to the length of the generated relaxed plan.

Is this an admissible heuristic?

- Yes if the relaxed plans are optimal (due to the plan preservation corollary).
- However, usually they are not, because our greedy planning algorithm is very poor.

(What about safety? Goal-awareness? Consistency?)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

The set cover problem

To obtain an admissible heuristic, we need to generate optimal relaxed plans. Can we do this efficiently?

This question is related to the following problem:

Problem (set cover)

Given: *a finite set U, a collection of subsets $C = \{C_1, \ldots, C_n\}$ with $C_i \subseteq U$ for all $i \in \{1, \ldots, n\}$, and a natural number K.*

Question: *Does there exist a set cover of size at most K, i. e., a subcollection $S = \{S_1, \ldots, S_m\} \subseteq C$ with $S_1 \cup \cdots \cup S_m = U$ and $m \leq K$?*

The following is a classical result from complexity theory:

Theorem

The set cover problem is NP-complete.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Hardness of optimal relaxed planning

Theorem (optimal relaxed planning is hard)

The problem of deciding whether a given relaxed planning task has a plan of length at most K is NP-complete.

Proof.

For membership in NP, guess a plan and verify. It is sufficient to check plans of length at most $|P|$, so this can be done in nondeterministic polynomial time.

For hardness, we reduce from the set cover problem.

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

STRIPS heuristic

Relaxation heuristics

The relaxation lemma

Greedy algorithm

Optimality

Discussion

Concrete Heuristics

Hardness of optimal relaxed planning (ctd.)

Proof (ctd.)

Given a set cover instance $\langle U, C, K \rangle$, we generate the following relaxed planning task $\Pi^+ = \langle P, I, A^+, G \rangle$:

- $P = U$
- $I = \emptyset \qquad \equiv I = \{p = 0 \mid p \in P\}$
- $A^+ = \{\langle \emptyset, \bigcup_{p \in C_i} \{p\}, \emptyset \rangle \mid C_i \in C\}$
- $G = U$

If S is a set cover, the corresponding actions form a plan. Conversely, each plan induces a set cover by taking the subsets corresponding to the actions. Clearly, there exists a plan of length at most K iff there exists a set cover of size K.

Moreover, Π^+ can be generated from the set cover instance in polynomial time, so this is a polynomial reduction. □

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

STRIPS heuristic

Relaxation heuristics

The relaxation lemma

Greedy algorithm

Optimality

Discussion

Concrete Heuristics

Using relaxations in practice

How can we use relaxations for heuristic planning in practice?

Different possibilities:

- Implement an optimal planner for relaxed planning tasks and use its solution lengths as an estimate, even though it is NP-hard.
  $\rightsquigarrow$ h^+ heuristic *(not that realistic. why?)*
- Do not actually solve the relaxed planning task, but compute an estimate of its difficulty in a different way.
  $\rightsquigarrow$ $h_{\max}$ heuristic, h_{add} heuristic
- Compute a solution for relaxed planning tasks which is not necessarily optimal, but "reasonable".
  $\rightsquigarrow$ h_{FF} heuristic

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Using relaxations in practice

How can we use relaxations for heuristic planning in practice?

Different possibilities:

- Implement an optimal planner for relaxed planning tasks and use its solution lengths as an estimate, even though it is NP-hard.
  $\rightsquigarrow$ h^+ heuristic *(not that realistic. why?)*
- Do not actually solve the relaxed planning task, but compute an estimate of its difficulty in a different way.
  $\rightsquigarrow$ $h_{\max}$ heuristic, h_{add} heuristic
- Compute a solution for relaxed planning tasks which is not necessarily optimal, but "reasonable".
  $\rightsquigarrow$ h_{FF} heuristic

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Using relaxations in practice

How can we use relaxations for heuristic planning in practice?

Different possibilities:

- Implement an optimal planner for relaxed planning tasks and use its solution lengths as an estimate, even though it is NP-hard.
  $\rightsquigarrow$ h^+ heuristic *(not that realistic. why?)*
- Do not actually solve the relaxed planning task, but compute an estimate of its difficulty in a different way.
  $\rightsquigarrow$ h_{max} heuristic, h_{add} heuristic
- Compute a solution for relaxed planning tasks which is not necessarily optimal, but "reasonable".
  $\rightsquigarrow$ h_{FF} heuristic

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
STRIPS heuristic
Relaxation heuristics
The relaxation lemma
Greedy algorithm
Optimality
Discussion
Concrete Heuristics

Reminder: Greedy algorithm for relaxed planning tasks

Greedy planning algorithm for $\langle P, A^+, I, G \rangle$

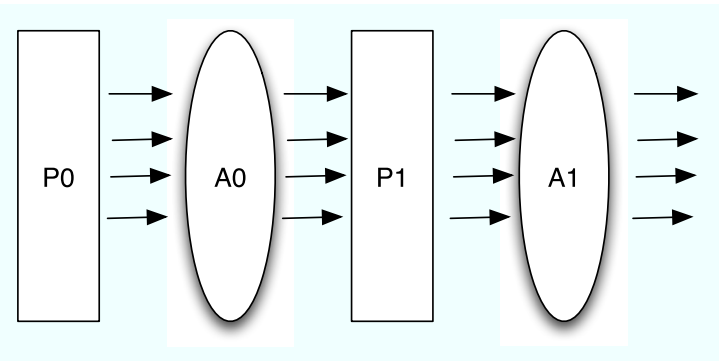
$s := I$
$\pi^+ := \epsilon$
forever:
    if $G \subseteq s$:
        return π^+
    else if there is an action $a^+ \in A^+$ applicable in s
        with $app_{a^+}(s) \neq s$:
        Append such an action a^+ to π^+.
        $s := app_{a^+}(s)$
    else:
        return unsolvable

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Graphical "interpretation": Relaxed planning graphs

- Build a layered reachability graph $P_0, A_0, P_1, A_1, \ldots$

$$
\begin{aligned}
P_0 &= \{p \in I\} \\
A_i &= \{a \in A \mid pre(a) \subseteq P_i\} \\
P_{i+1} &= P_i \cup \{p \in add(a) \mid a \in A_i\}
\end{aligned}
$$



- Terminate when $G \subseteq P_i$

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Running example

$$
\begin{aligned}
P &= \{a, b, c, d, e, f, g, h\} \\
I &= \{a\} \\
G &= \{c, d, e, f, g\} \\
a_1 &= \langle \{a\}, \{b, c\}, \emptyset \rangle \\
a_2 &= \langle \{a, c\}, \{d\}, \emptyset \rangle \\
a_3 &= \langle \{b, c\}, \{e\}, \emptyset \rangle \\
a_4 &= \langle \{b\}, \{f\}, \emptyset \rangle \\
a_5 &= \langle \{d\}, \{e, f\}, \emptyset \rangle \\
a_6 &= \langle \{d\}, \{g\}, \emptyset \rangle
\end{aligned}
$$

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

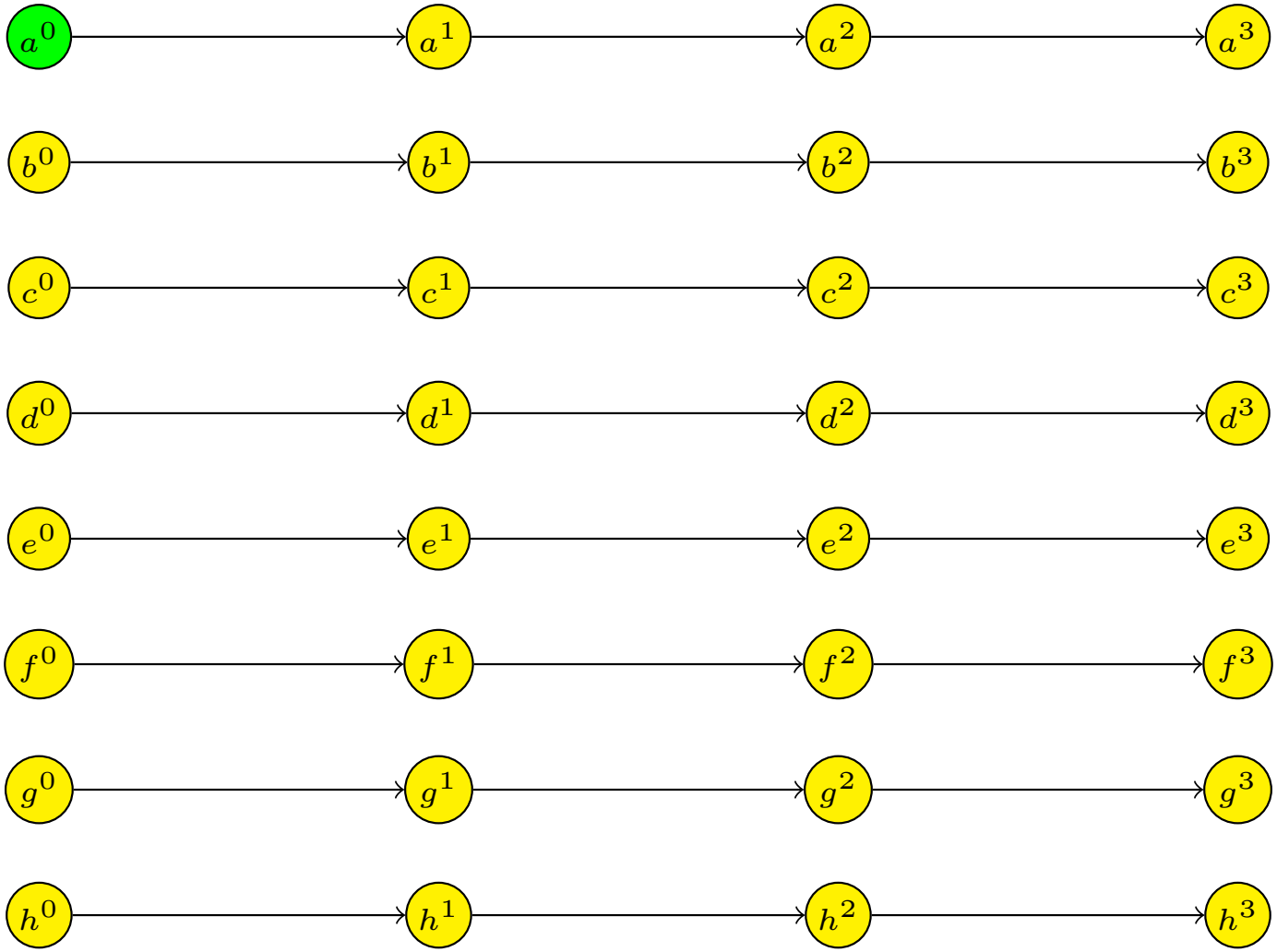
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: Relaxed planning graph



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

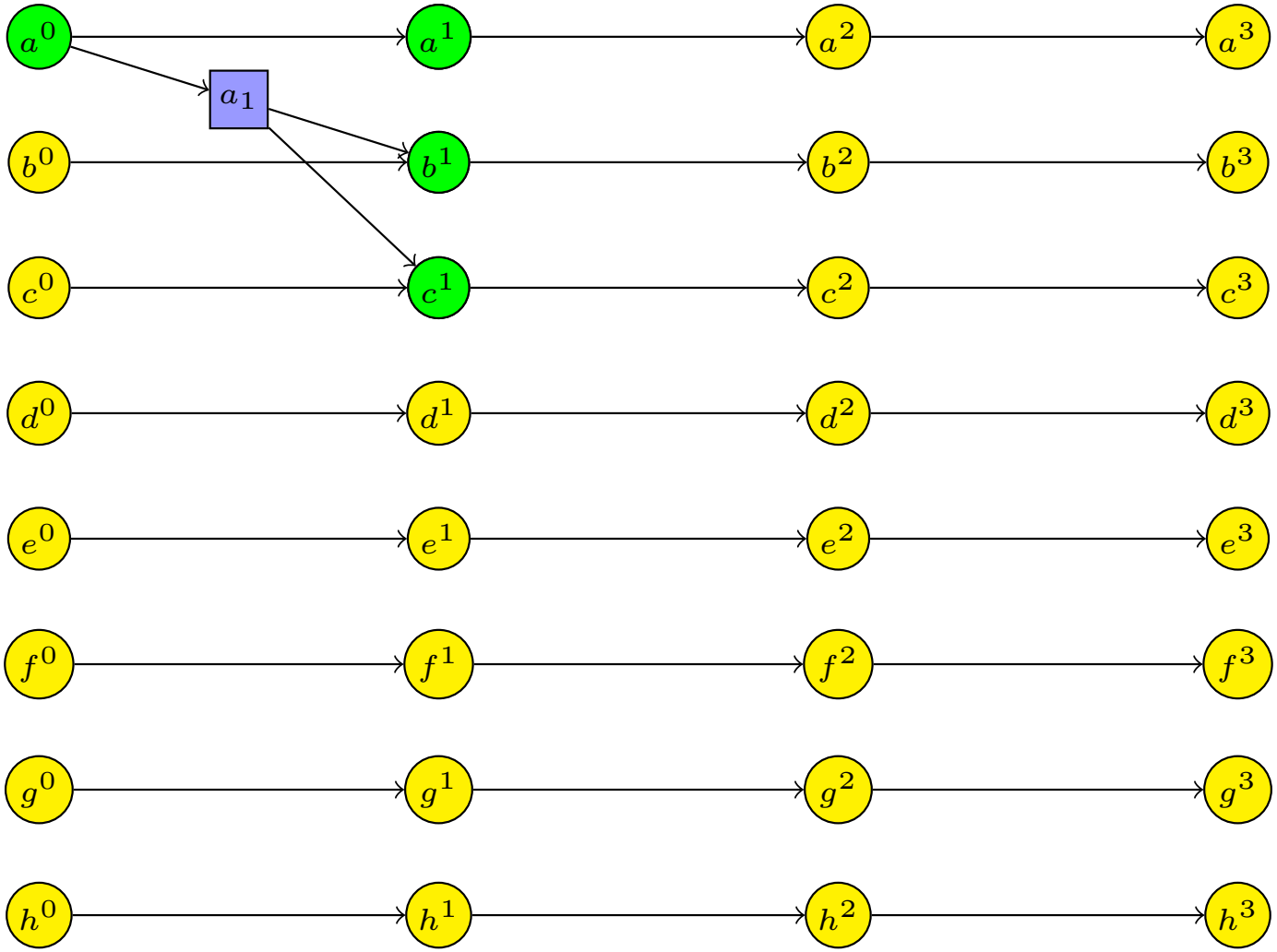
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: Relaxed planning graph



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

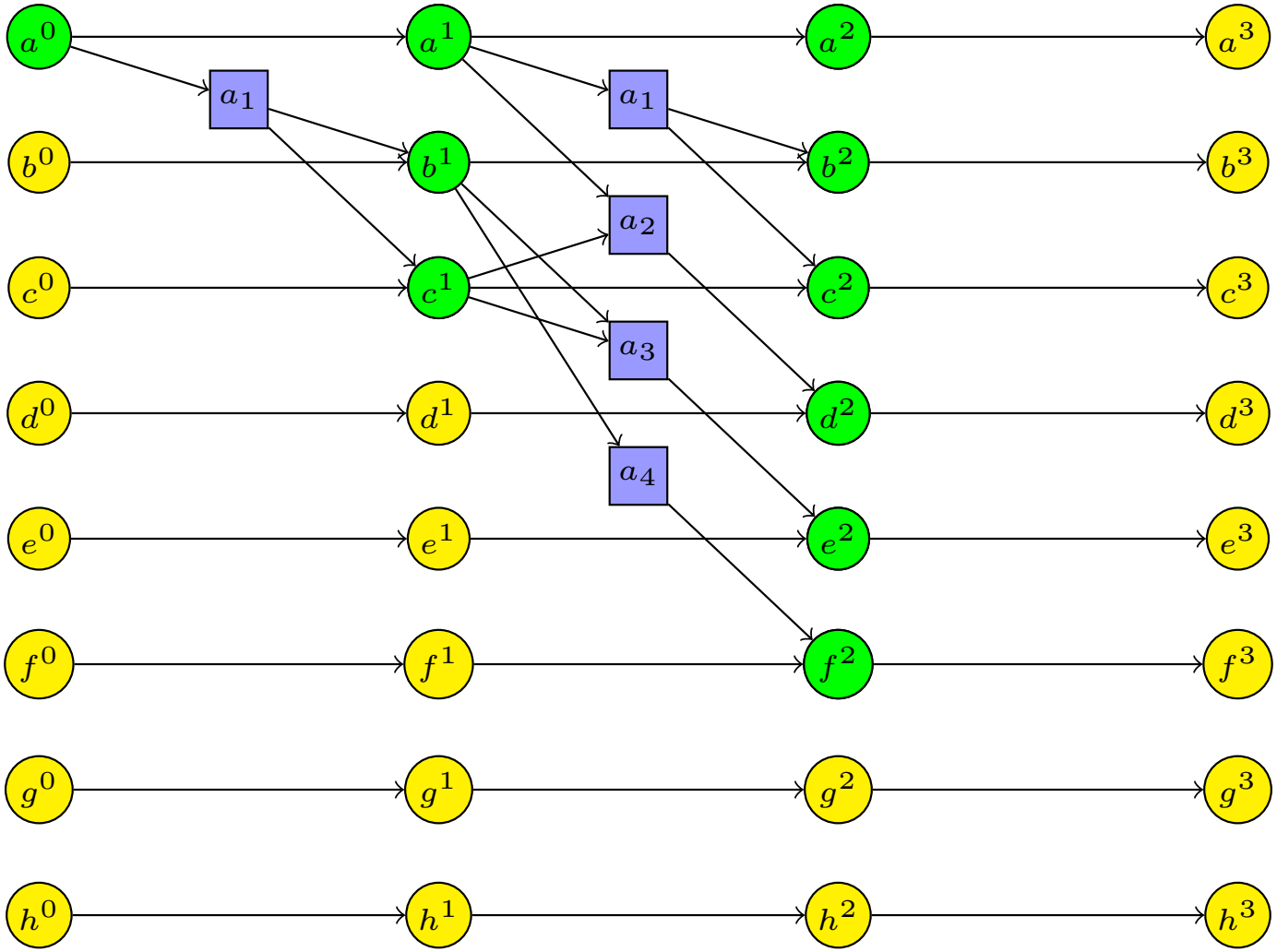
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: Relaxed planning graph



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

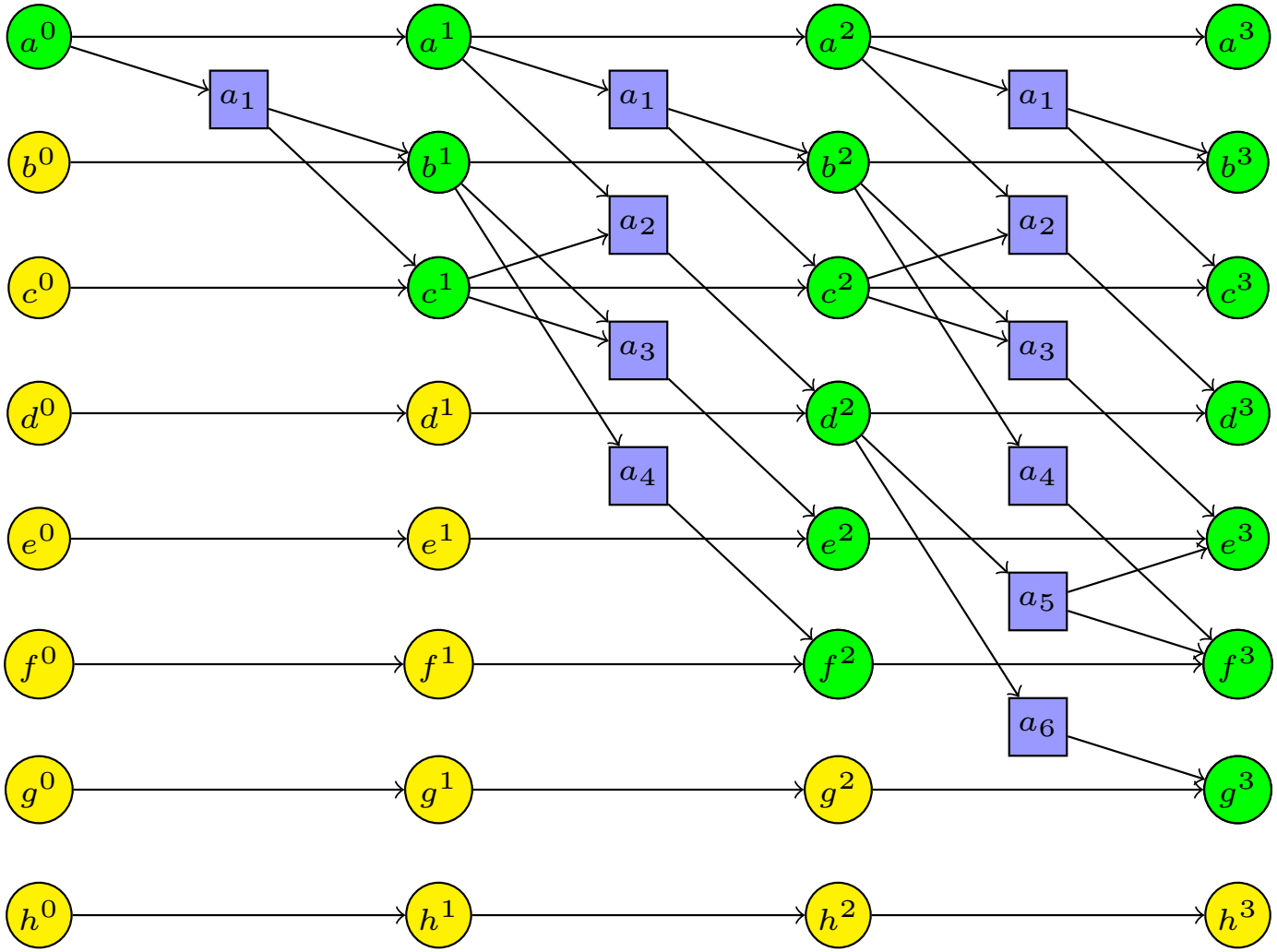
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: Relaxed planning graph



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

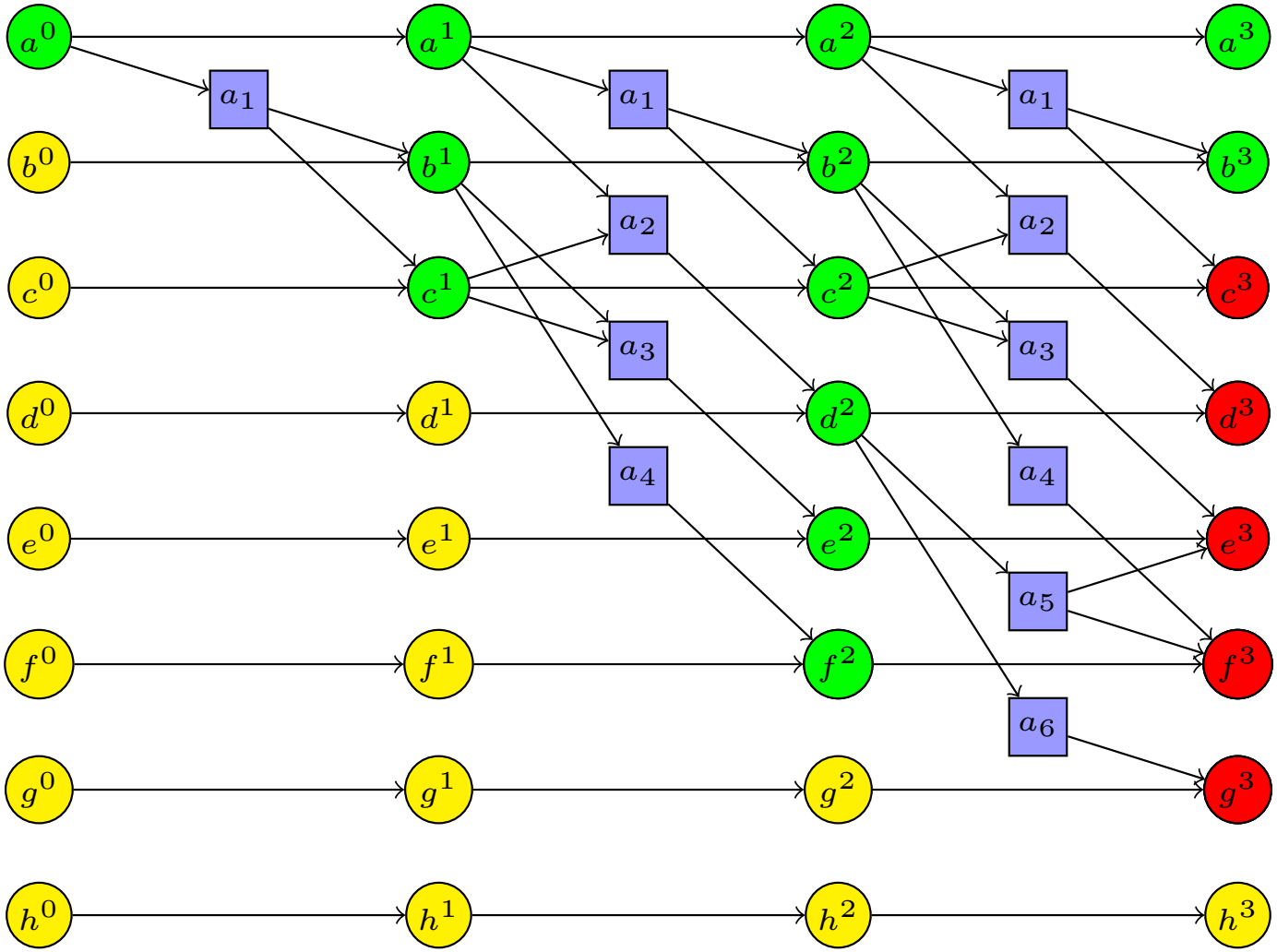
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: Relaxed planning graph



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

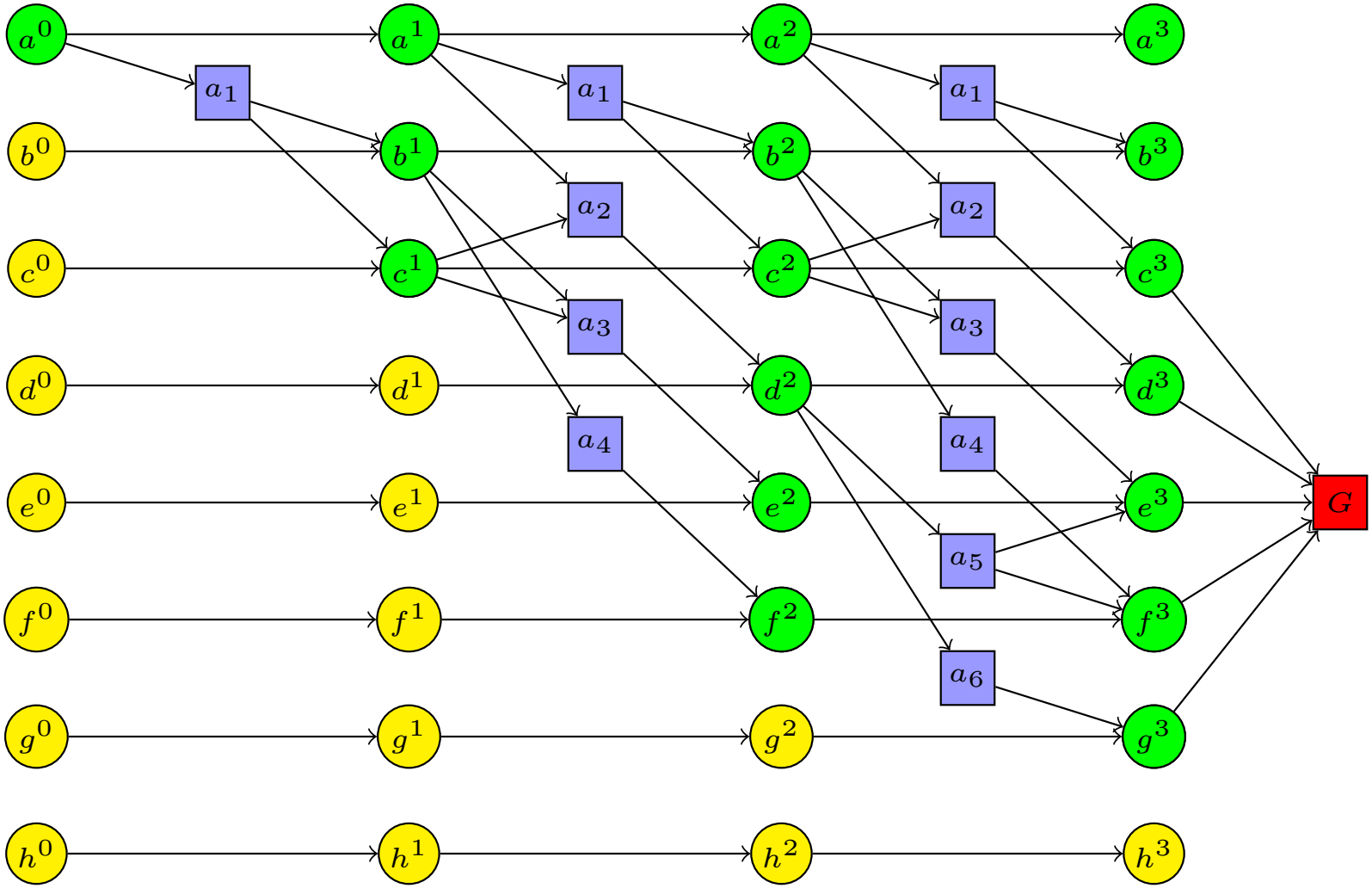
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: Relaxed planning graph



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Generic relaxed planning graph heuristics

Computing heuristics from relaxed planning graphs

def *generic-rpg-heuristic*$(\langle P, I, O, G\rangle, s)$:
    $\Pi^+ := \langle P, s, O^+, G\rangle$
    for $k \in \{0, 1, 2, \ldots\}$:
        $rpg := RPG_k(\Pi^+)$
        if $G \subseteq P_k$:
            Annotate nodes of *rpg*.
            if termination criterion is true:
                return heuristic value from annotations
        else if $k = |P|$:
            return ∞

$\rightsquigarrow$ generic template for heuristic functions

$\rightsquigarrow$ to get concrete heuristic: fill in highlighted parts

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Concrete examples for the generic heuristic

Many planning heuristics fit the generic template:

- max heuristic h_{max}
- additive heuristic h_{add}
- FF heuristic h_{FF}
- ...

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

h_{max}

h_{add}

h_{FF}

Comparison & practice

Forward cost heuristics

- The simplest relaxed planning graph heuristics are forward cost heuristics.
- Examples: $h_{\max}$, h_{add}
- Here, node annotations are cost values (natural numbers).
- The cost of a node estimates how expensive (in terms of required operators) it is to make this node true.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Forward cost heuristics: fitting the template

Forward cost heuristics

Computing annotations:

- Propagate cost values bottom-up using a combination rule for action nodes and a combination rule for proposition nodes.
- At action nodes, add 1 after applying combination rule.

Termination criterion:

- stability: terminate if $P_k = P_{k-1}$ and cost for each proposition node $p^k \in P_k$ equals cost for $p^{k-1} \in P_{k-1}$

Heuristic value:

- The heuristic value is the cost of the auxiliary goal node.

- Different forward cost heuristics only differ in their choice of combination rules.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

The max heuristic $h_{\max}$ (again)

Forward cost heuristics: max heuristic $h_{\max}$

Combination rule for action nodes:

- $\mathit{cost}(u) = \max(\{\mathit{cost}(v_1), \ldots, \mathit{cost}(v_k)\})$
  (with $\max(\emptyset) := 0$)

Combination rule for proposition nodes:

- $\mathit{cost}(u) = \min(\{\mathit{cost}(v_1), \ldots, \mathit{cost}(v_k)\})$

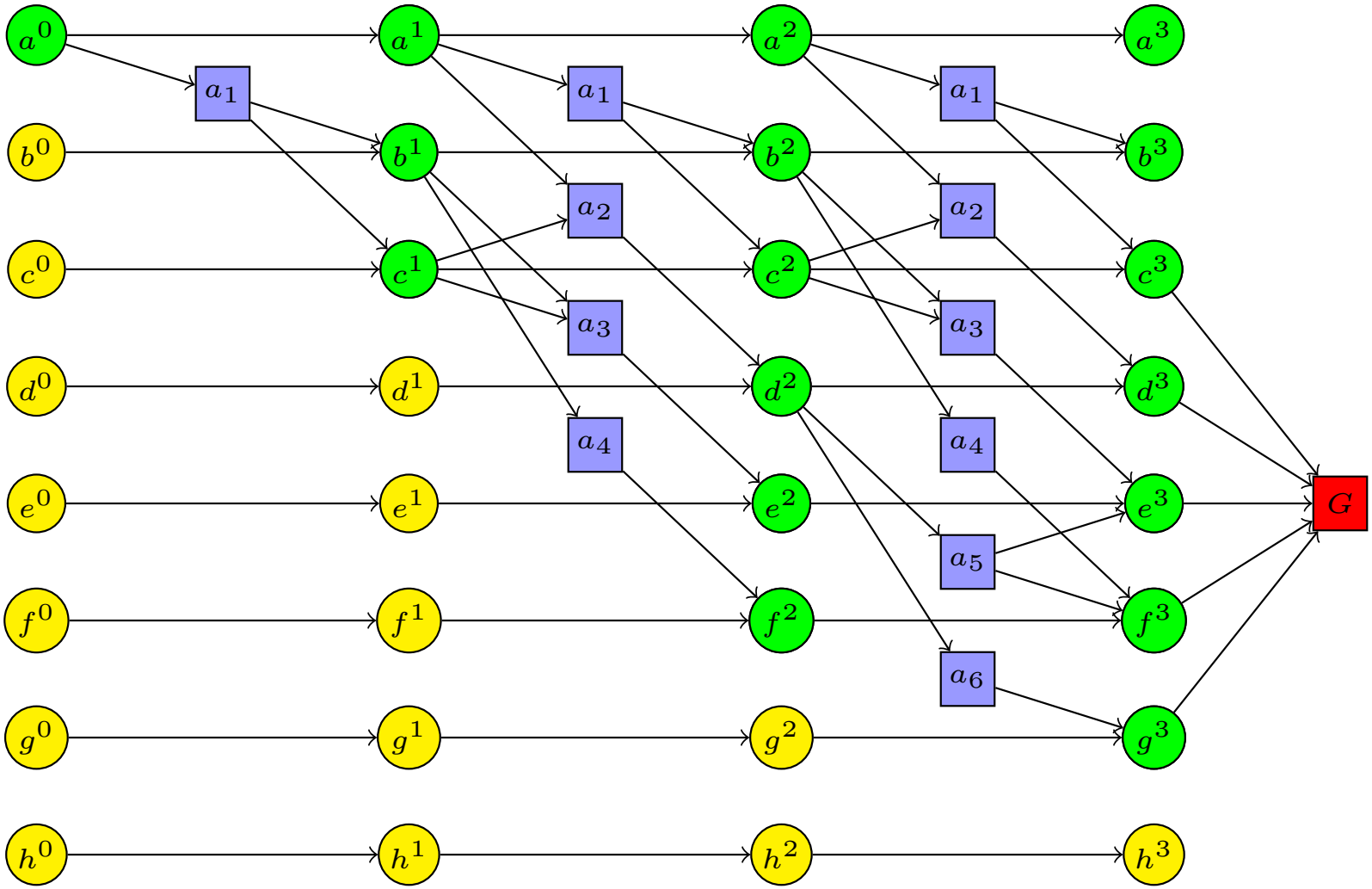
In both cases, $\{v_1, \ldots, v_k\}$ is the set of immediate predecessors of u.

Intuition:

- Action rule: If we have to achieve several preconditions, estimate this by the most expensive cost.
- Proposition rule: If we have a choice how to achieve a proposition, pick the cheapest possibility.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

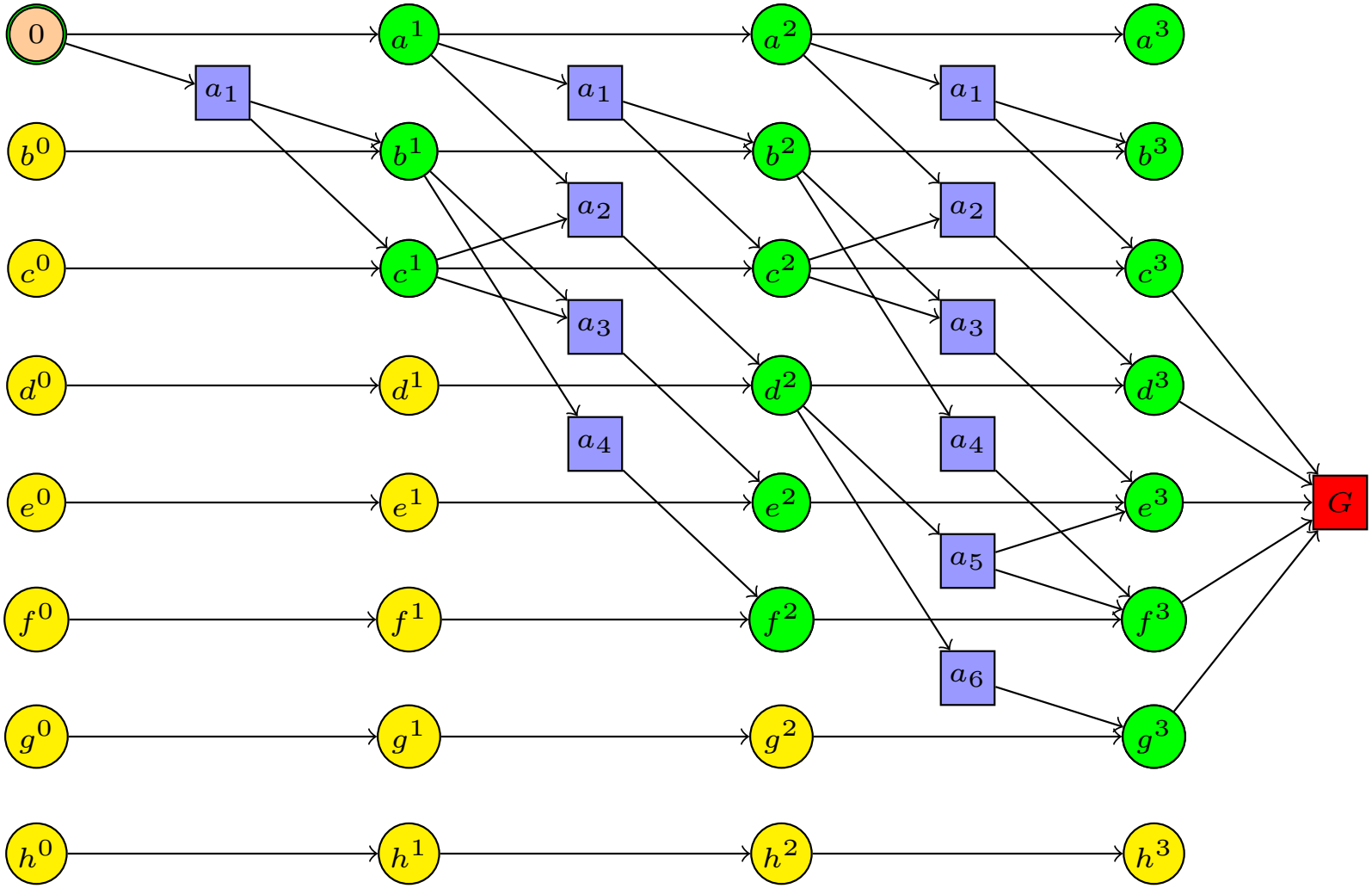
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

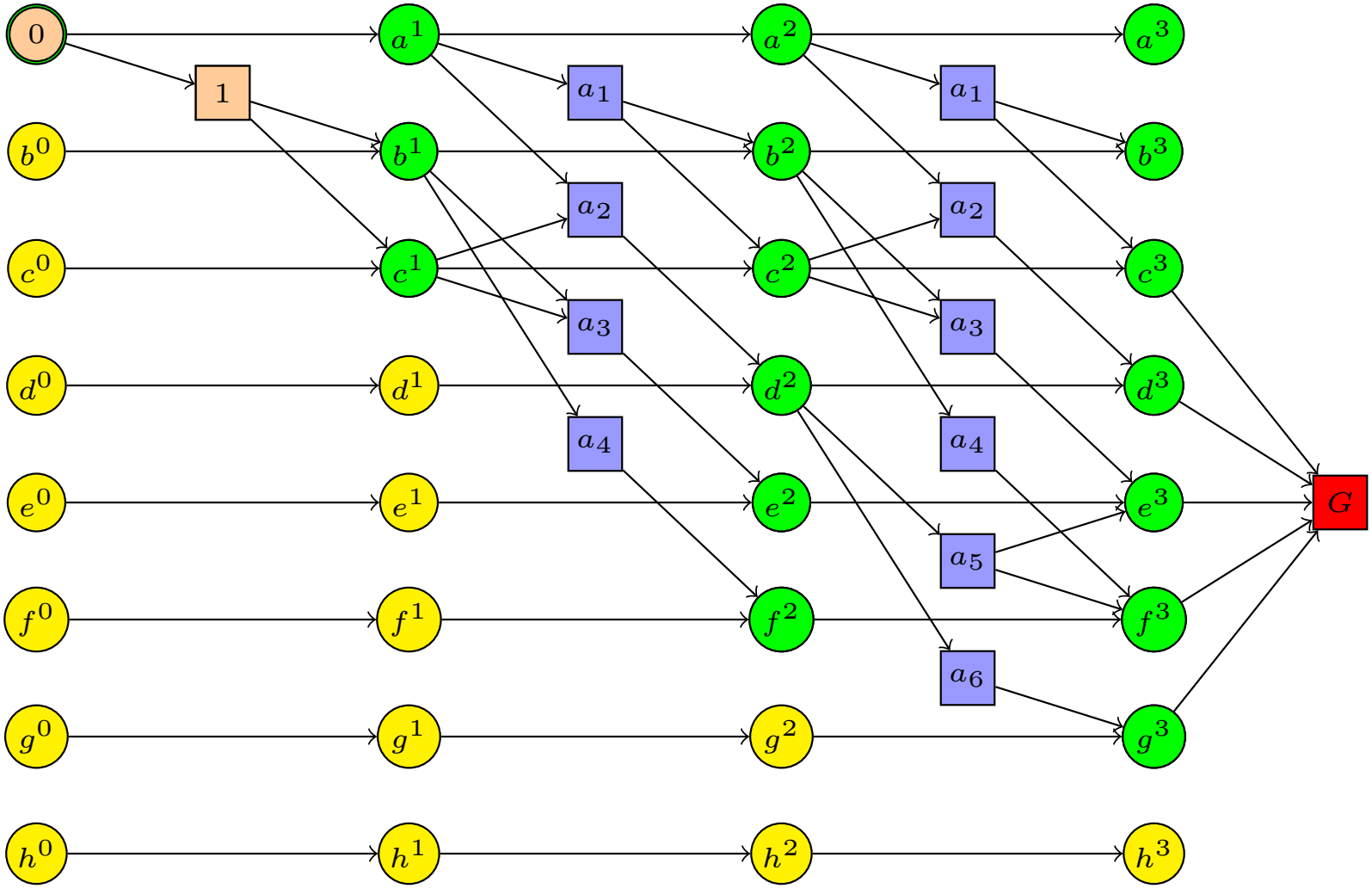
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

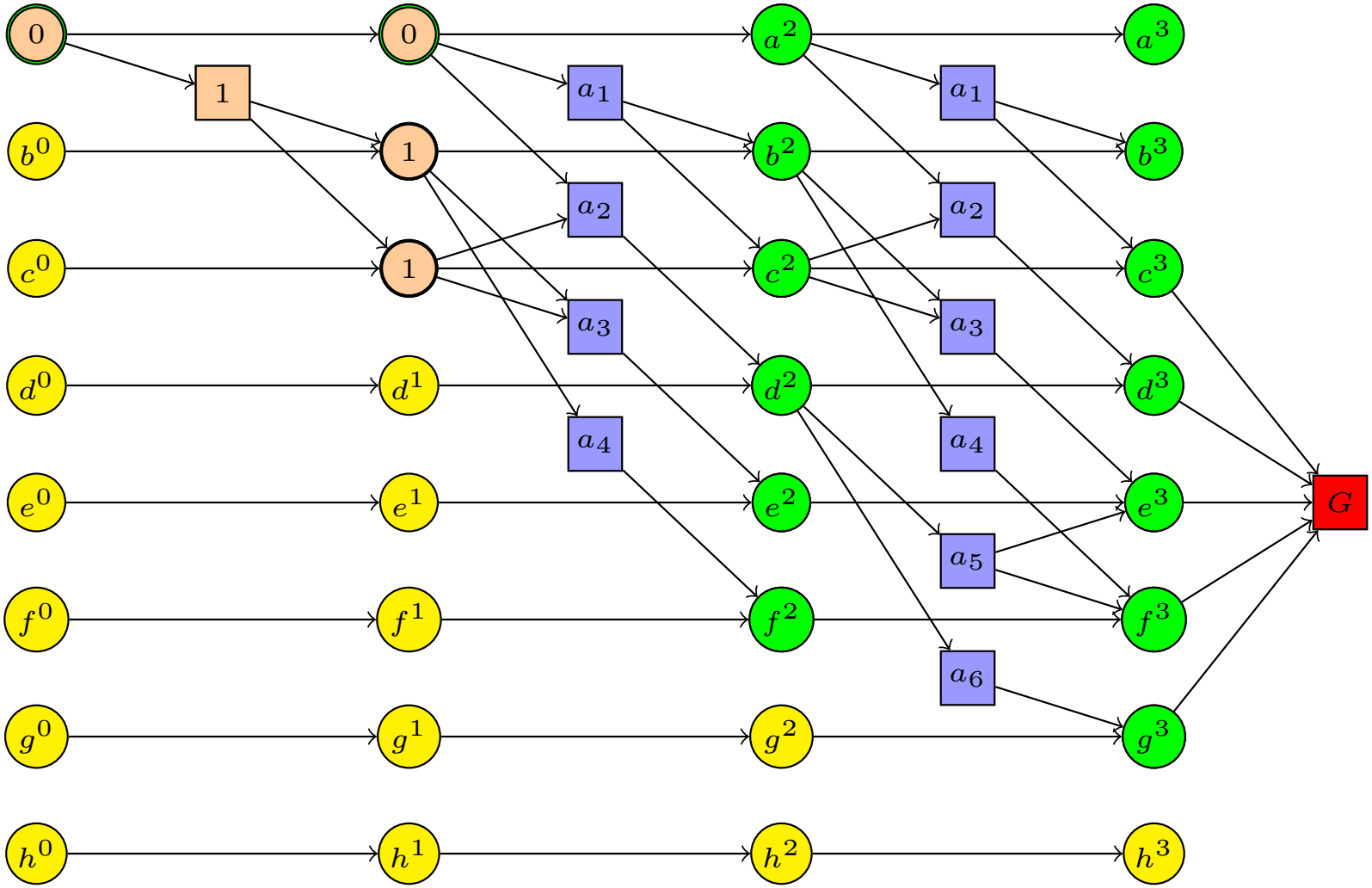
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

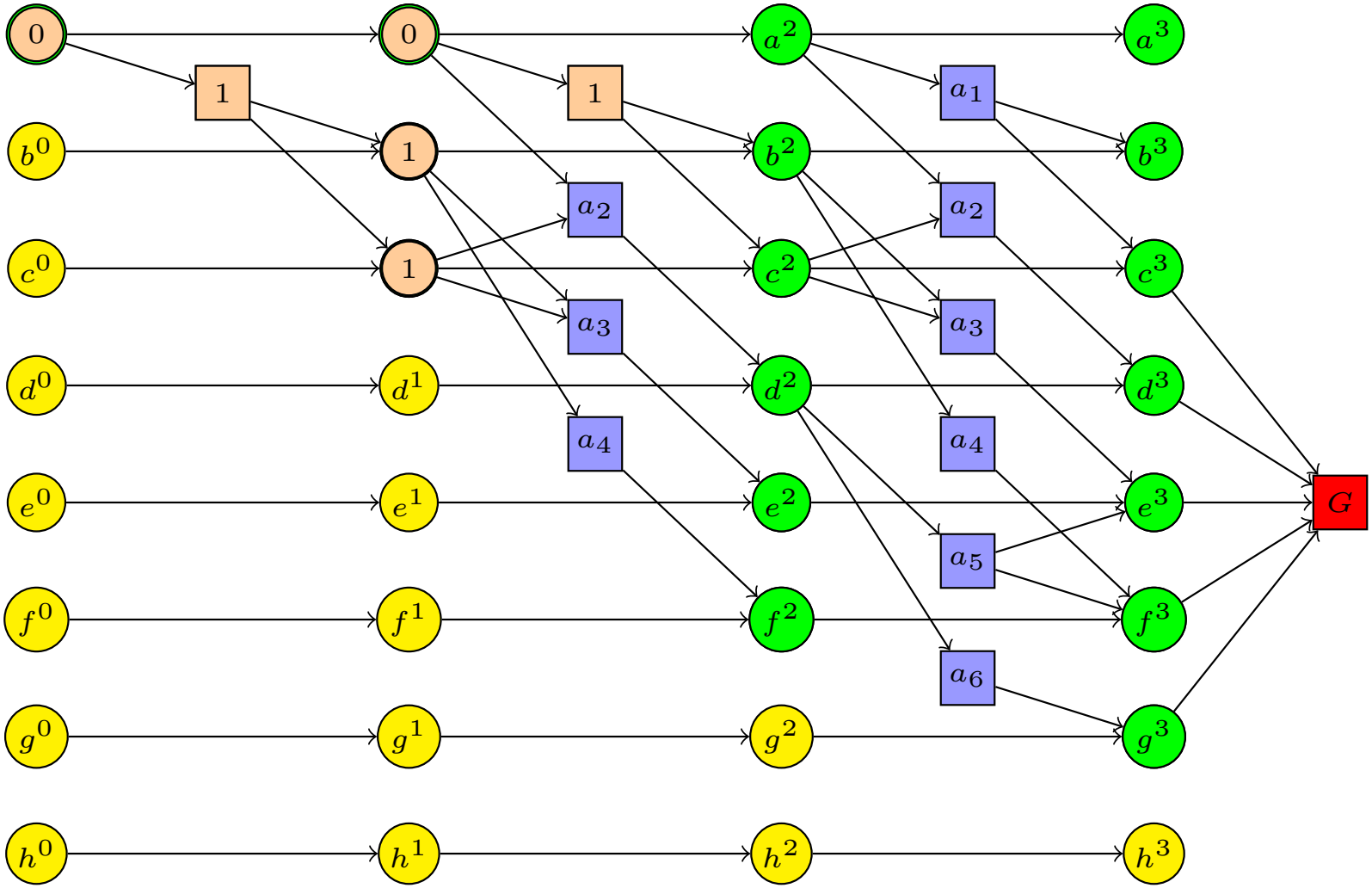
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

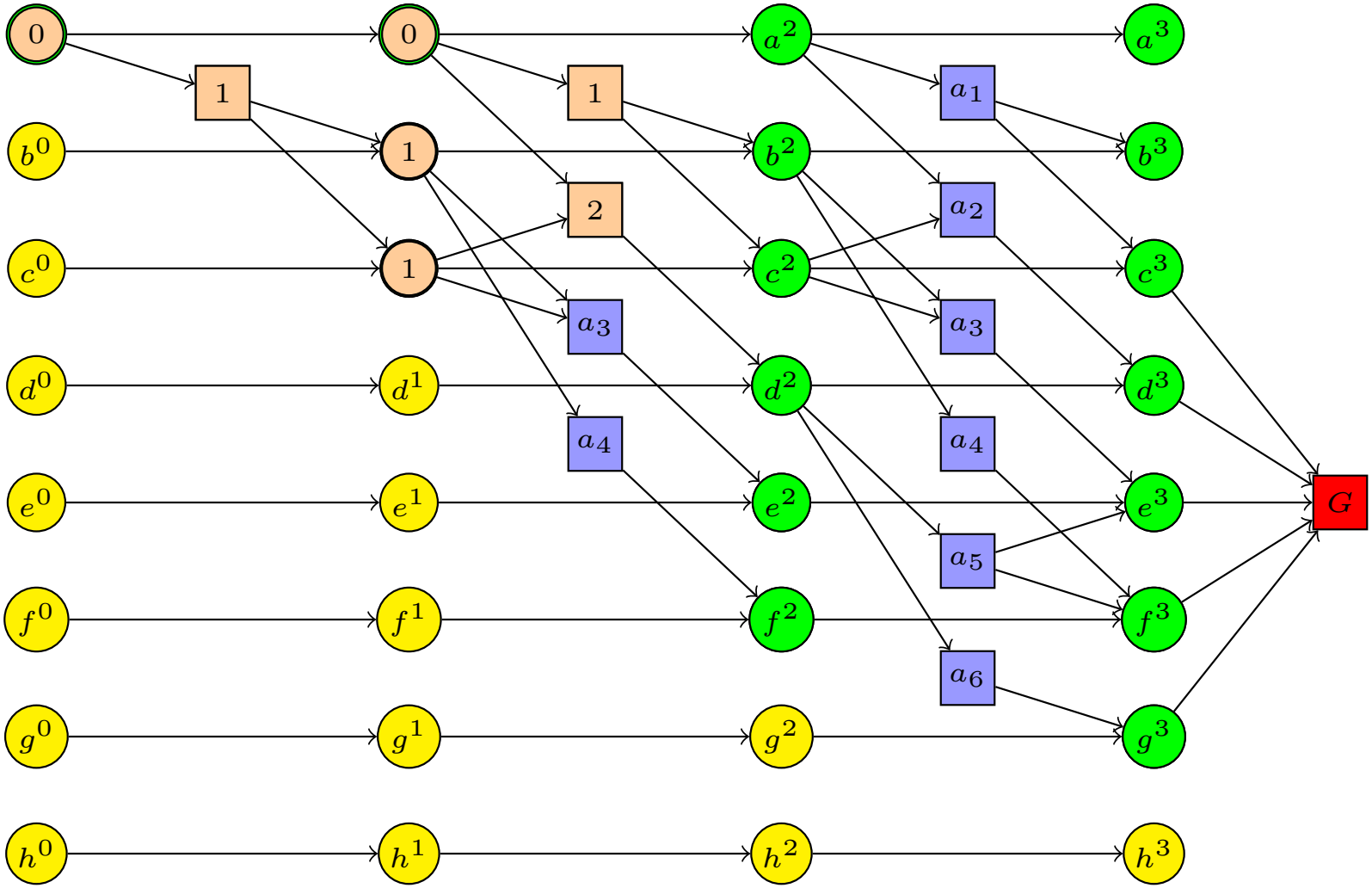
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

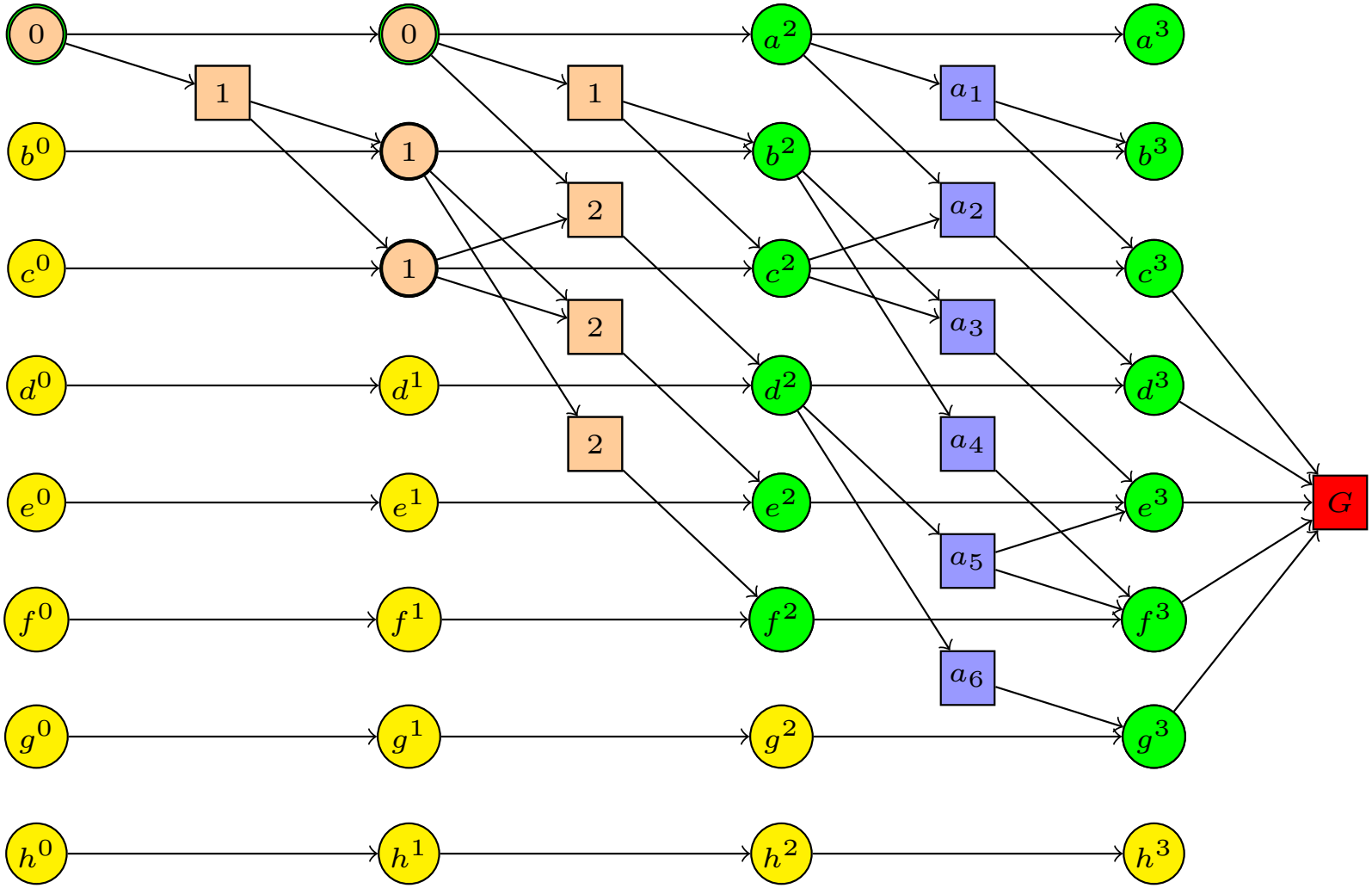
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

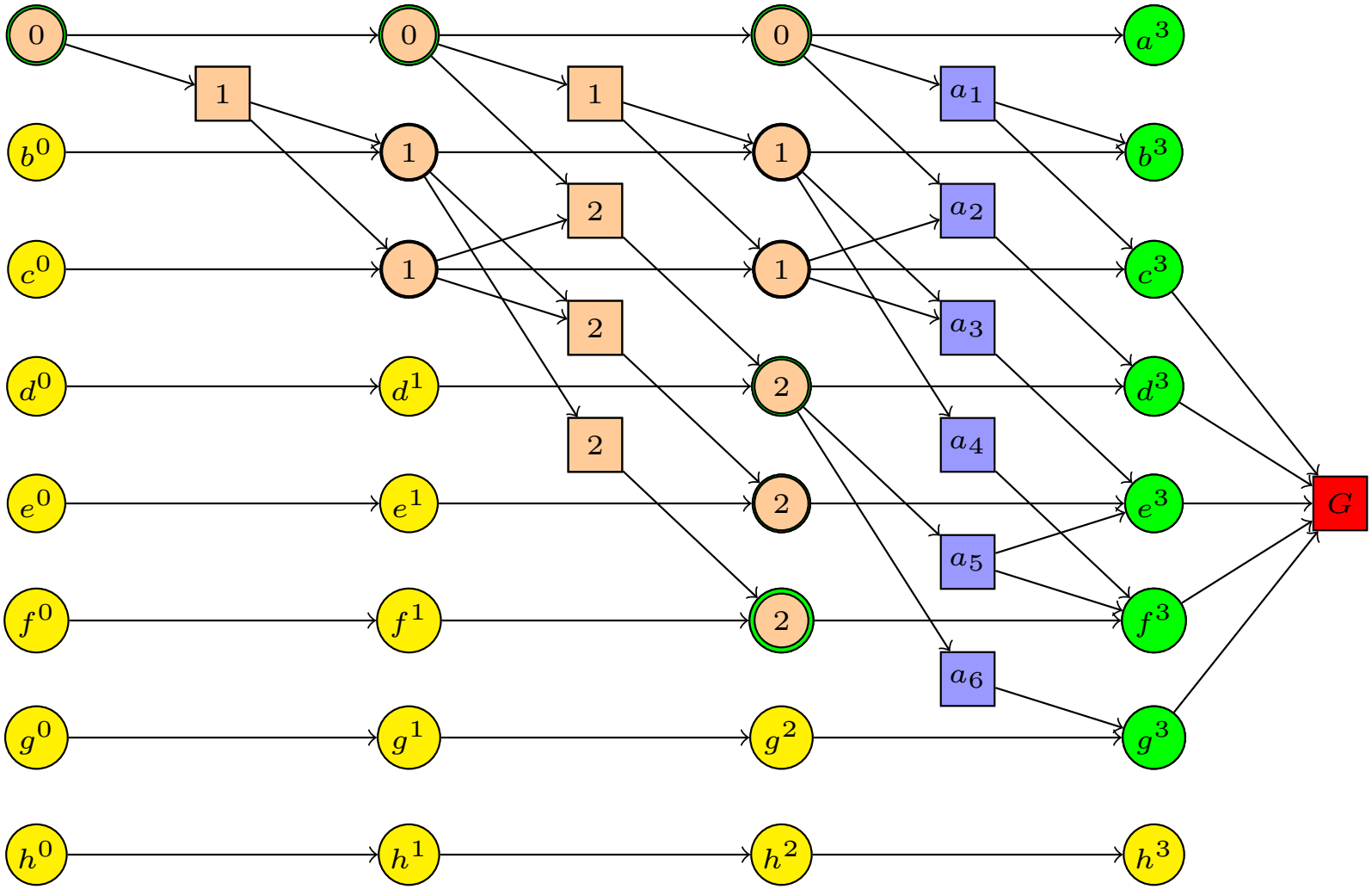
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

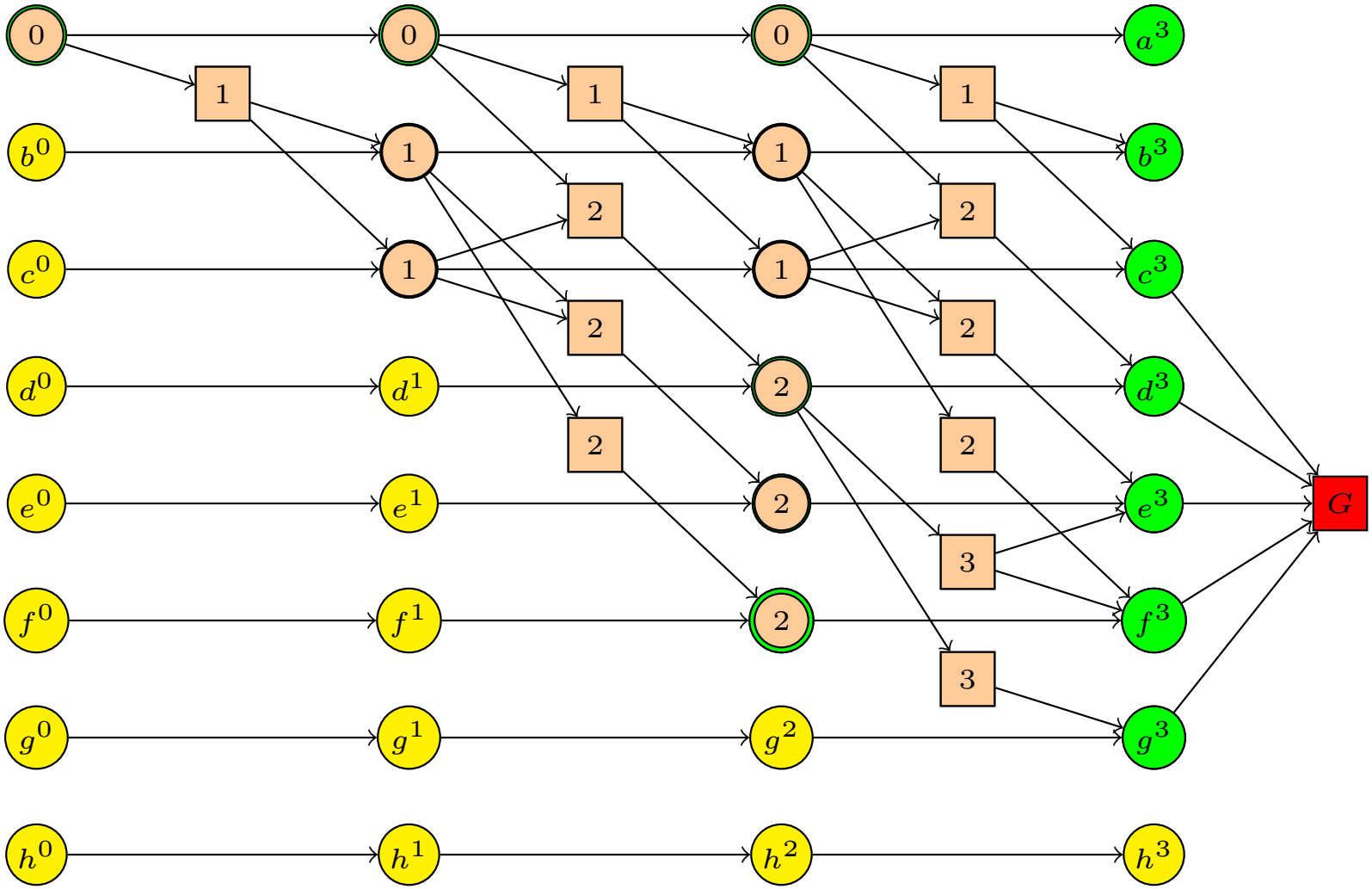
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

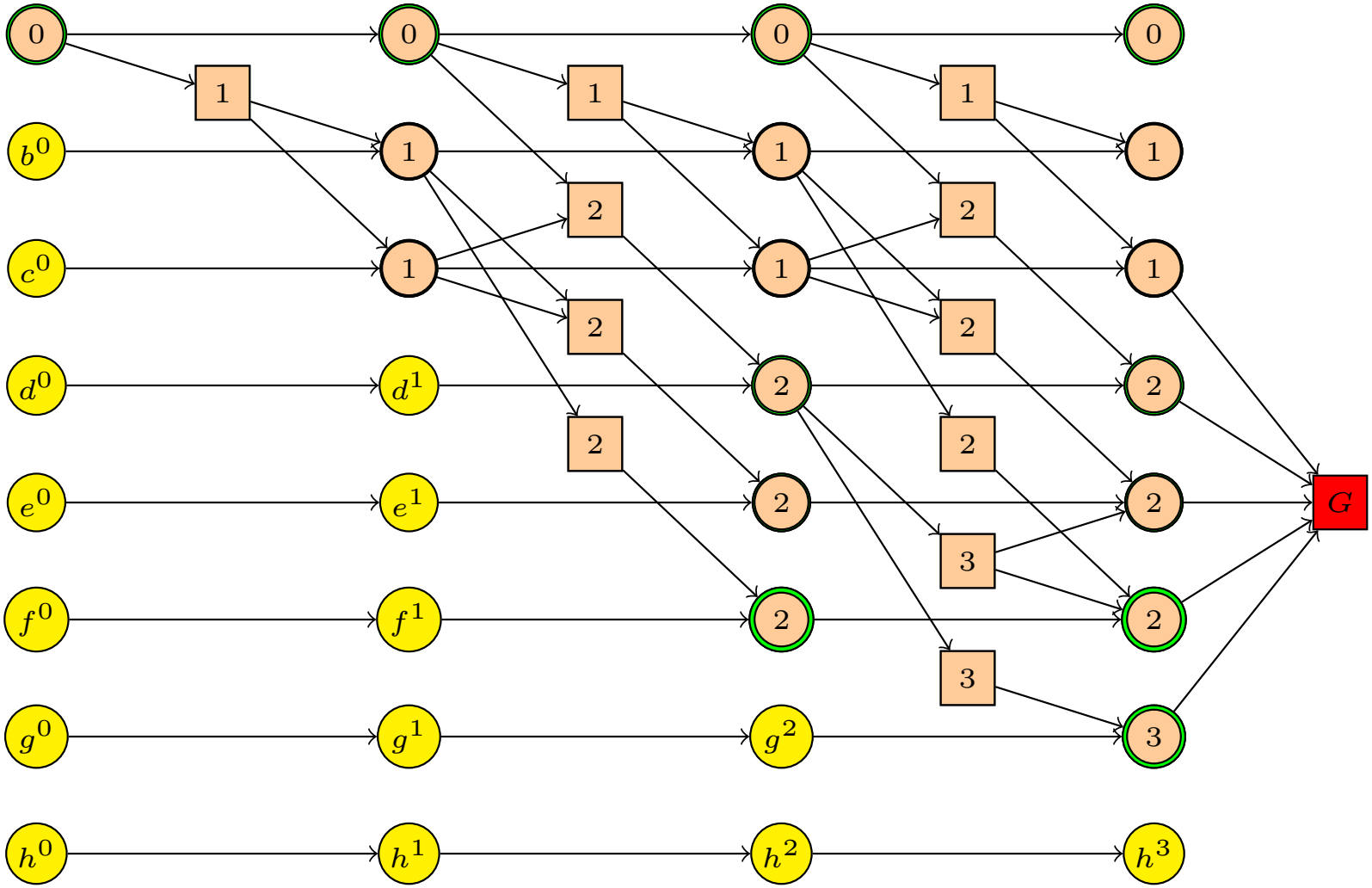
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

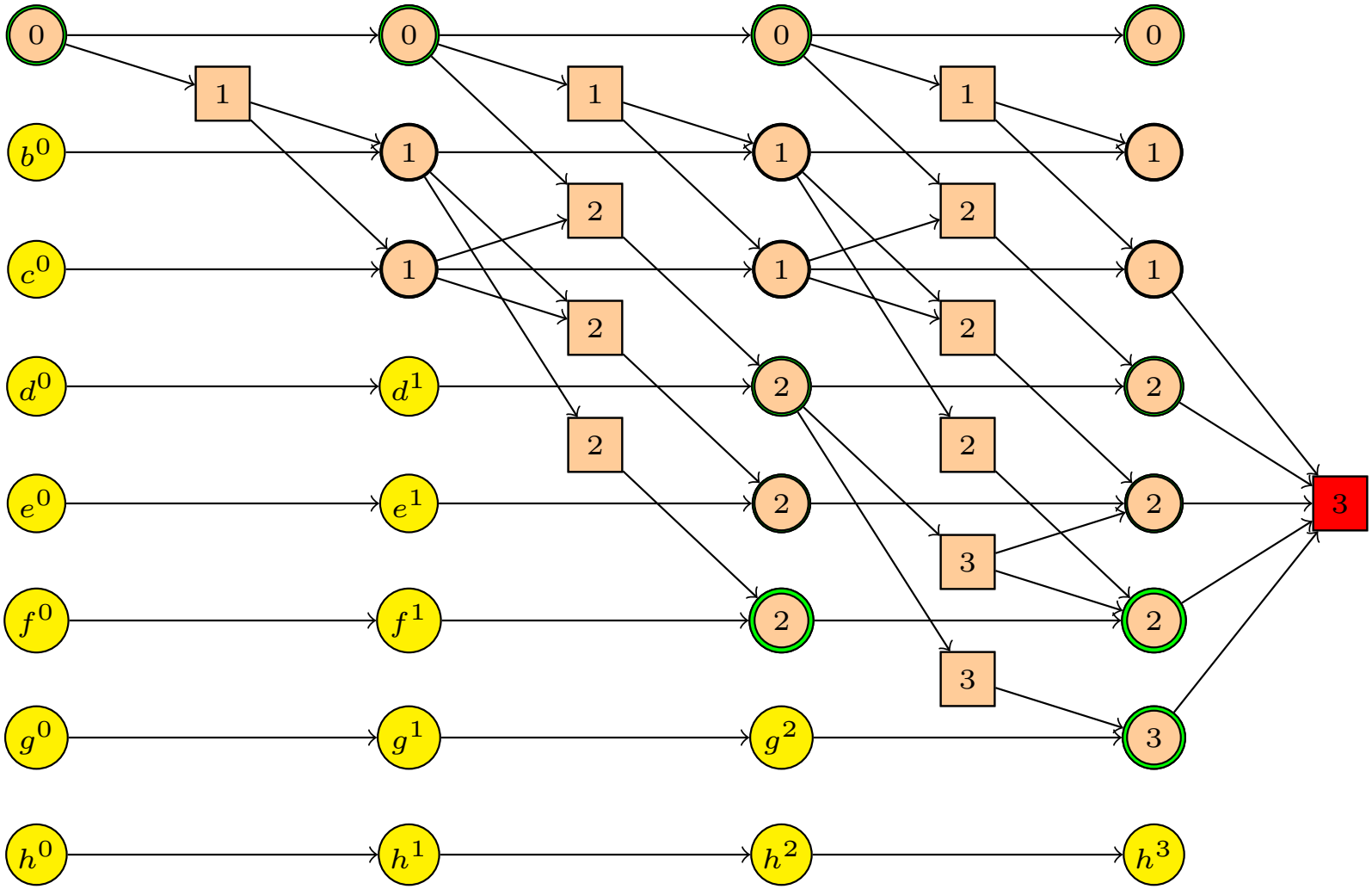
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: $h_{\max}$



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

The additive heuristic

Forward cost heuristics: additive heuristic h_{add}

Combination rule for action nodes:

- $\mathit{cost}(u) = \mathit{cost}(v_1) + \ldots + \mathit{cost}(v_k)$
  (with $\sum(\emptyset) := 0$)

Combination rule for proposition nodes:

- $\mathit{cost}(u) = \min(\{\mathit{cost}(v_1), \ldots, \mathit{cost}(v_k)\})$

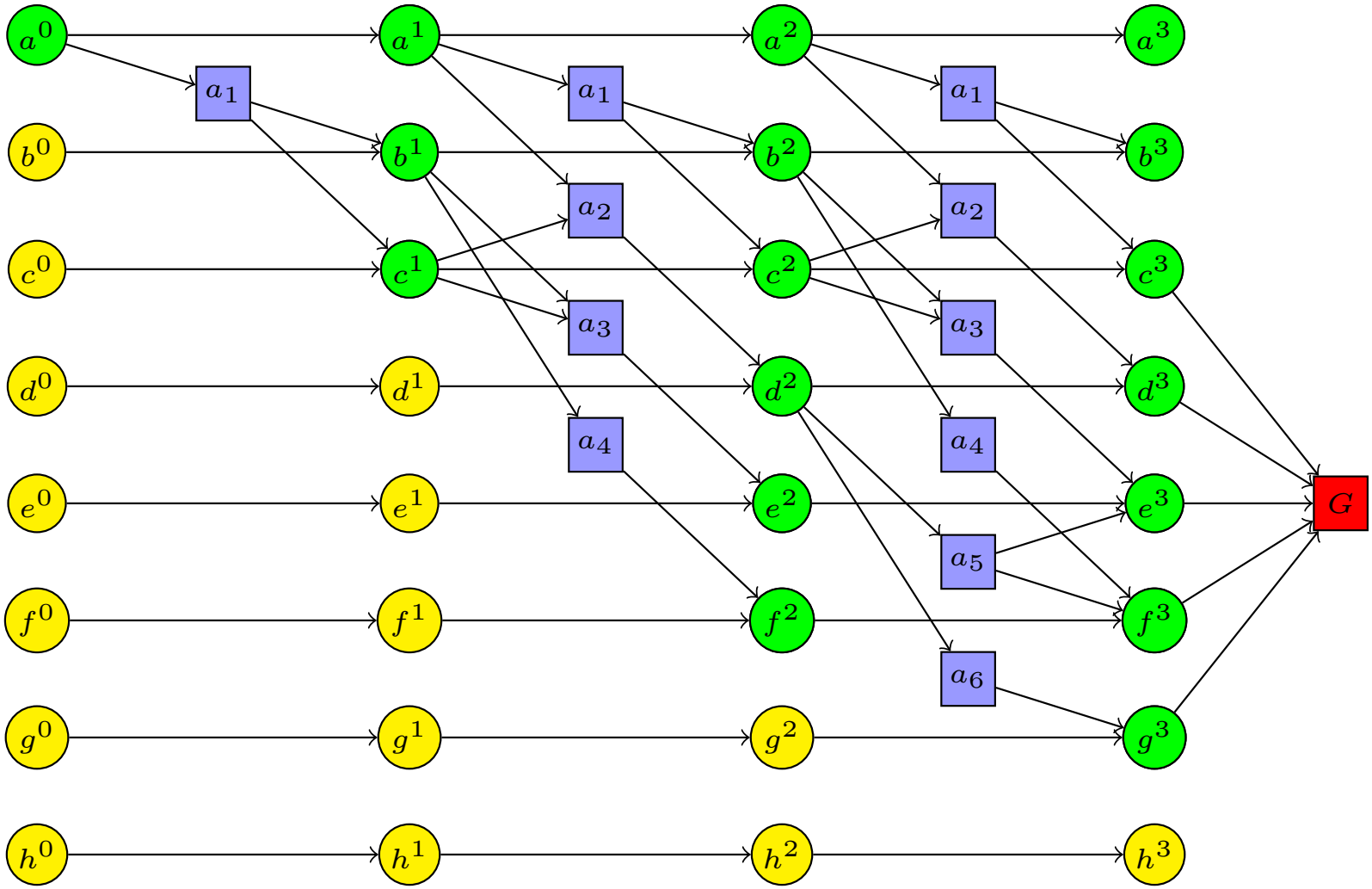
In both cases, $\{v_1, \ldots, v_k\}$ is the set of immediate predecessors of u.

Intuition:

- Action rule: If we have to achieve several preconditions, estimate this by the cost of achieving each in isolation.
- Proposition rule: If we have a choice how to achieve a proposition, pick the cheapest possibility.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Running example: h_{add}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

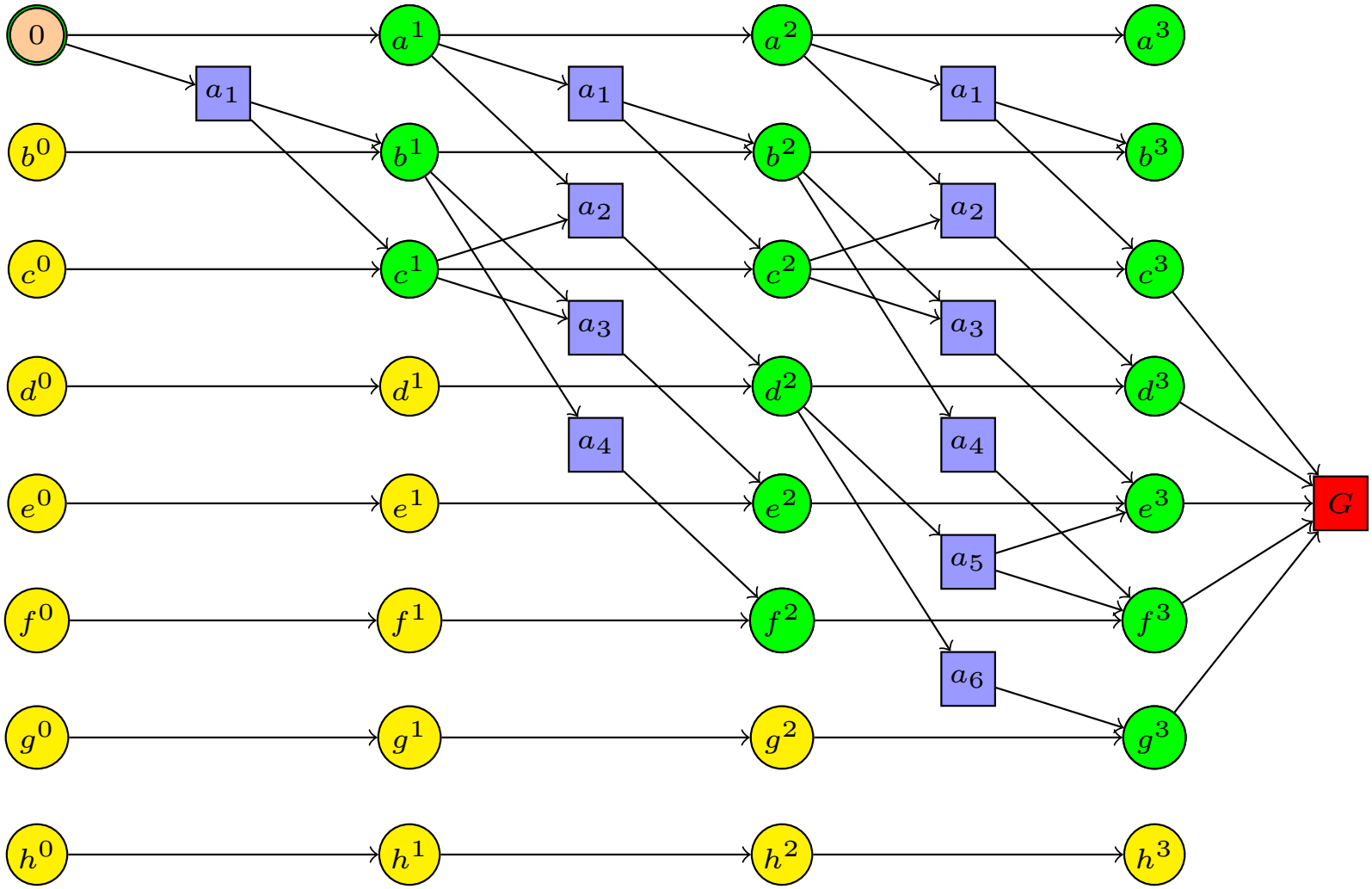
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{add}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

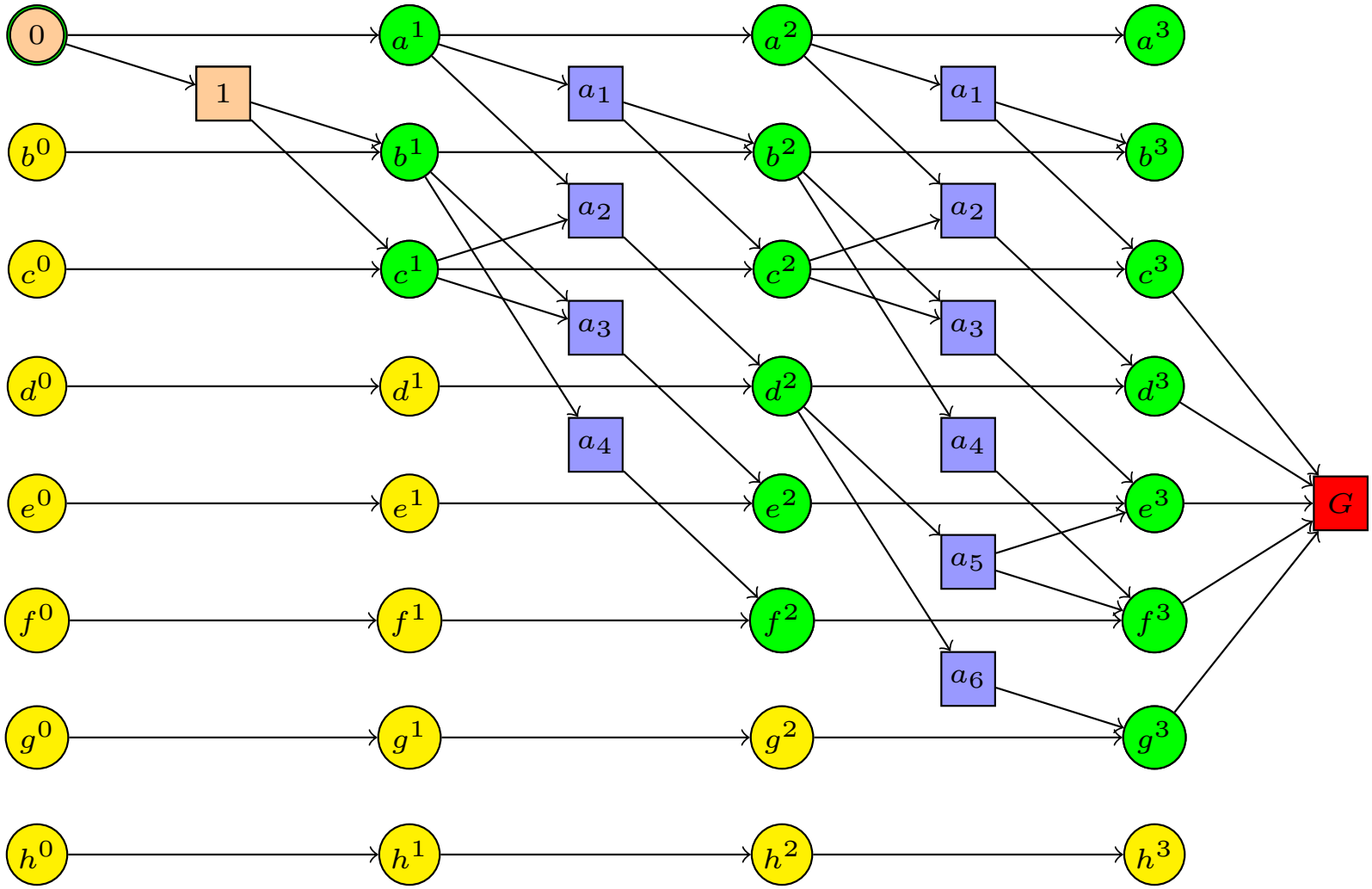
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{add}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

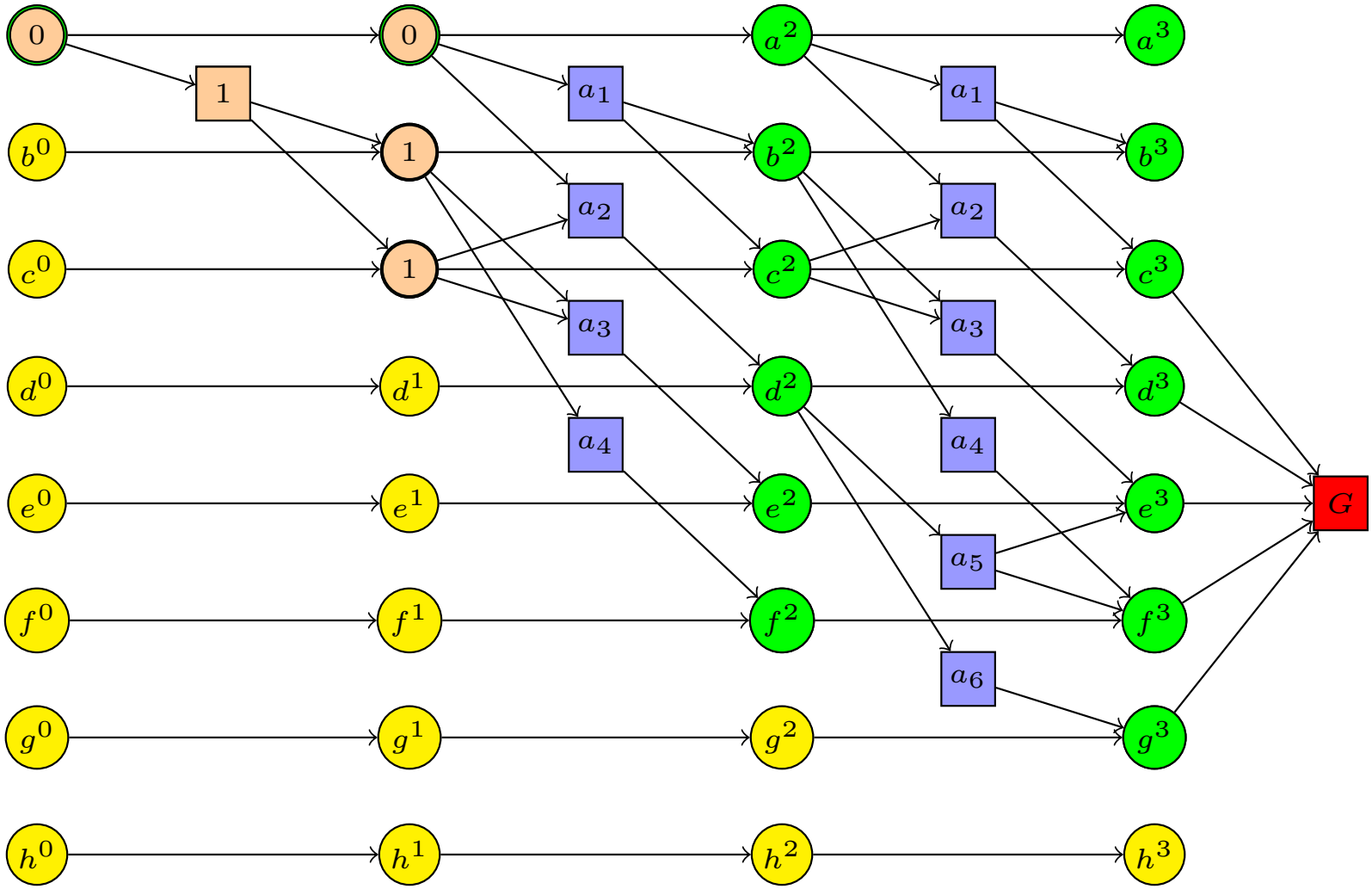
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{add}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

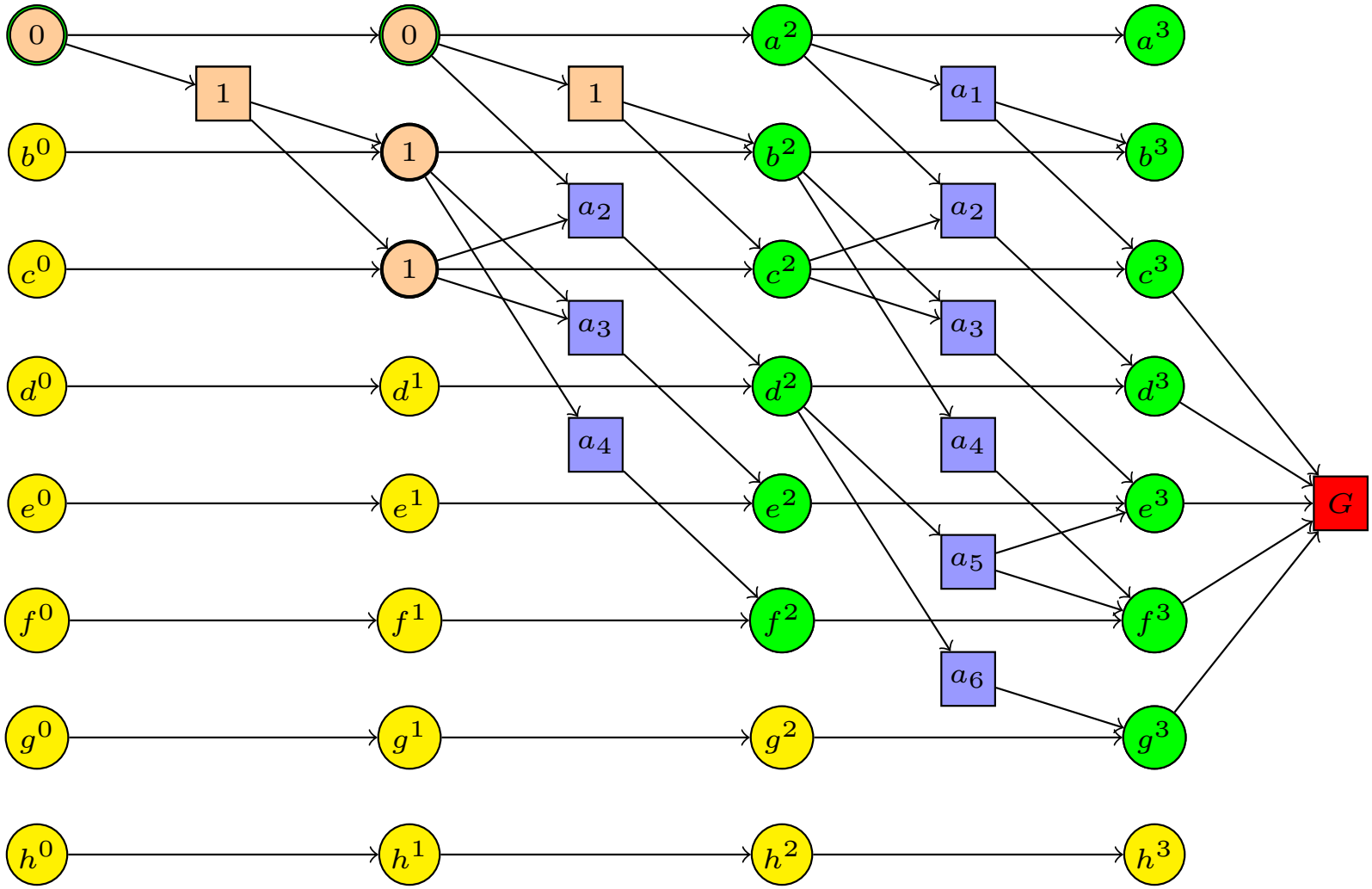
$h_{\max}$

h_{add}

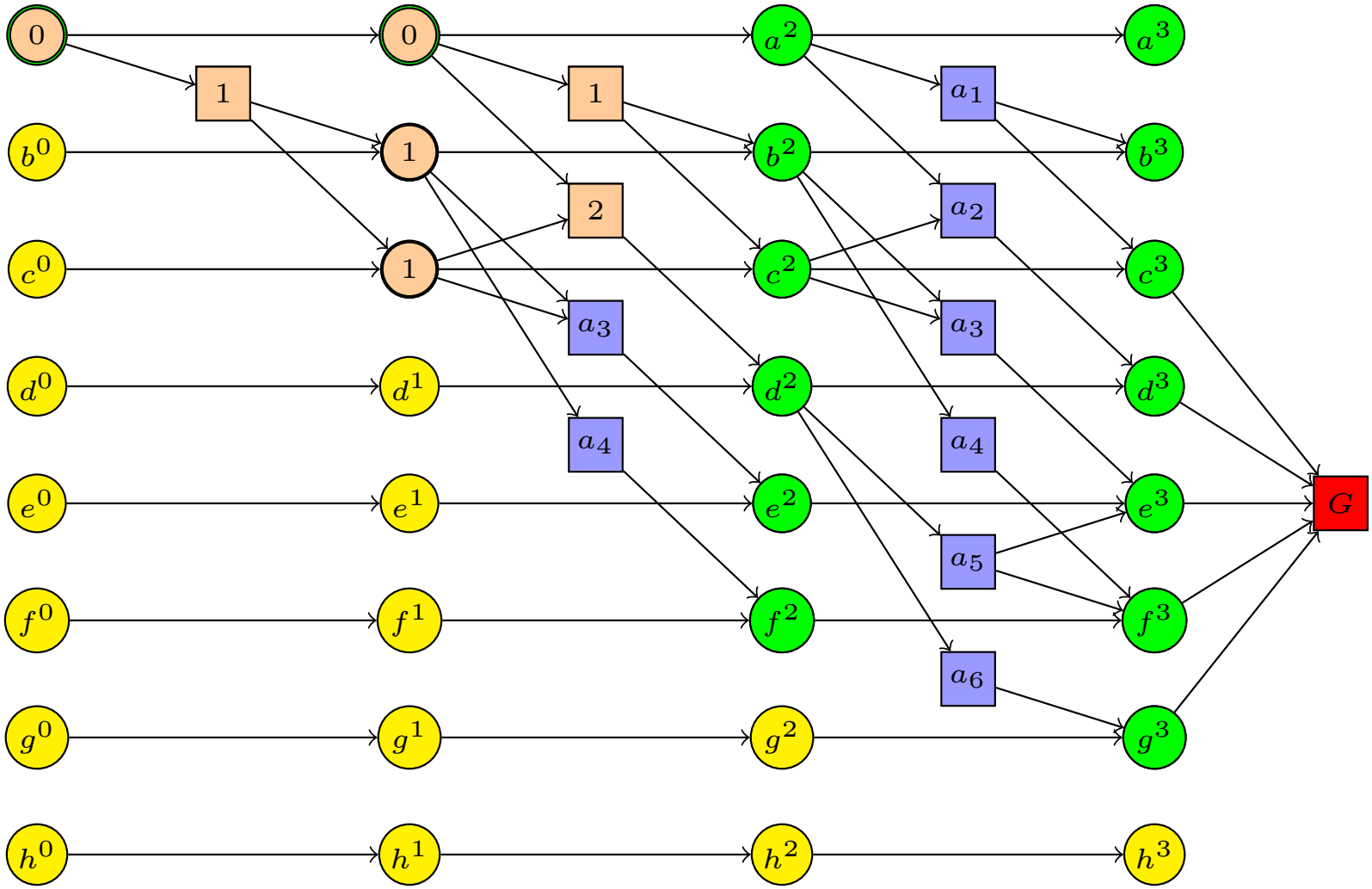
h_{FF}

Comparison & practice

Running example: h_{add}



Running example: h_{add}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

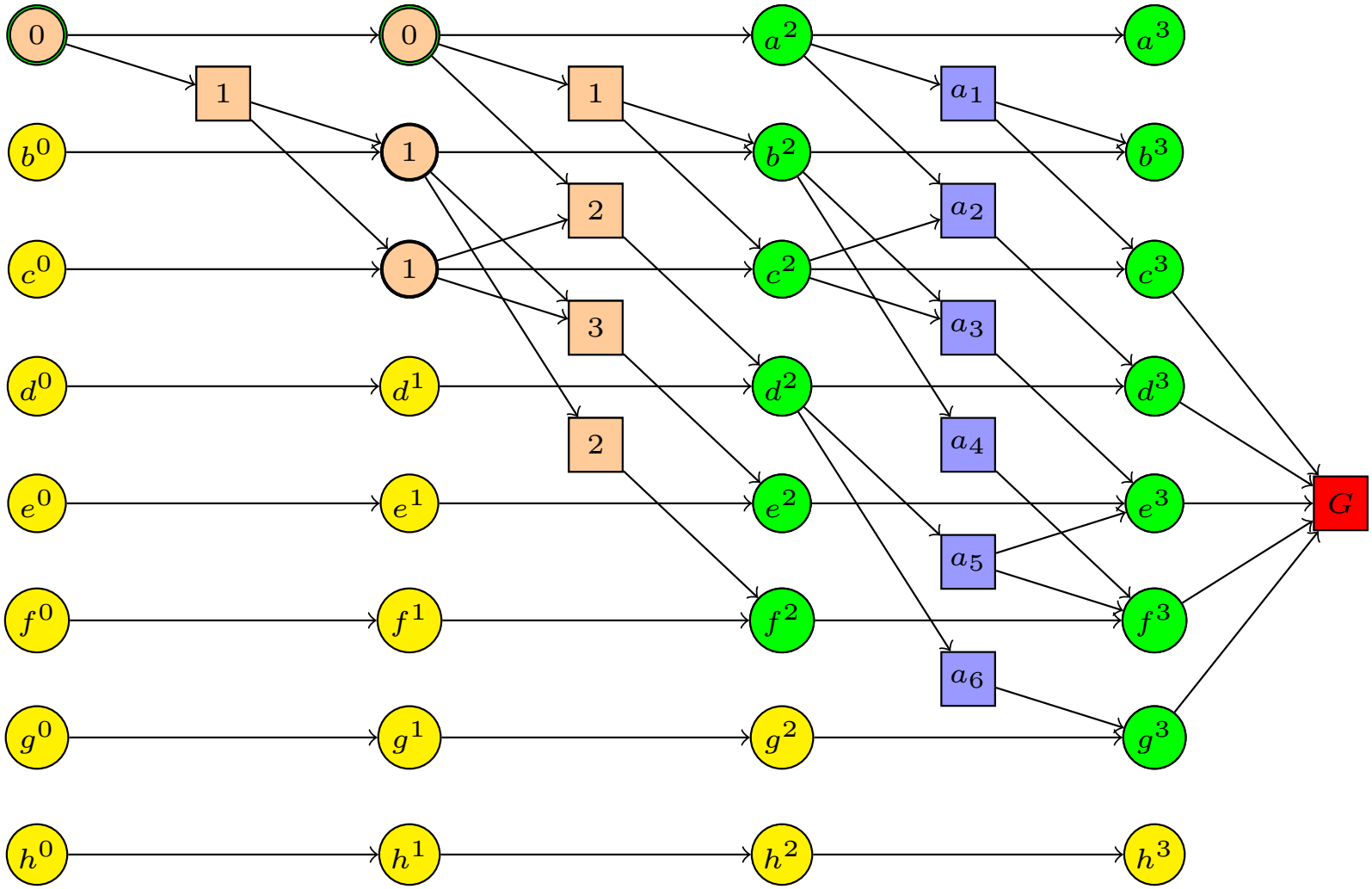
$h_{\max}$

h_{add}

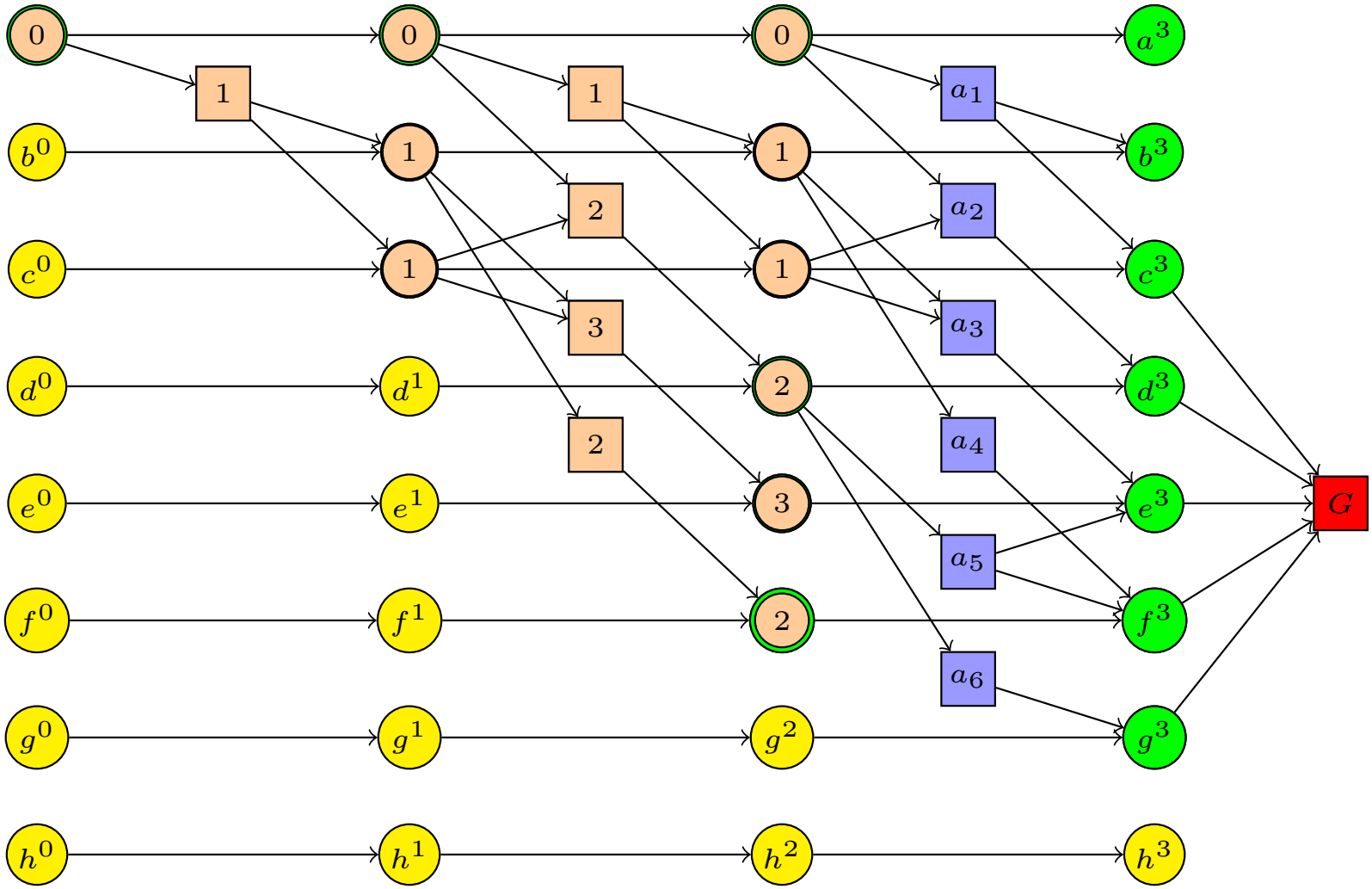
h_{FF}

Comparison & practice

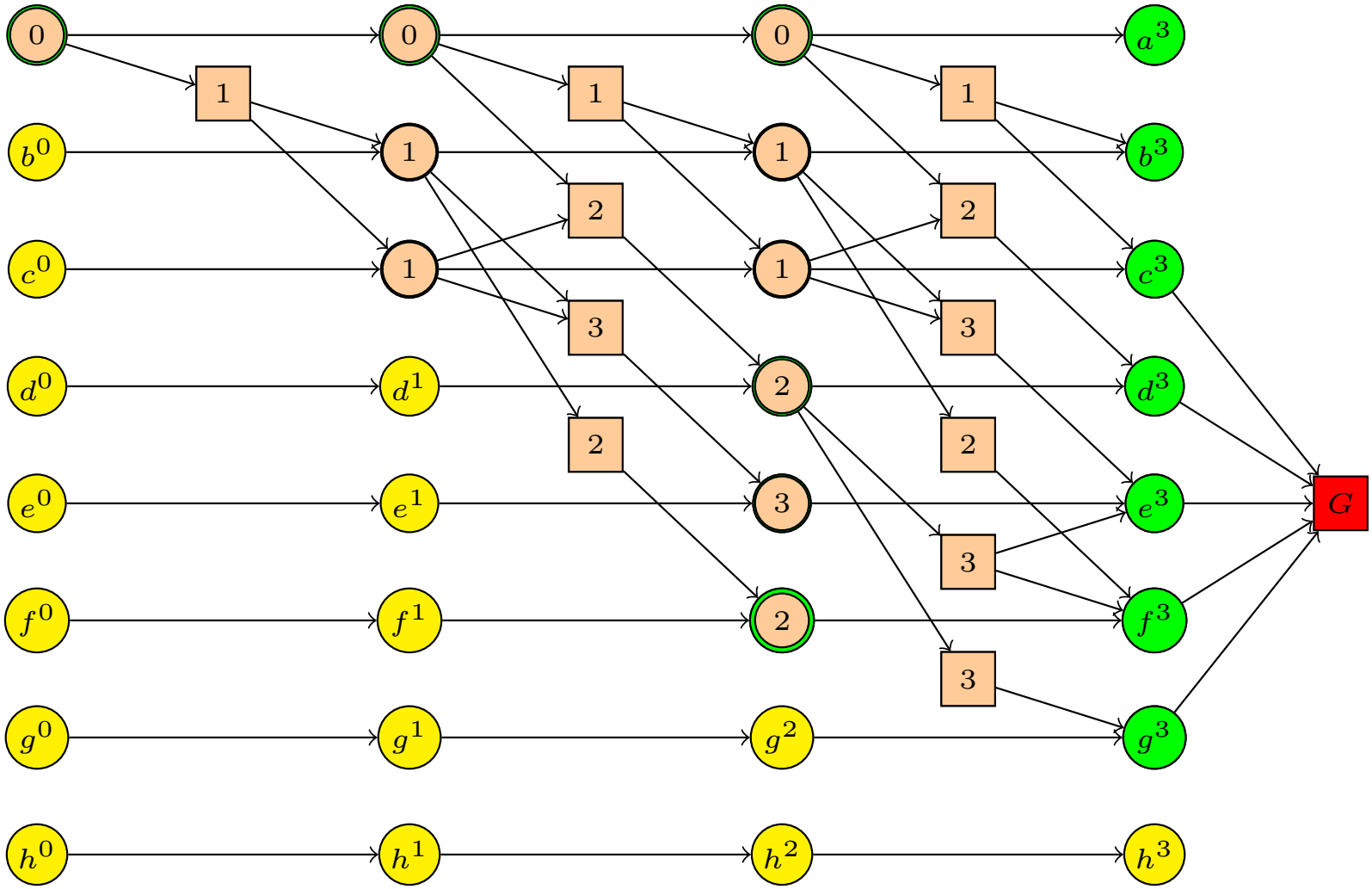
Running example: h_{add}



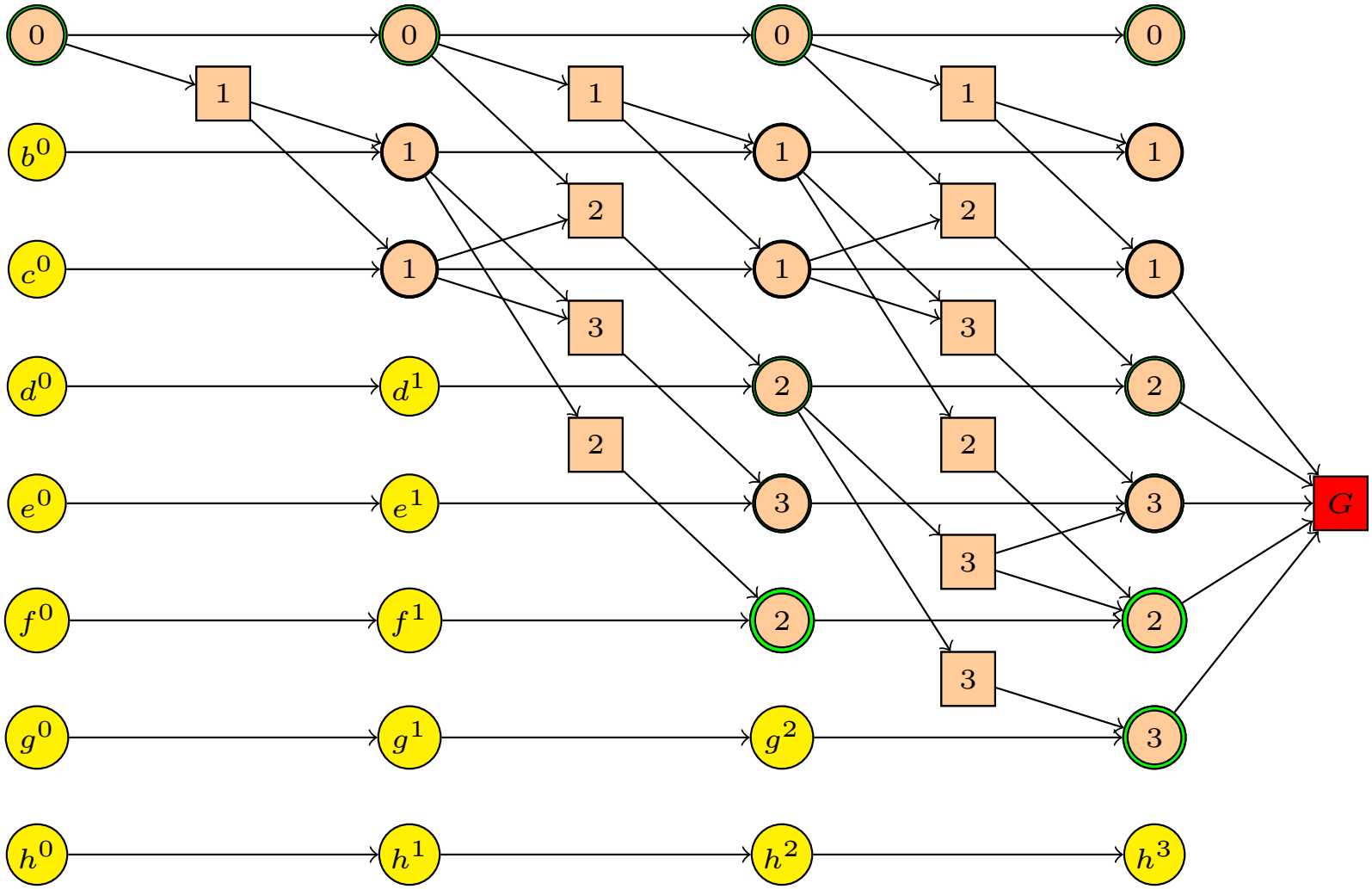
Running example: h_{add}



Running example: h_{add}



Running example: h_{add}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

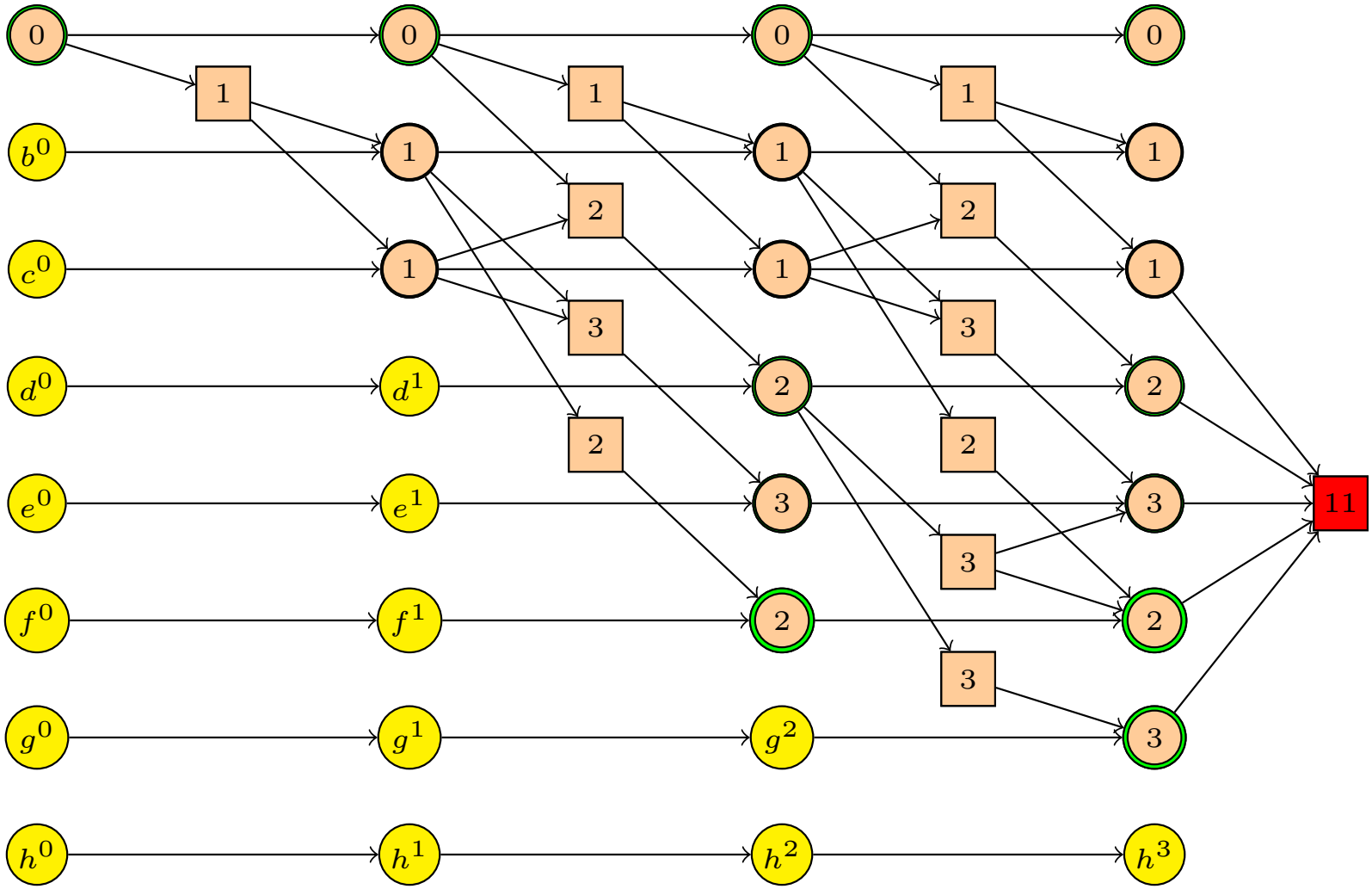
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{add}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Remarks on h_{add}

- h_{add} is safe and goal-aware.
- Unlike h_{max}, h_{add} is a very informative heuristic in many planning domains.
  Q: Intuitively, when it will be informative?
- The price for this is that it is not admissible (and hence also not consistent), so not suitable for optimal planning.
- In fact, it almost always overestimates the h^+ value because it does not take positive interactions into account.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
h_{max}
h_{add}
h_{FF}
Comparison & practice

FF heuristic: fitting the template

The FF heuristic h_{FF}

Computing annotations:

- Annotations are Boolean values, computed top-down.

  A node is marked when its annotation is set to 1 and unmarked if it is set to 0. Initially, the goal node is marked, and all other nodes are unmarked.

  We say that an action node is justified if all its true immediate predecessors are marked, and that a proposition node is justified if at least one of its immediate predecessors is marked.

  ...

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

FF heuristic: fitting the template (ctd.)

The FF heuristic h_{FF} (ctd.)

Computing annotations:

- ...

  Apply these rules until all marked nodes are justified:

  1. Mark all immediate predecessors of a marked unjustified ACTION node.
  2. Mark the immediate predecessor of a marked unjustified PROP node with only one immediate predecessor.
  3. Mark an immediate predecessor of a marked unjustified PROP node connected via an idle arc.
  4. Mark any immediate predecessor of a marked unjustified PROP node.

  The rules are given in priority order: earlier rules are preferred if applicable.

FF heuristic: fitting the template (ctd.)

The FF heuristic h_{FF} (ctd.)

Termination criterion:

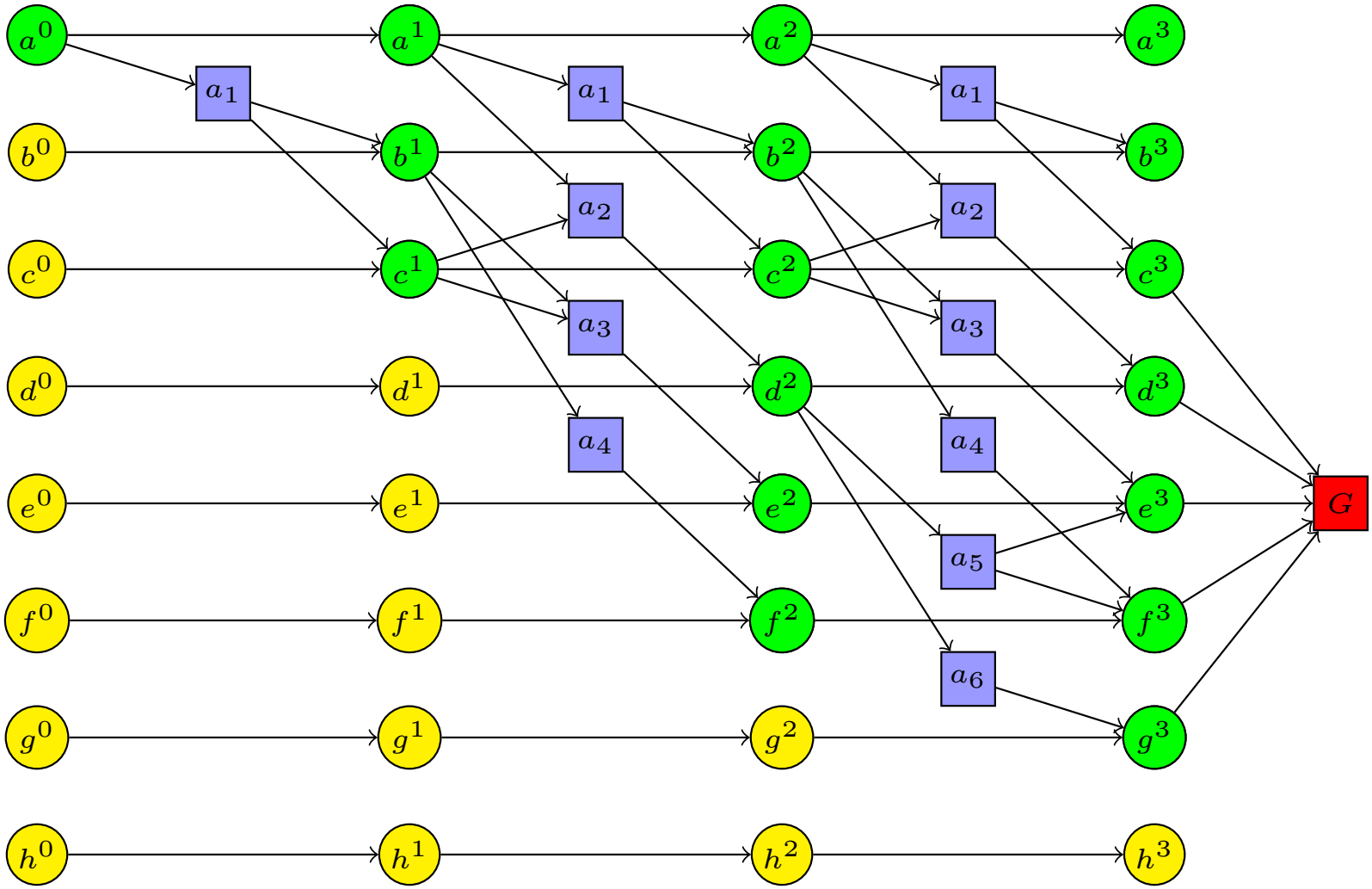
- Always terminate at first layer where goal node is true.

Heuristic value:

- The heuristic value is the number of marked action nodes.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

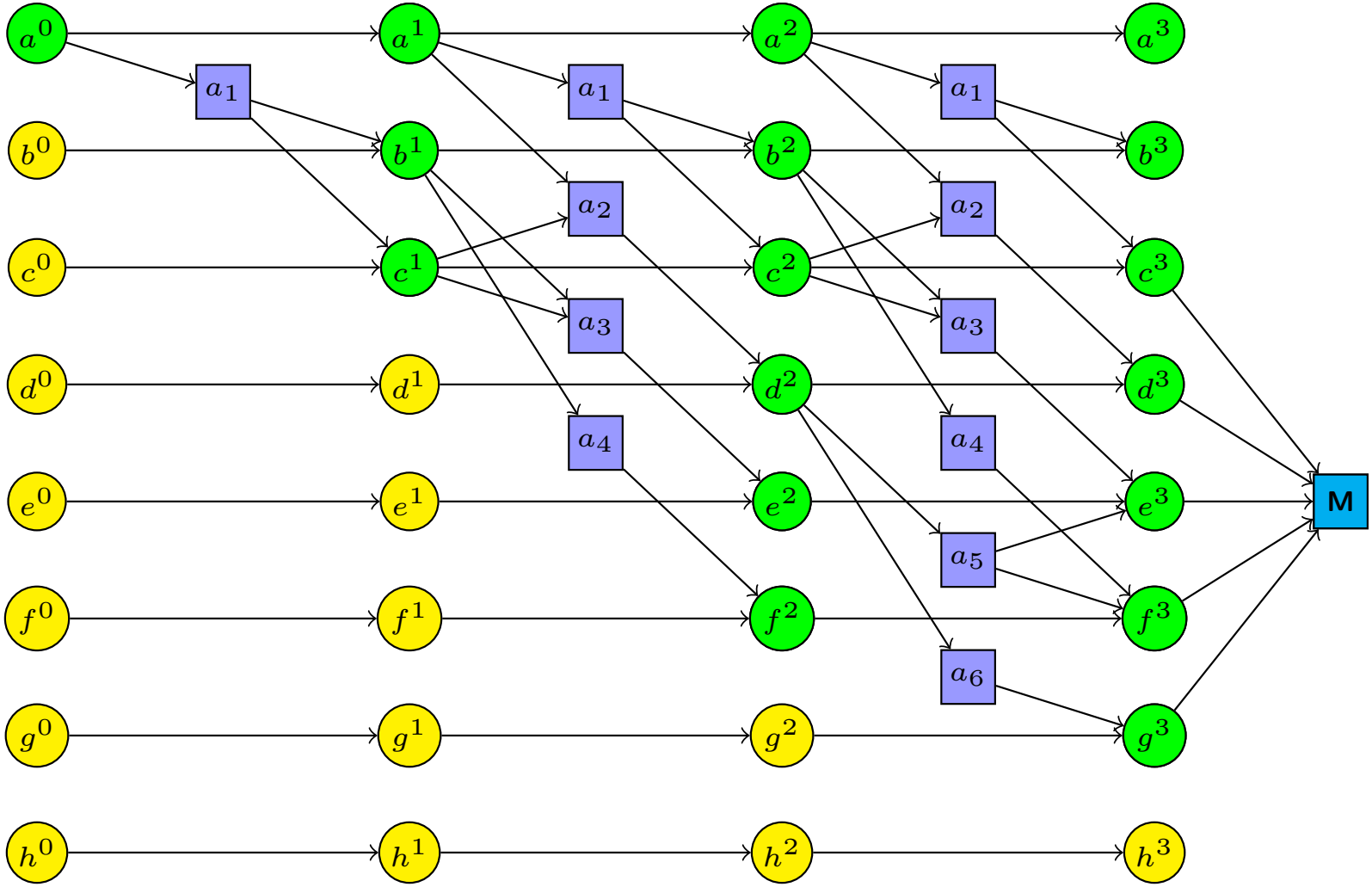
$h_{\max}$

h_{add}

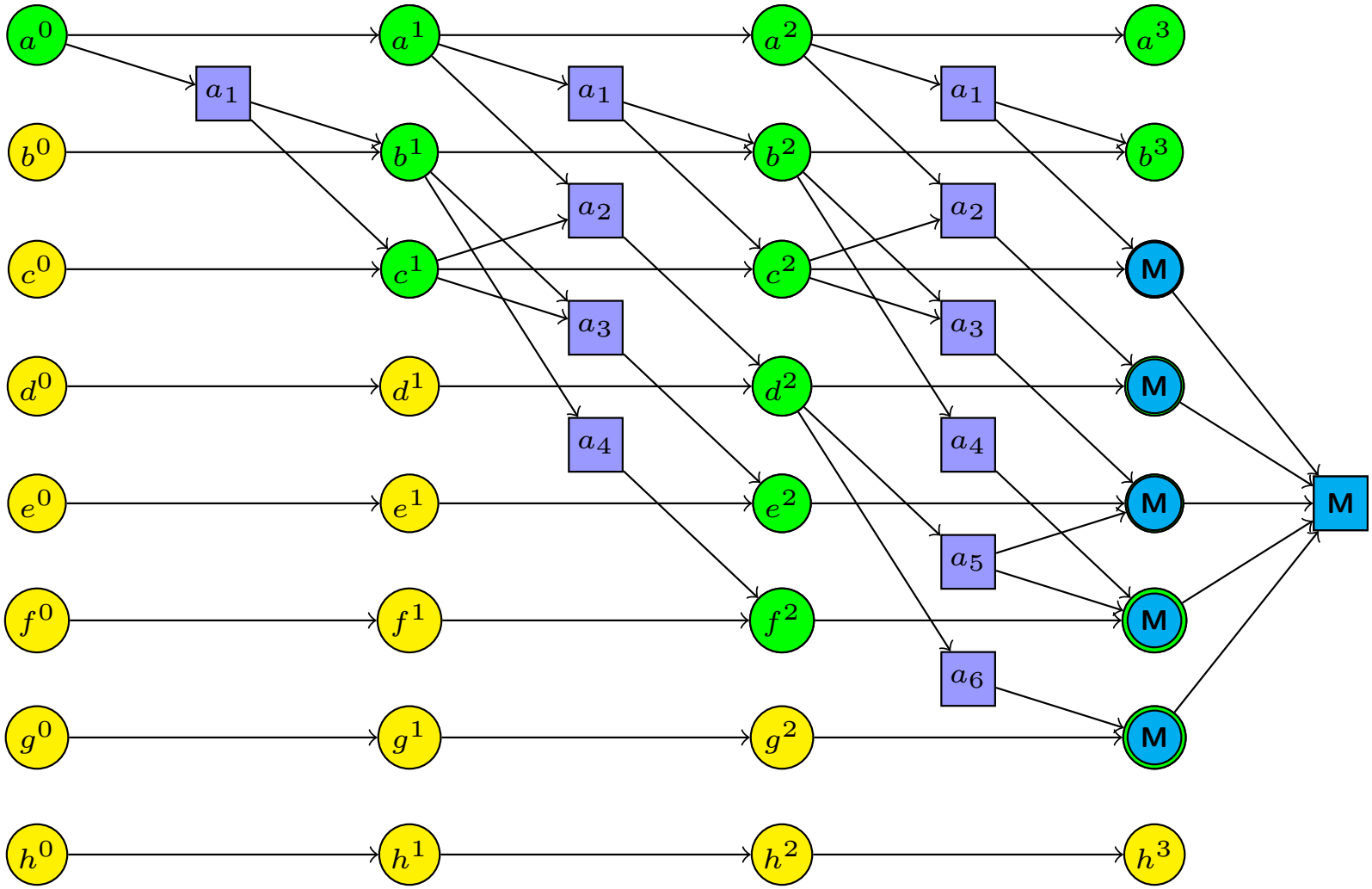
h_{FF}

Comparison & practice

Running example: h_{FF}



Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

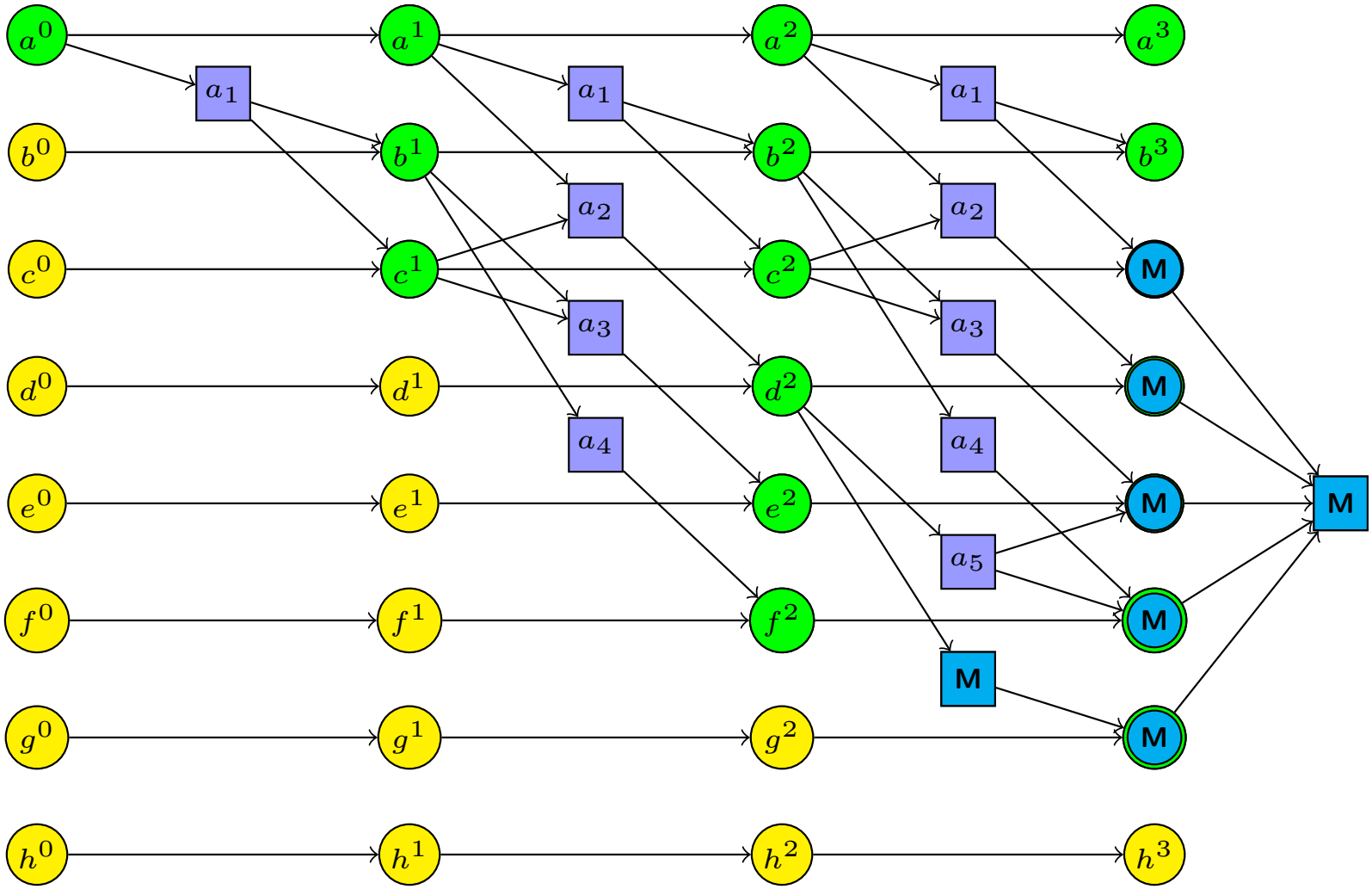
$h_{\max}$

h_{add}

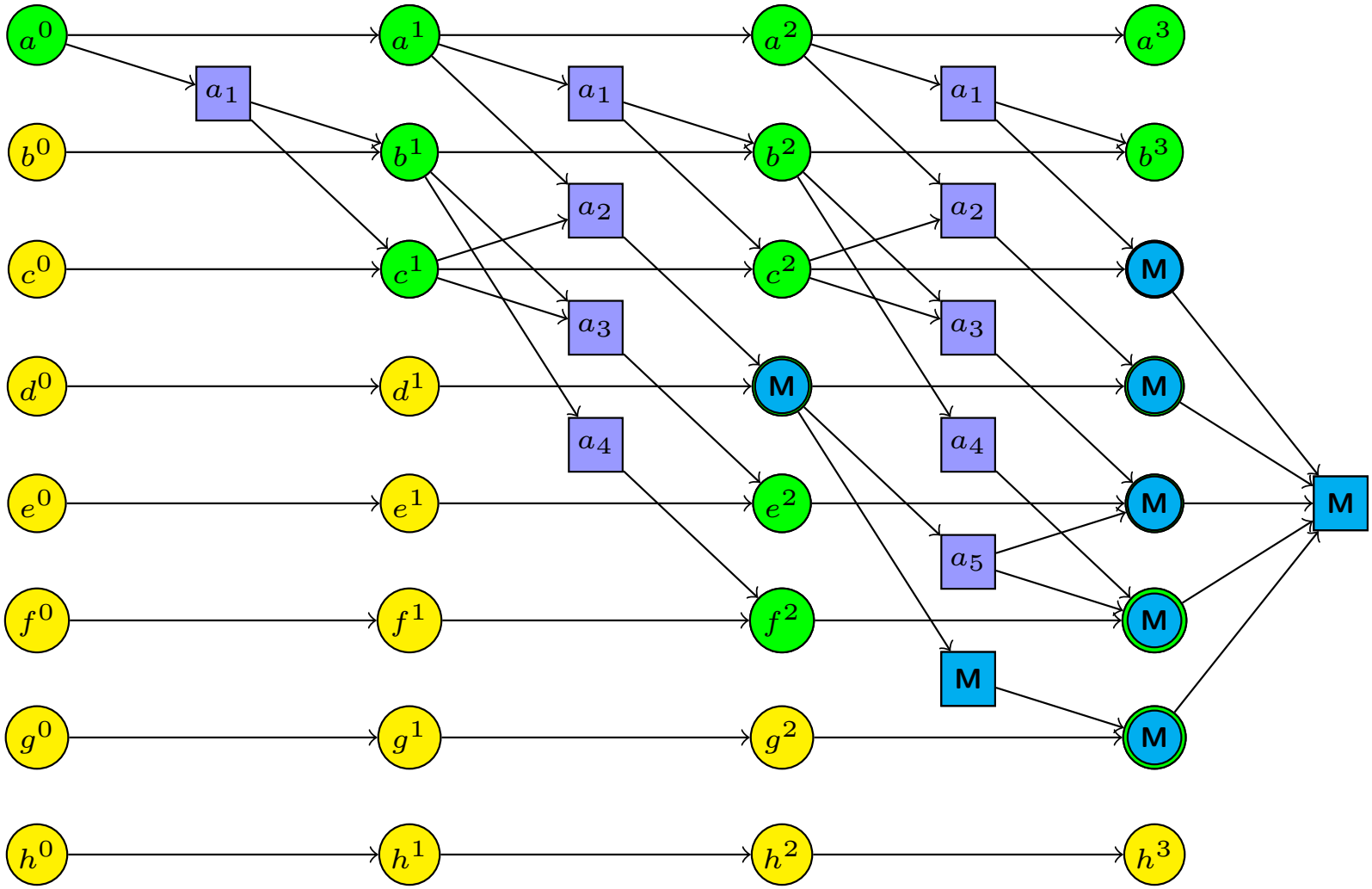
h_{FF}

Comparison & practice

Running example: h_{FF}



Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

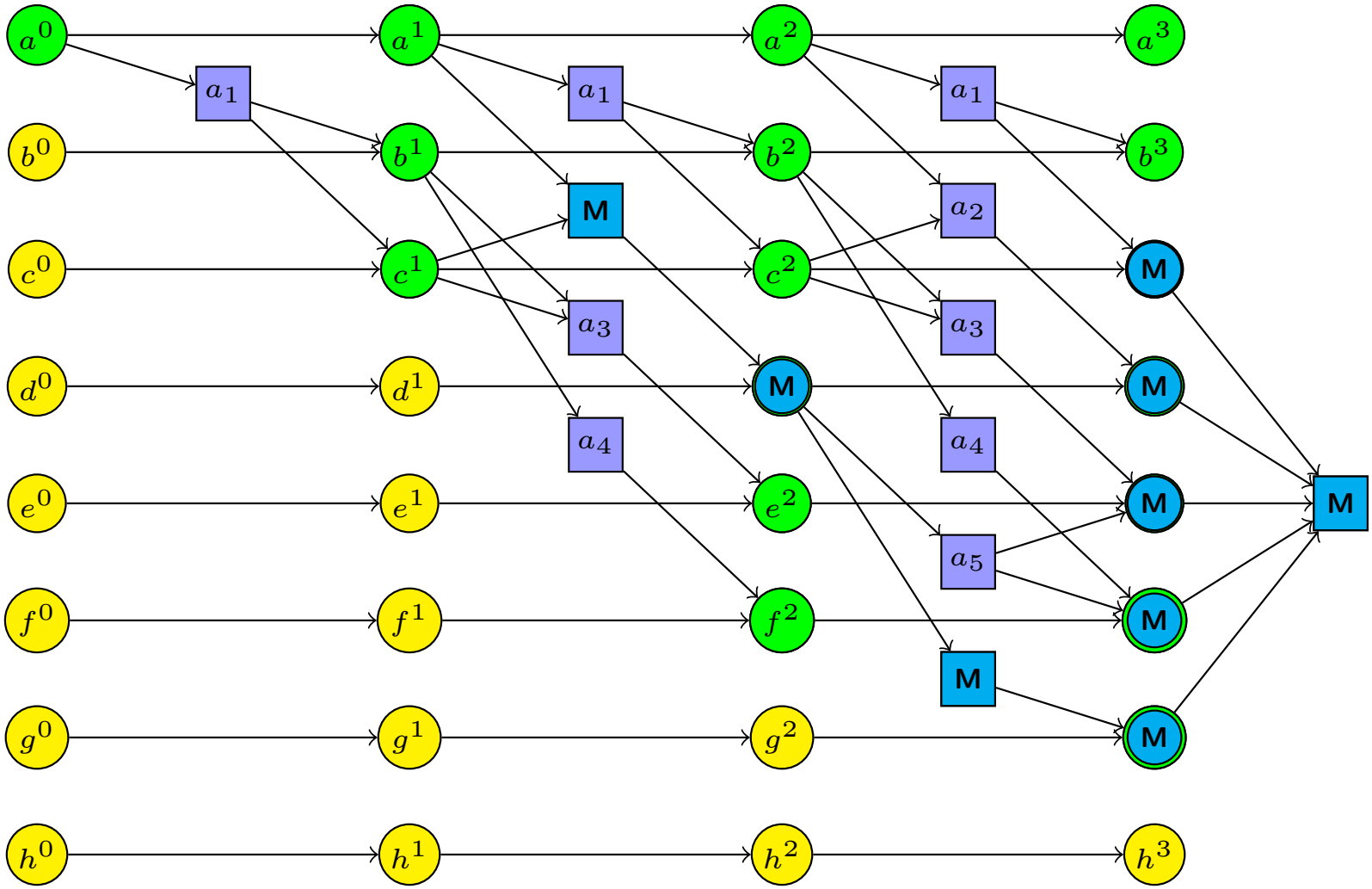
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

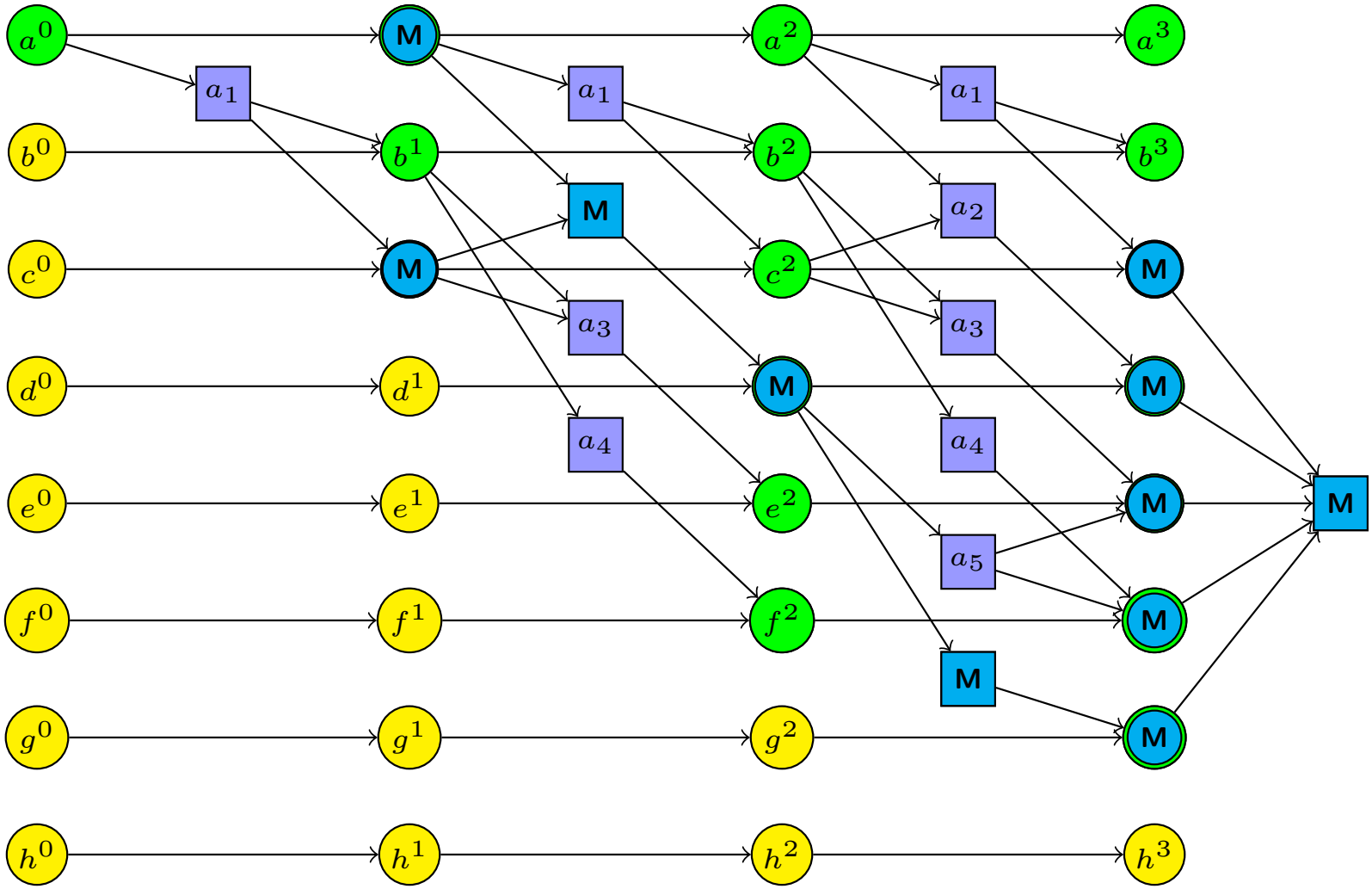
$h_{\max}$

h_{add}

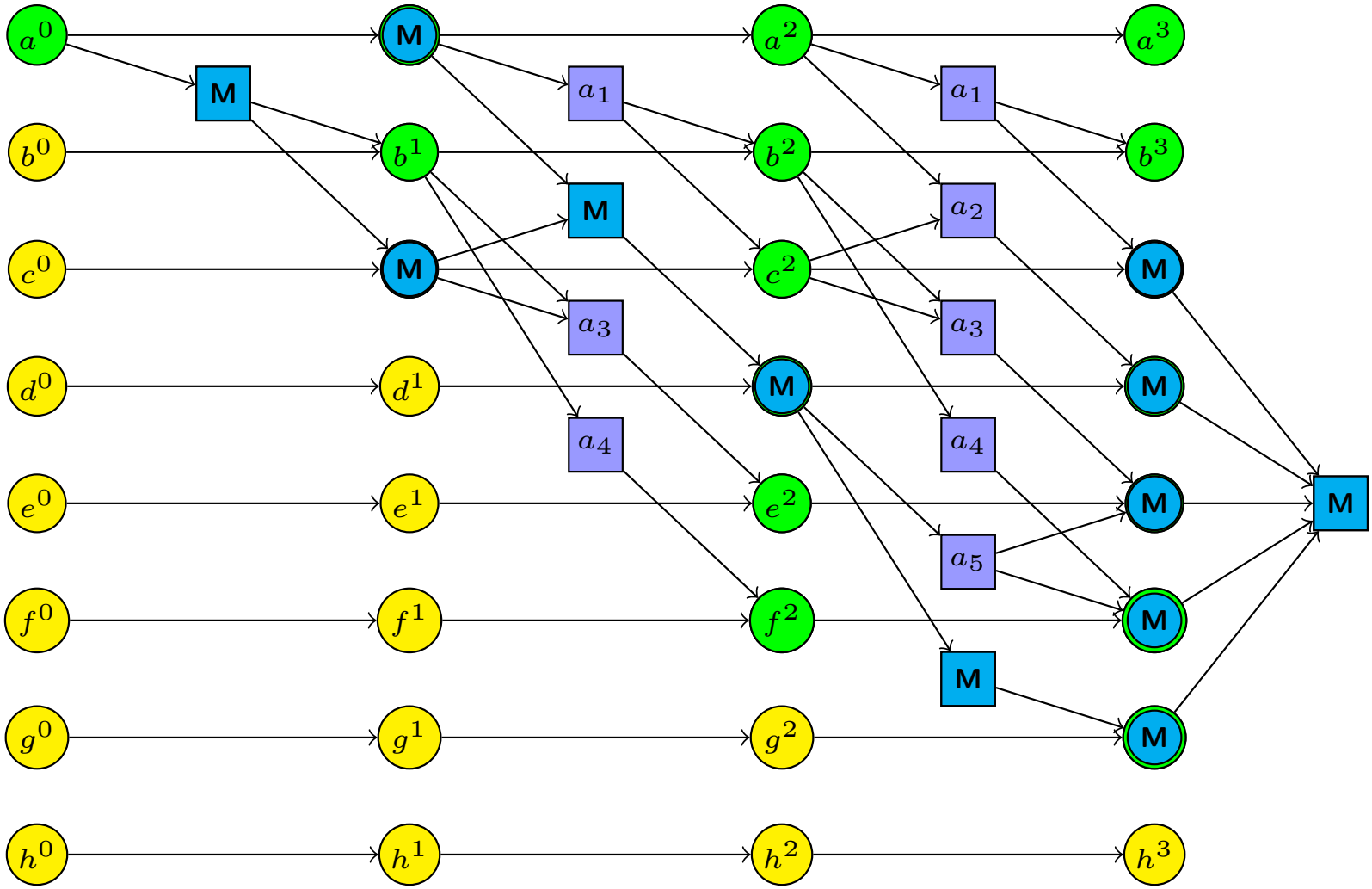
h_{FF}

Comparison & practice

Running example: h_{FF}



Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

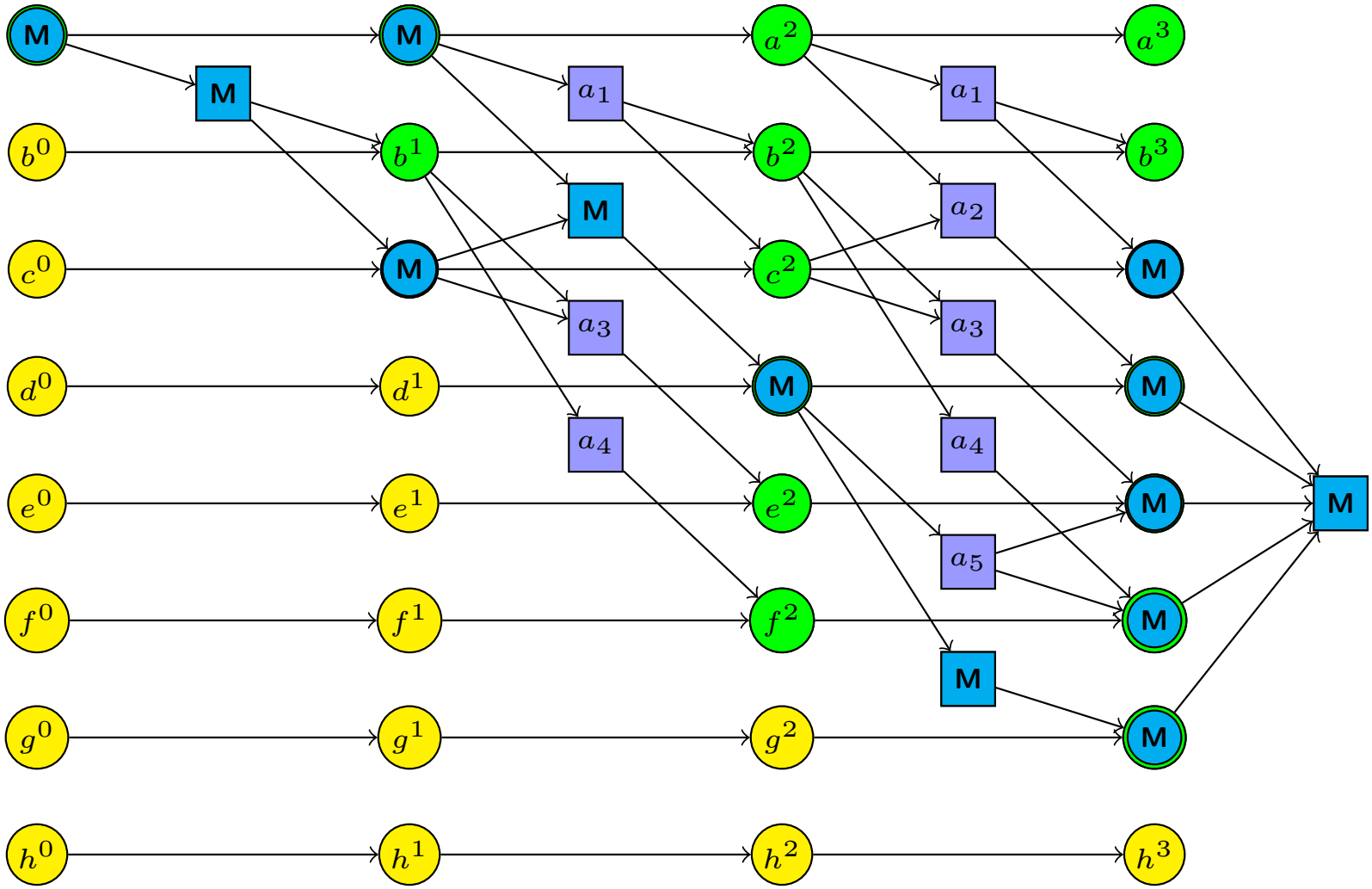
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

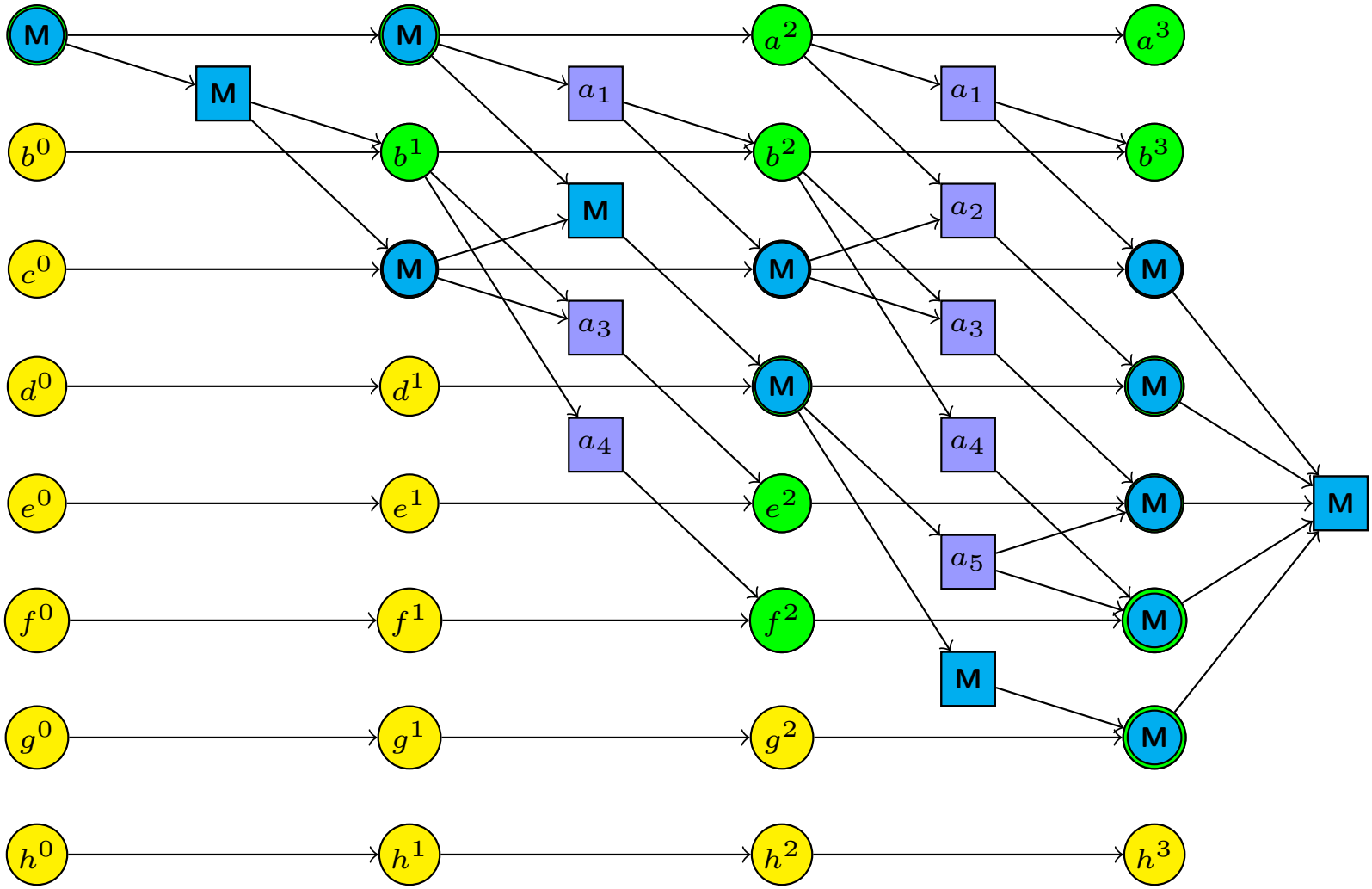
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

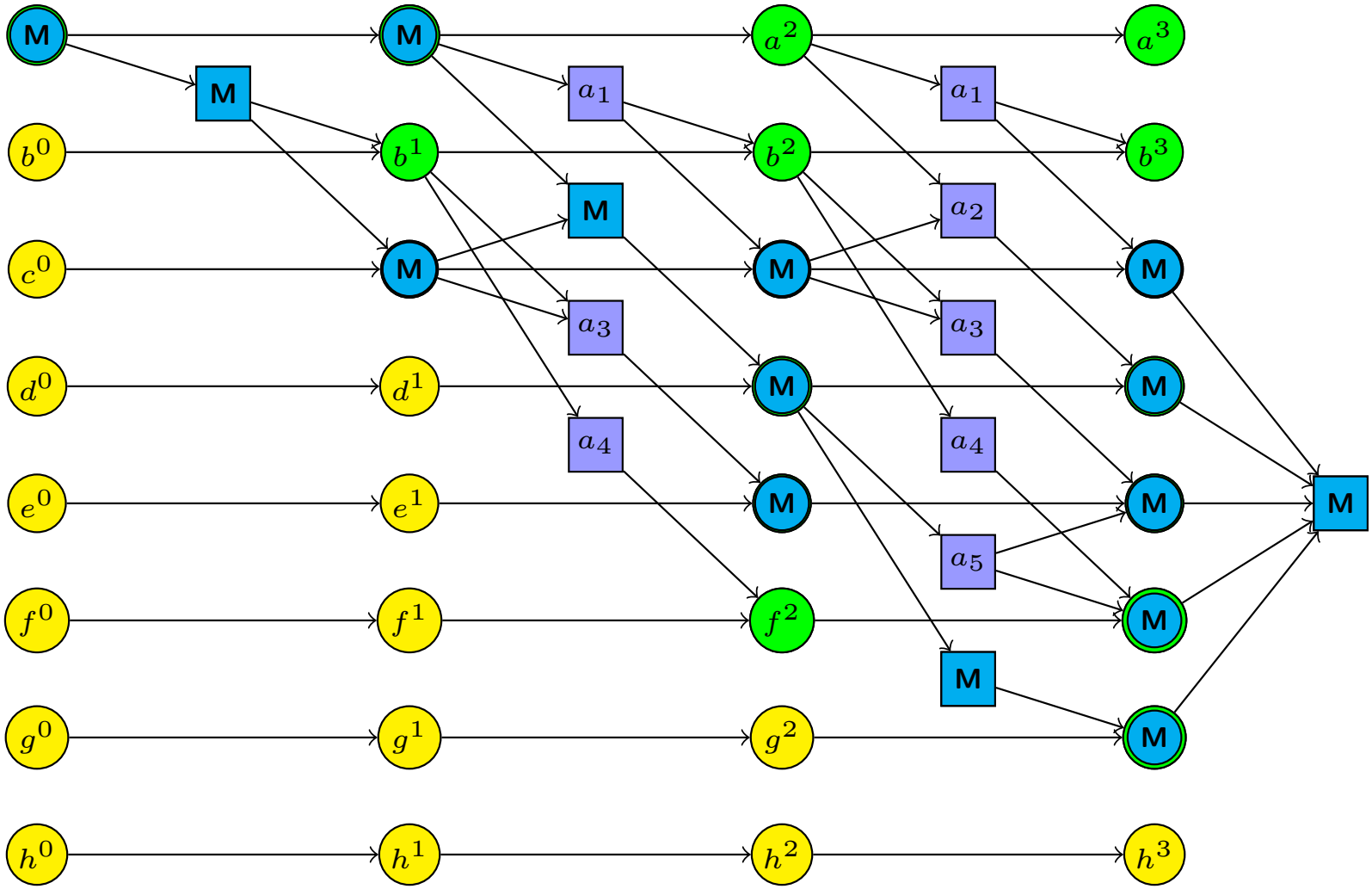
$h_{\max}$

h_{add}

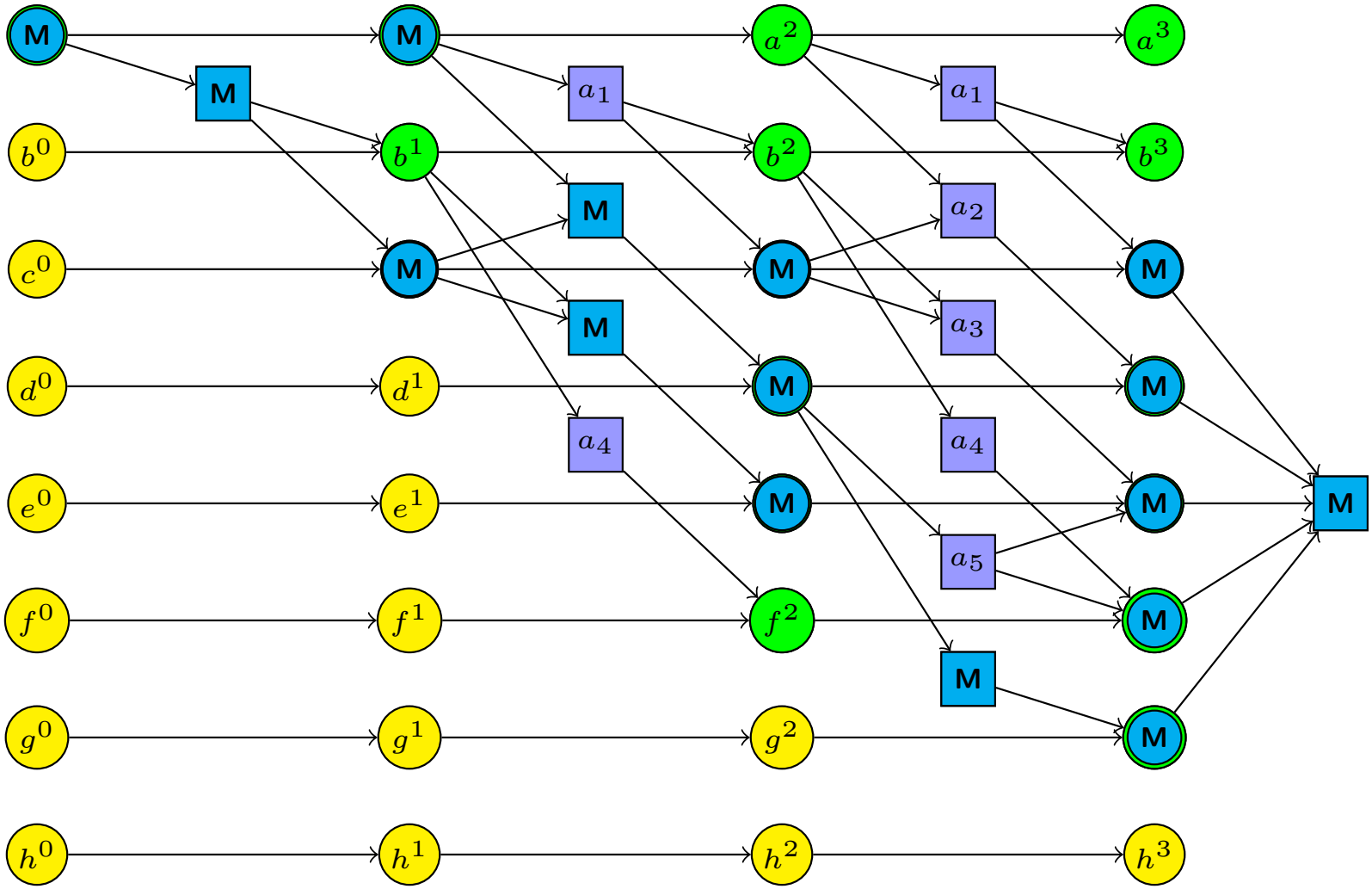
h_{FF}

Comparison & practice

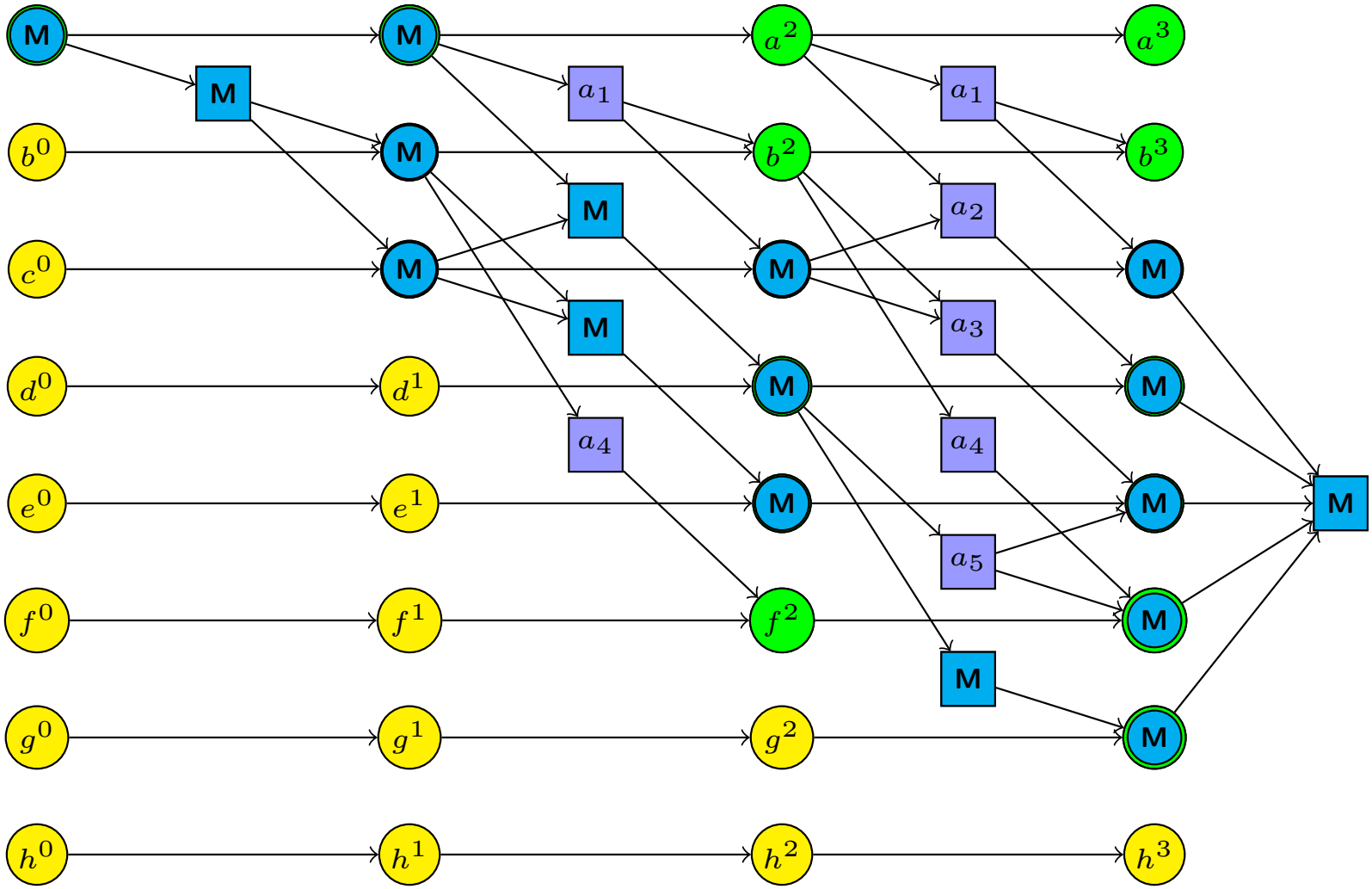
Running example: h_{FF}



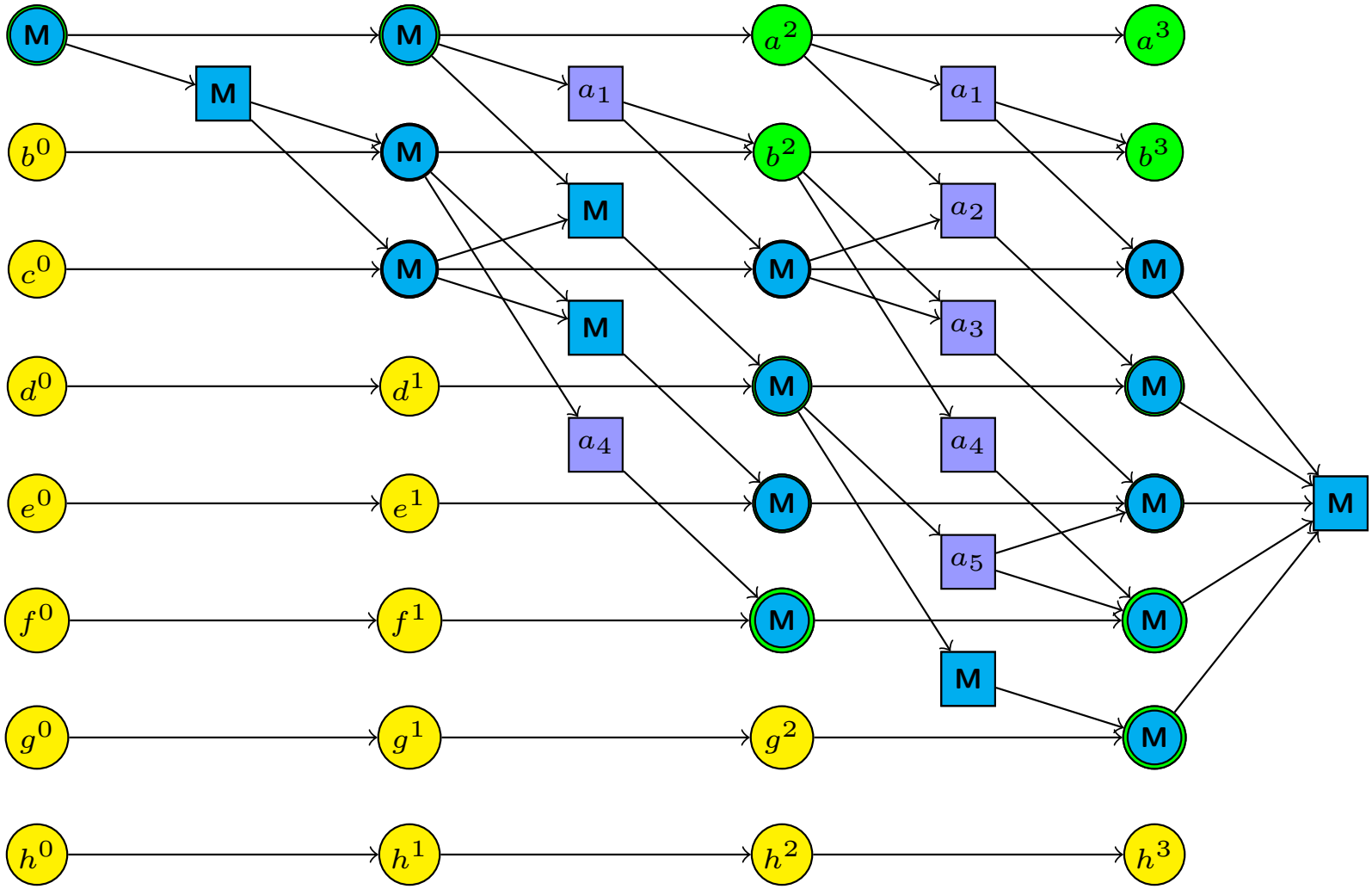
Running example: h_{FF}



Running example: h_{FF}



Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

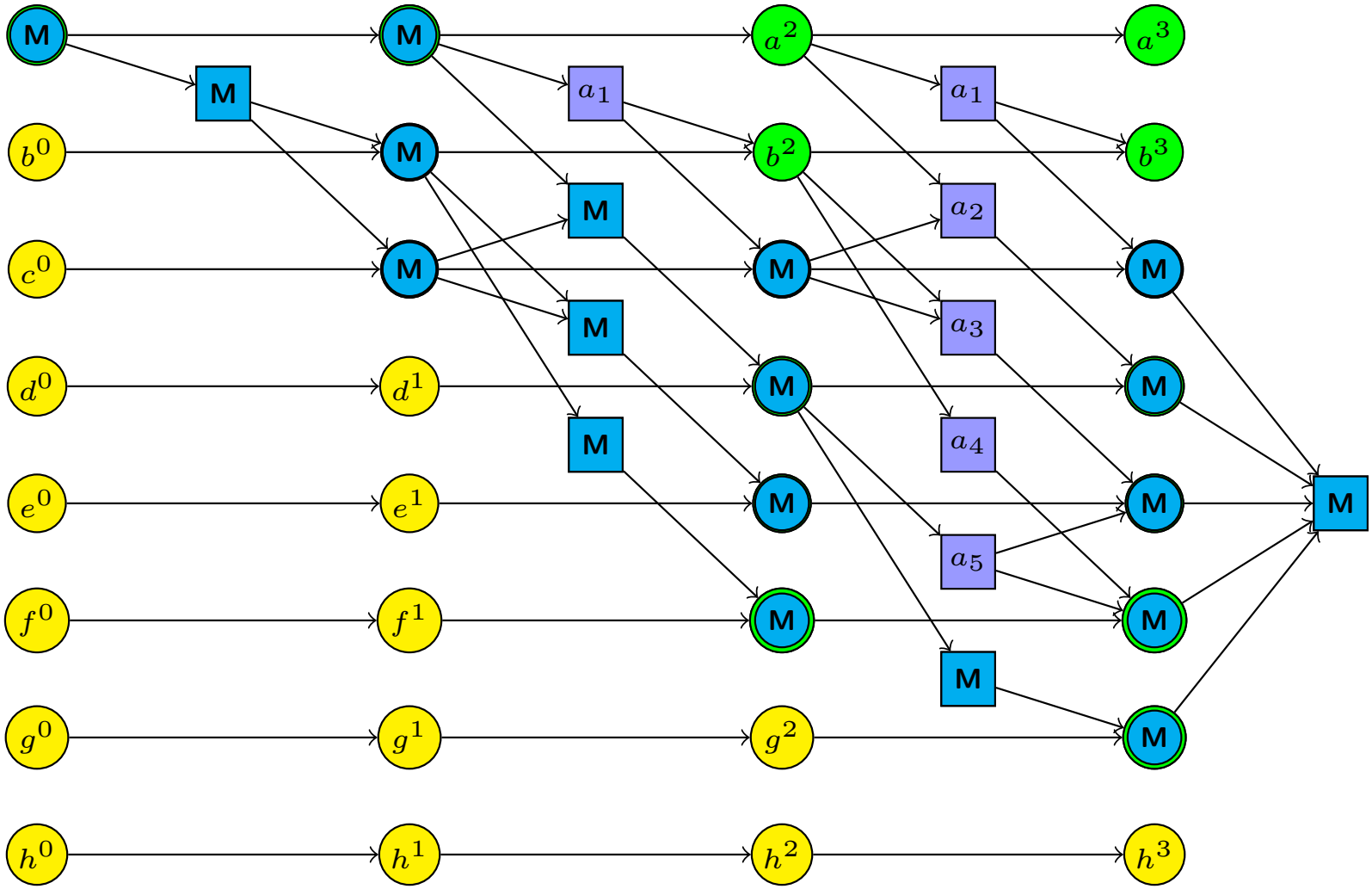
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{FF}



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

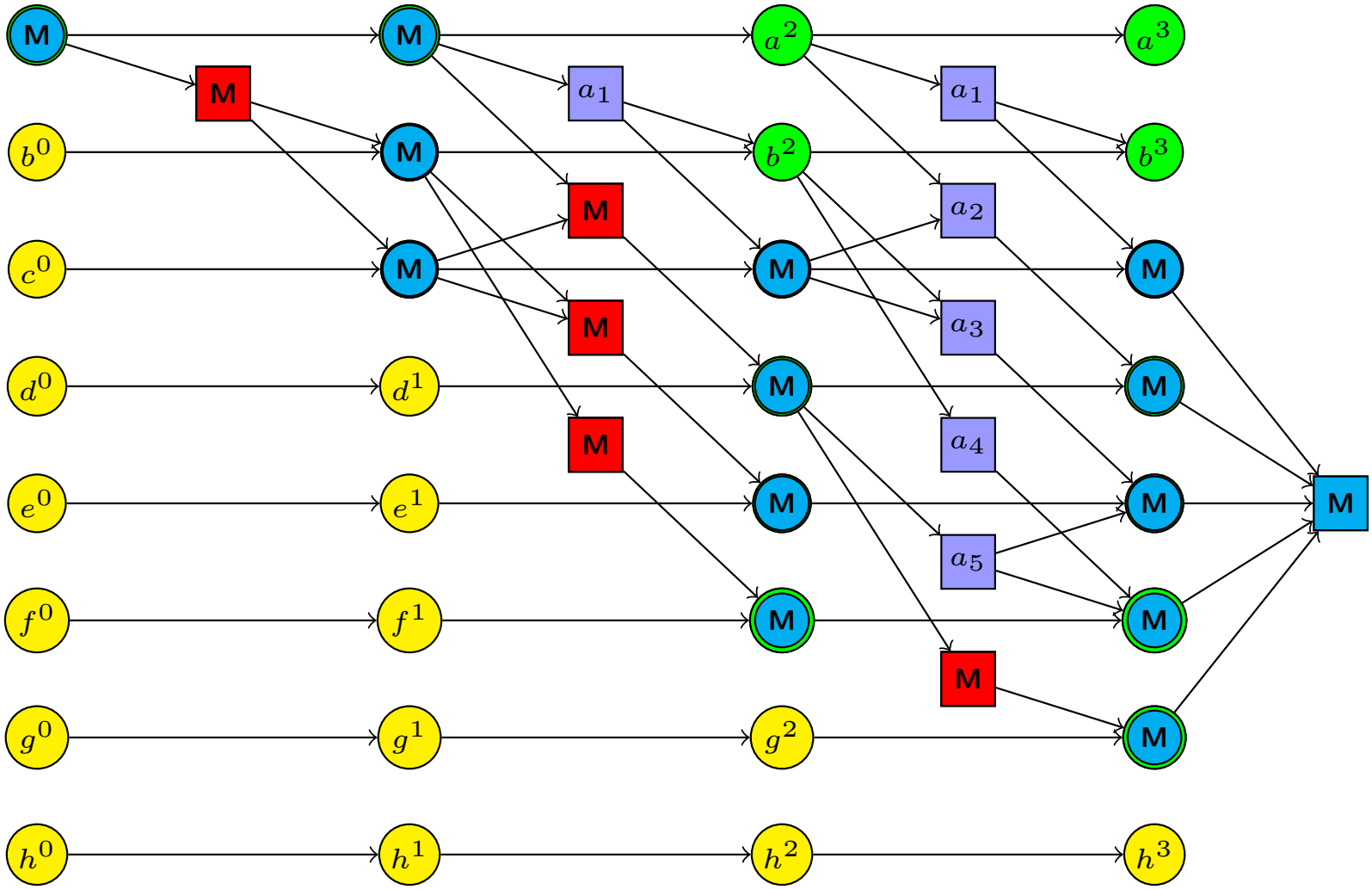
$h_{\max}$

h_{add}

h_{FF}

Comparison & practice

Running example: h_{FF}



Remarks on h_{FF}

- Like h_{add}, h_{FF} is safe and goal-aware, but neither admissible nor consistent.
- Always more accurate than h_{add} with respect to h^+.
  - Marked actions define a relaxed plan.
- h_{FF} can be computed in linear time.
  - The h_{FF} value depends on tie-breaking when the marking rules allow several possible choices, so h_{FF} is not well-defined without specifying the tie-breaking rule.
  - The best implementations of FF use additional rules of thumb to try to reduce the size of the generated relaxed plan.

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Problem specification
STRIPS language
Obtaining heuristics
Concrete Heuristics
Template
$h_{\max}$
h_{add}
h_{FF}
Comparison & practice

Comparison of relaxation heuristics

Relationship between relaxation heuristics

Let s be a state of planning task $\langle P, I, O, G \rangle$. Then:

- $h_{\mathsf{max}}(s) \leq h^+(s) \leq h^*(s)$
- $h_{\mathsf{max}}(s) \leq h^+(s) \leq h_{\mathsf{FF}}(s) \leq h_{\mathsf{add}}(s)$
- h^* and h_{FF} are pairwise incomparable
- h^* and h_{add} are incomparable

Moreover, h^+, h_{max}, h_{add}, and h_{FF} assign ∞ to the same set of states.

Note: For inadmissible heuristics, dominance is in general neither desirable nor undesirable. For relaxation heuristics, the objective is usually to get as close to h^+ as possible.

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Problem specification

STRIPS language

Obtaining heuristics

Concrete Heuristics

Template

h_{max}

h_{add}

h_{FF}

Comparison & practice